Final Study Guide: AI-Powered Commodity Price Predictor

Phase 2 (final phase): complete project study material and complete VS Code source-code guide for the supervisor presentation and viva

Document Type: Final FYP study material and viva preparation guide (two parts)

Date: October 9, 2026

Project: AI-Powered Commodity Price Predictor: next-day return forecasting for Bitcoin, Gold and Silver

Repository: `crypto-forex-prediction-system`, branch `main`, commit `83323b4` (clean working tree), 45 commits, 2026-01-31 to 2026-10-04

Team: teamlocalhost, BSCS, University of Lahore (Alyan Shahid, Furqan Abid, Muhammad Huzaifa)

Structure: Part 1: Complete Project Study Material (Chapters 1 to 15). Part 2: Complete VS Code Folder, File and Source Code Explanation (Chapters 16 to 23). Then Glossary and Appendices

Source of truth: The code, data files and result files in the repository, checked while writing

---

The project at a glance

Assets	Bitcoin (`BTC-USD`, spot, 7 days a week), Gold futures (`GC=F`) and Silver futures (`SI=F`), both COMEX front-month, exchange days only
What is predicted	The next trading day's **log return** $y = \ln(P_{t+1}/P_t)$, shown to the user as a price, an UP / DOWN call and a 68 % band
Data	Yahoo Finance daily bars (via `yfinance`), five macro series (DXY, WTI, VIX, 10-year yield, S&P 500) and the Fear & Greed index (alternative.me); frozen evaluation data 2018-01-01 to 2026-09-12; Bitcoin 3,177 raw rows, Gold and Silver 2,186 rows each
Features	29 (Bitcoin), 28 (Gold), 29 (Silver): returns, momentum, volatility, RSI / ADX / ROC, Bollinger, ATR, MACD and EMA ratios, macro returns, sentiment. All stationary and backward-looking
Models	Six trained: Ridge, Random Forest, LightGBM, CatBoost, GRU, LSTM. Baselines: Naive (random walk), Naive-Mean (drift), ARIMA. Experiment: Stacked ensemble
Served forecast	**Combined** = equal-weight mean of the six predicted log returns (no fitted weights)
Split	Train to 2025-09-10 · validation to 2026-02-17 · test 2026-02-18 to 2026-09-12 (Bitcoin) / 2026-09-11 (metals). Samples (train / val / test): Bitcoin 2750 / 160 / 207, Gold and Silver 1874 / 109 / 143. Tuning and model comparison: 4-fold expanding walk-forward inside train+validation
Interfaces	Streamlit dashboard (4 tabs) and FastAPI (`/health`, `/models`, `/predict/{asset}`)
Quality	47 automated tests pass (re-run on 2026-10-09), GitHub Actions CI, fixed seeds, full pipeline reproduces to 1e-13 (audit of 2026-10-04)
Verdict	A correct, honest, reproducible forecasting **system** whose models have **no skill over the random walk**; its calibrated uncertainty band is the useful output

Executive Summary

Our project is a machine-learning system that studies the daily prices of **Bitcoin, Gold and Silver**, learns from about eight years of history, and forecasts the **next trading day's closing price**. It downloads prices and market indicators from Yahoo Finance and the Fear & Greed index, turns them into 28 or 29 numbers per day, trains **six machine-learning models** (Ridge regression, Random Forest, LightGBM, CatBoost, GRU and LSTM) to predict tomorrow's log return, averages the six predictions into one **Combined forecast**, and shows it in a **Streamlit dashboard** and a **FastAPI** endpoint together with a calibrated 68 % uncertainty band and its own measured test results.

The honest headline result. On days that no model had seen (143 to 207 days of 2026) and on about six years of walk-forward predictions, **no model, and not the Combined forecast, predicts tomorrow's price better than the baseline "tomorrow = today"** (the random walk). On the untouched test set the Combined forecast's return error differs from the baseline by -0.34 % (Bitcoin), -0.04 % (Gold) and -0.29 % (Silver), with Diebold-Mariano p-values of 0.49, 0.97 and 0.46 (none significant). This is a rigorously measured negative result that agrees with efficient-market theory. It is not a bug, and the project is built so that the result can be trusted.

What the data does support is the *size* of tomorrow's move. The served 68 % band, $P_t \cdot \exp(\hat{y} \pm k \cdot \sigma_t)$, contained the actual close on 65.7 %, 69.2 % and 69.2 % of the test days (Bitcoin, Gold, Silver; nominal 68.3 %).

> **Short answer to "does it work?"**
> It works as a **correct, leakage-audited, reproducible forecasting system**; it does **not** work as a way to beat the market. Never say "98 % accurate": a price error of about 1.3 % is exactly what "tomorrow = today" also scores. Say instead: "we built a fair test of whether ML can forecast next-day returns, and the answer is no, with evidence."

How to Use This Guide

- **Part 1 (Chapters 1 to 15)** teaches the project as a machine-learning system: problem, workflow, technologies, data, target, features, ML basics, every model, validation, results, ensemble, dashboard, limits, reliability, summary.
- **Part 2 (Chapters 16 to 23)** is for the moment an examiner opens VS Code: the real folder tree, every source file, a function-by-function table, how the files connect, the code you must understand, how to run everything, one prediction traced through the code, and likely code questions.
- Each topic gives **simple English first**, then the technical explanation. Boxes marked "Short answer" are meant to be said aloud. Amber boxes mark honest caveats.
- Every number comes from the repository (`results/*.csv`, `data/models/model_status.json`, the processed data files) or from a run of the project's own code during preparation; Appendix B lists exactly what was checked.
- **Two-hour revision path:** Chapter 1, 2, 5, 9, 10, 15, then Chapter 22 and 23. **One-hour path:** Chapter 15, Chapter 10 tables, Chapter 23.

Notation used throughout

Symbol	Meaning	Where it lives in the code
P_t	Closing price of day t (Bitcoin: 00:00 UTC bar close; Gold and Silver: 13:30 ET COMEX settlement)	`price` column
$y_t = \ln(P_{t+1} / P_t)$	The **target**: the next-day log return, indexed by the day t on which the forecast is made	`_task_target`, `build_dataset`
$\hat{y}_t$	The predicted log return (after converting back from the standardised model output)	`_predict_one`
z_t	The standardised target the models are trained on: $(y - \mu_{train}) / \sigma_{train}$	`data["fwd"]`, `data["inv"]`
σ_t	RiskMetrics EWMA volatility ($\lambda = 0.94$) of returns up to day t	`ewma_vol` feature
k	Band multiplier calibrated on training-period walk-forward errors (0.825 / 0.909 / 0.870)	`model_status.json: band.k`
Random walk / naive	The forecast $\hat{y} = 0$, i.e. "tomorrow's price = today's price"	`NaiveModel`
DM, DA, MAE, RMSE, MAPE	Diebold-Mariano test, directional accuracy, and error metrics (Chapter 10)	`src/utils/metrics.py`

Table of Contents

PART 1: Complete Project Study Material 8

Chapter 1: Project Introduction 8
- 1.1 Project title and identity 8
- 1.2 Background and motivation 8
- 1.3 Problem statement 8
- 1.4 Why this problem matters 9
- 1.5 What the system aims to achieve 9
- 1.6 Scope of the project 10
- 1.7 Intended users and potential applications 10
- 1.8 What makes this an AI / machine-learning project 10
- 1.9 What the system actually delivers 11
- 1.10 Important limitations (summary; full discussion in Chapter 13) 11
- 1.11 How to say it to an examiner 12

Chapter 2: Complete System Workflow 13
- 2.1 System workflow diagram 13
- 2.2 Stage-by-stage explanation 14
- 2.3 Order of execution (Makefile) 15
- 2.4 Two kinds of data: frozen and live 16

Chapter 3: Technologies and Libraries 17
- 3.1 Programming, tooling and automation 17
- 3.2 Data acquisition and data processing 18
- 3.3 Machine learning and statistics 19
- 3.4 Visualisation, dashboard and API 19
- 3.5 Technologies that are NOT used (so you do not claim them) 19
- 3.6 How the technologies fit the workflow 20

Chapter 4: Dataset and Data Processing 21
- 4.1 Data source and how the data is obtained 21
- 4.2 What data exists (assets, date range, frequency, records) 21
- 4.3 Column dictionary 22
- 4.4 How the three assets' pipelines differ 22
- 4.5 Missing values, duplicates and data cleaning 23
- 4.6 Preprocessing, scaling and normalisation 23
- 4.7 Return statistics of the training window (from results/eda_summary.csv) 24
- 4.8 Data quality limitations (honest list) 25

Chapter 5: Target Variable and Prediction Horizon 26
- 5.1 What exactly is predicted 26
- 5.2 The formula in simple language 26
- 5.3 A real worked example (Gold, test period) 27
- 5.4 How the target is created in the code 27
- 5.5 Standardisation of the target 28
- 5.6 Why this target is suitable 28
- 5.7 How the target relates to the input features 28
- 5.8 Other targets defined in the code (experiments only) 28

Chapter 6: Feature Engineering 29
- 6.1 Policy: three rules every feature follows 29
- 6.2 The close-time rule (the most important design decision) 29
- 6.3 Complete feature dictionary 30
- 6.4 The technical indicators explained one by one 32
- 6.5 One real feature row (Gold, 2026-05-12, frozen dataset) 33
- 6.6 Which features matter, and what the evidence says 34
- 6.7 How features differ across assets and models 34

Chapter 7: Machine Learning Fundamentals Used in Our Project 36

Chapter 8: Every Machine Learning Model 40
- 8.1 Naive (random walk): the baseline every model must beat 41
- 8.2 Naive-Mean (historical mean, "drift") 41

8.3 ARIMA (statistical time-series baseline) ... 42
8.4 Ridge regression ... 42
8.5 Random Forest ... 43
8.6 LightGBM (gradient-boosted trees) ... 44
8.7 CatBoost (gradient boosting with symmetric trees) ... 45
8.8 GRU (Gated Recurrent Unit) ... 45
8.9 LSTM (Long Short-Term Memory) ... 46
8.10 Stacked ensemble (experiment, not served) ... 47
8.11 Side-by-side comparison ... 48
Chapter 9: Training, Validation and Testing ... 49
9.1 Dataset splitting (actual dates and record counts) ... 49
9.2 Why time-series validation differs from ordinary random splitting ... 50
9.3 Walk-forward (expanding window) validation ... 50
9.4 Hyper-parameter selection ... 51
9.5 Model fitting: two phases for every model with a stopping rule ... 51
9.6 Saved model artefacts ... 52
9.7 How data leakage can occur, and what the code does about it ... 53
9.8 Weaknesses, and how the methodology was corrected over time ... 54
Chapter 10: Model Evaluation and Actual Results ... 55
10.1 The metrics we use ... 55
10.2 Which result is which (train, validation, walk-forward, pooled, test) ... 56
10.3 Training versus validation (over-fitting check) ... 56
10.4 Untouched test set, per asset ... 57
10.5 Walk-forward validation (model-selection evidence) ... 60
10.6 Pooled walk-forward evidence (about six years, train+validation only) ... 61
10.7 Do the models beat the baseline? (the central question) ... 62
10.8 The uncertainty band: the one useful output ... 64
10.9 Performance by market regime (test period) ... 65
10.10 Strategy backtest (context only) ... 65
10.11 Design experiments (validation only) ... 66
10.12 What the results demonstrate (and what they do not) ... 66
Chapter 11: Ensemble and Final Prediction ... 67
11.1 Which models contribute to the final result ... 67
11.2 How each prediction is generated and combined (the real code) ... 67
11.3 Weighting: none ... 68
11.4 The mathematical formula ... 68
11.5 Does the ensemble perform better than the individual models? ... 69
11.6 Why combining can help, and why it can also perform worse ... 70
11.7 The actual end-to-end flow when a user requests a prediction ... 70
Chapter 12: Streamlit Application and User Experience ... 71
12.1 Entry point, layout and navigation ... 71
12.2 Sidebar controls and what they call ... 72
12.3 Tab 1: Forecast ... 72
12.4 Tab 2: Predict a Day (unseen test) ... 73
12.5 Tab 3: Model Performance ... 73
12.6 Tab 4: Methodology ... 73
12.7 Errors, warnings and messages ... 74
12.8 Working, not connected, or not implemented? ... 74
12.9 The user journey, from opening the page to the final result ... 74
Chapter 13: Strengths, Limitations and Future Improvements ... 75
13.1 Genuine technical strengths ... 75
13.2 Limitations by area ... 76
13.3 Future improvements (each with a reason) ... 77
Chapter 14: Project Security, Reliability and Reproducibility ... 78
14.1 What happens when something goes wrong ... 78
14.2 Dependencies, configuration and secrets ... 78
14.3 Security observations ... 79
14.4 Reproducibility, serialisation and preprocessing consistency ... 79
Chapter 15: Complete Project Summary (revision sheet) ... 80

PART 2: Complete VS Code Folder, File and Source Code Explanation **81**
Chapter 16: Complete Folder Structure 81
16.1 The actual folder and file tree 81
16.2 Every folder: purpose, contents, connections 84
16.3 Every Python file at a glance 86
16.4 Non-Python files and artefacts 89
16.5 Generated files, caches and files that are not application source 90
Chapter 17: Every Important Source File Explained 91
config.py 91
src/data/data_collection.py 92
src/data/external_data.py 93
src/data/market_calendar.py 93
src/data/preprocessing.py 94
src/data/sync_live_data.py 95
src/data/eda.py 96
src/models/registry.py 96
src/models/model_gru.py 97
src/models/model_lstm.py 98
src/models/ensemble_model.py 98
src/training/tune_models.py 99
src/training/train_models.py 100
src/evaluation/cross_validation.py 101
src/evaluation/backtesting.py 101
src/evaluation/plots.py 102
src/inference/prediction.py 103
src/utils/metrics.py 104
src/utils/inverse_transform.py 105
src/utils/reproducibility.py 105
src/utils/logging_config.py 106
src/api/app.py 106
app/streamlit_app.py 107
src/experiments/run_experiments.py 108
src/experiments/audit_improvements.py 108
src/experiments/before_after.py 109
scripts/retrain.py 109
tests/ (6 files, 47 test cases) 110
notebooks/ and the empty __init__.py files 110
Chapter 18: Function-by-Function Reference 111
Configuration and data acquisition 112
Preprocessing and feature engineering 113
Models, saving and loading 115
Tuning, training, validation, evaluation 117
Metrics and utilities 118
Inference and API 119
Streamlit, scripts, experiments 120
Chapter 19: How the Files Connect 121
19.1 Which file runs first? 121
19.2 Layered module map 122
19.3 Exact import graph (generated from the source) 123
19.4 Data flow: which file writes what, and who reads it 124
19.5 Where does each key event happen? 125
Chapter 20: Important Code Explained 126
20.1 Target generation and the split-by-target-date masks 126
20.2 Feature alignment: the close-time rule 126
20.3 Time-series cross-validation: the walk-forward loop 127
20.4 Model training: the two-phase fit of the neural networks 128
20.5 Model evaluation: the Diebold-Mariano test 129
20.6 Ensemble calculation in the evaluation (walk-forward) 130
20.7 The calibrated 68 % band 130

20.8 Prediction: the live next-day forecast ... 131
20.9 The main dashboard control flow ... 132
Chapter 21: Configuration, Dependencies and Running the Project ... 133
21.1 Required software ... 133
21.2 Dependencies (requirements.txt) ... 133
21.3 Configuration and environment ... 133
21.4 What must exist before each step ... 134
21.5 Commands (exact, from the Makefile) ... 134
21.6 Reproducing the results ... 134
21.7 Troubleshooting ... 135
Chapter 22: Trace One Prediction Through the Actual Code ... 136
22.1 The call chain ... 136
22.2 Step by step with file and function names ... 137
22.3 How the data changes along the way (real example) ... 138
22.4 The same request through the API ... 138
22.5 What differs in "Predict a Day" ... 139
Chapter 23: Likely Code-Related Viva Questions ... 140
Glossary ... 147
Appendix A: Quick reference of the key numbers ... 149
Appendix B: Verification record (what was checked, and how) ... 150
Appendix C: Discrepancies and stale items found while checking ... 151
Appendix D: Dashboard screenshots (captured 2026-10-09, Gold, live data) ... 152

PART 1
Complete Project Study Material

The project as a working machine-learning system: what it does, why, how, with which tools, and what the results really mean

15 chapters | simple English first, then the technical explanation | all numbers verified against the repository

Chapter 1: Project Introduction

1.1 Project title and identity

Item	Value (verified in the repository)
Project title	**AI-Powered Commodity Price Predictor** (the title of the FastAPI service in `src/api/app.py`)
README title	"Multi-Asset Next-Day Return Prediction: Bitcoin, Gold, Silver"
Repository / folder	`crypto-forex-prediction-system` (the GitHub-style folder name; the first commit of 2026-01-31 was titled "BTC & Forex Price Prediction System")
Level	Final Year Project, BSCS, University of Lahore (team name `teamlocalhost`, shown in the README footer)
Assets	Bitcoin (a cryptocurrency), Gold and Silver (precious-metal futures). The word "commodity" in the title covers the two metals; Bitcoin is the crypto asset
Phase	Phase 2 (final). Phase 1 produced the first prototype; this guide documents the final audited system

1.2 Background and motivation

Prices of Bitcoin, Gold and Silver move every day. Traders, investors and risk managers would love to know tomorrow's price, and many websites and papers claim that machine learning (ML) can predict it. But financial prices are noisy, fat-tailed and influenced by news nobody can foresee. Many student projects report an impressive "95 % accuracy" that disappears once the evaluation is done fairly, because they measure price *levels* (where "tomorrow = today" already scores about 95 %), because future information leaks into the inputs, or because the test set is used to tune the model.

We therefore did not only want to *build* a predictor. We wanted to build one **correctly**, test it **fairly** against the simplest possible baseline, and report the result **honestly**. That makes the project useful as a template for how a time-series ML project should be done.

1.3 Problem statement

> **Problem statement**
> Given only public daily data available at the moment a day's close is fixed, can machine-learning models forecast the **next trading day's return** of Bitcoin, Gold and Silver **better than the naive forecast "tomorrow = today"**, and how large and how certain is the forecast?

This is a *regression* problem on a *time series*: the output is a number (the next-day log return), the data are ordered in time, and the evaluation must respect that order.

1.4 Why this problem matters

- **Practical interest.** Even a small edge in next-day direction or size would be valuable, so the question is worth asking carefully.
- **Methodological interest.** Financial series are the classic place where ML projects go wrong (look-ahead leakage, random shuffling, tuning on the test set). Showing the correct way is a contribution in itself.
- **Risk management interest.** Even when direction is unpredictable, the *size* of tomorrow's move (volatility) is partly predictable. Our served 68 % band uses exactly that.
- **Engineering interest.** The project covers the full ML life cycle: data acquisition, cleaning, features, training, validation, evaluation, deployment (dashboard and API), tests and reproducibility.

1.5 What the system aims to achieve

Main objectives

1. **Collect** daily prices of Bitcoin, Gold, Silver plus macro and sentiment series from free public sources, keeping only *complete* daily bars (`src/data/`).
2. **Engineer** stationary, backward-looking features with a strict *close-time rule* so that no feature contains information from after the price is fixed (`src/data/preprocessing.py`).
3. **Train and compare** six ML models and three baselines on identical days under time-ordered validation (`src/training/`, `src/evaluation/`).
4. **Evaluate honestly** with return-space metrics, directional accuracy with a significance test, the Diebold-Mariano test against the random walk, and one untouched test period.
5. **Serve** one combined forecast with its uncertainty band and its measured reliability in a Streamlit dashboard and a FastAPI endpoint (`app/`, `src/api/`, `src/inference/`).

Secondary objectives

1. Make the whole pipeline **reproducible** (fixed seeds, pinned versions, one loader function, one config file).
2. Protect against mistakes with an **automated test suite** (47 tests) and continuous integration.
3. Run **design experiments** (data size, feature groups, prediction horizon, volatility target, ensemble variants) using validation data only.
4. Provide a **live demonstration** mode: "Predict a Day" on days the models never saw, and a live per-model scorecard on the most recent days.
5. Document limits honestly (no financial advice, no claim of skill).

1.6 Scope of the project

In scope (implemented)	Out of scope (not implemented)
Daily frequency; one-day-ahead horizon (the served task)	Intraday, minute-level or tick data
Three assets: Bitcoin, Gold futures, Silver futures	Other assets, forex pairs, stocks (the repository folder name mentions forex, but no forex pair is modelled)
Price features, macro returns (S&P 500, VIX, DXY, WTI, 10-year yield), Fear & Greed (Bitcoin only), gold return (Silver only)	News, tweets, on-chain data, order books, fundamentals
Six ML models, three baselines, one stacked experiment, equal-weight Combined forecast	Transformers, reinforcement learning, trading execution, portfolio optimisation
Streamlit dashboard (4 tabs) and FastAPI (3 endpoints)	User accounts, authentication, database, deployment to a cloud server
Honest evaluation and calibrated 68 % band	Any claim of profit or trading performance

1.7 Intended users and potential applications

- **Examiners and supervisors:** a transparent example of a fair ML-for-finance evaluation.
- **Students:** a reference implementation of time-series validation, leakage prevention and metric reporting, plus a full study guide (this document).
- **Researchers/analysts (as a prototype):** the calibrated volatility band is a reasonable one-day "range of plausible prices" for each asset.
- **Not intended for:** real trading decisions. The dashboard footer states this on every page, and the evidence in Chapter 10 supports it.

1.8 What makes this an AI / machine-learning project

- The forecasting rule is **learned from data**, not written by hand: six algorithms fit their parameters (coefficients, trees, neural-network weights) to about 1,900 to 2,750 training examples.
- It is **supervised learning**: every training example pairs inputs (features of day t) with a known answer (the return that happened on day t+1).
- It includes the standard ML life cycle: feature engineering, a train / validation / test split, hyper-parameter tuning, regularisation, early stopping, evaluation against baselines, and model persistence.
- Two of the six models (GRU and LSTM) are **deep-learning** recurrent neural networks built with TensorFlow / Keras; four are classical or tree-based ML.

1.9 What the system actually delivers

Deliverable	Status	Where
Data collection from Yahoo Finance and alternative.me with the complete-bar rule	Working	`src/data/data_collection.py`, `external_data.py`, `market_calendar.py`
Cleaning, feature engineering, close-time alignment, split, scaler	Working	`src/data/preprocessing.py`
Walk-forward hyper-parameter tuning	Working	`src/training/tune_models.py`
Two-phase training of 6 models x 3 assets = 18 models, saved to disk	Working	`src/training/train_models.py`, `data/models/`
Single test evaluation, pooled walk-forward tests, band calibration, `model_status.json`	Working	`src/evaluation/backtesting.py`
Combined forecast, UP / DOWN, 68 % band, live scorecard, predict-for-date	Working	`src/inference/prediction.py`
Streamlit dashboard with 4 tabs	Working (verified live on 2026-10-09)	`app/streamlit_app.py`
FastAPI service	Working (3 endpoints; code-reviewed, covered by inference tests)	`src/api/app.py`
Design experiments E1 to E5 and before/after comparison	Working	`src/experiments/`
Tests (47) and CI workflow	Working (47 passed on 2026-10-09)	`tests/`, `.github/workflows/ci.yml`
Four companion notebooks	Present; they display saved results (some text is stale, see Appendix C)	`notebooks/`
Forward prediction log / automatic daily job	**Not implemented** (built once, removed at the team's request)	git history only

1.10 Important limitations (summary; full discussion in Chapter 13)

- **No demonstrated predictive skill** over "tomorrow = today". This is the central finding.
- The **test window is short** (143 to 207 days) and was inspected during development, so it is not a pristine one-shot holdout; the pooled walk-forward record (about six years) is the larger-sample evidence and agrees.
- The metals' high **UP-call rate (66 to 100 % per model on the test days; 97 % Gold and 91 % Silver for the Combined forecast)** means their direction hit-rate reflects the rising average price (drift), not timing.
- Free **Yahoo data** is revised after futures contract rolls and Bitcoin's newest bar sometimes appears with a delay.
- The system is a **research prototype**, not a trading tool and not financial advice.

1.11 How to say it to an examiner

30-second answer

"We built a machine-learning system that forecasts the next-day return of Bitcoin, Gold and Silver from public daily data. We engineered it so the test is fair: no future information in the inputs, time-ordered validation, simple baselines and significance tests. Six models and their equal-weight combination are all statistically tied with 'tomorrow equals today', which is what efficient-market theory predicts. What the data does let us predict is the size of tomorrow's move, and our dashboard shows that as a calibrated 68 % range."

Two-minute answer (outline)

1. Problem: forecast next-day price of three assets and measure honestly whether ML beats the random walk.

2. Data: Yahoo Finance daily bars 2018 to 2026, five macro series, Fear & Greed; complete bars only.

3. Method: log-return target; 28/29 backward-looking features; chronological split; six models; walk-forward tuning; one test evaluation.

4. Special care: found and fixed a close-time leak in Gold/Silver that had produced a fake 62 % directional accuracy; the corrected result is null.

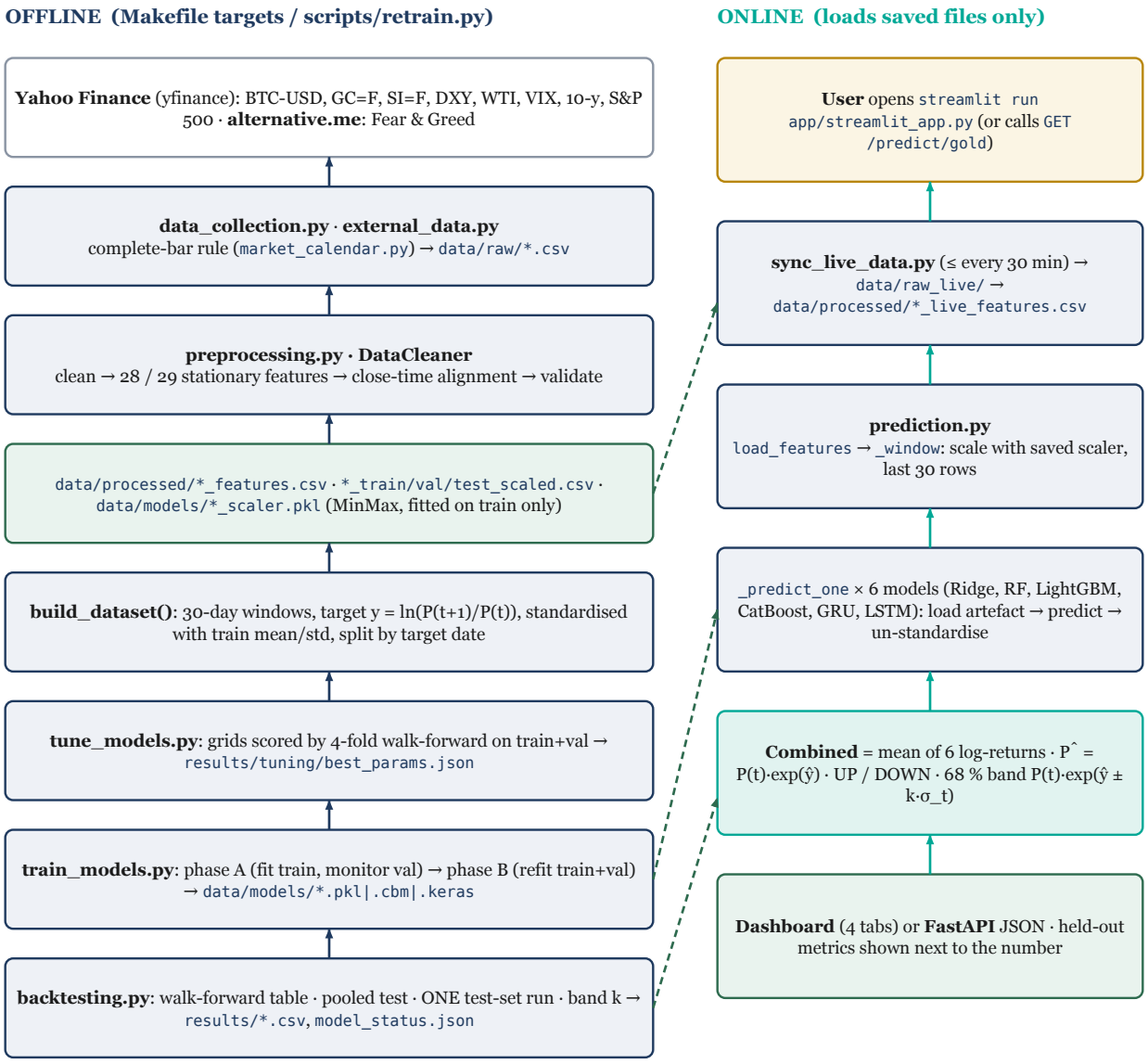
5. Output: Combined forecast (mean of 6), UP/DOWN, calibrated 68 % band, and the reliability numbers next to it, in Streamlit and FastAPI.

6. Conclusion: correct, reproducible system; no skill over the baseline; useful volatility band; limits stated.

Chapter 2: Complete System Workflow

This chapter follows the data from the internet to the dashboard, exactly as the code does it. Read the diagram first, then the stage table.

2.1 System workflow diagram



Dashed green = files written offline and read online. The dashboard never trains.

*The left column is the **offline** pipeline: it is run by the team (with `make` targets or `scripts/retrain.py`), takes minutes to hours, and writes files. The right column is **online**: the dashboard and the API only read those files. This separation answers a very common viva question: "**does the dashboard train models when it starts?**" No. It loads the saved scaler and the six saved models for the chosen asset and runs them; training happened earlier.*

2.2 Stage-by-stage explanation

Stage	What happens	Why it is needed	In → out	Component
1. Data acquisition	Download daily bars (open, high, low, close, volume) for BTC-USD, GC=F, SI=F; macro closes for DX-Y.NYB, CL=F, ^VIX, ^TNX, ^GSPC; Fear & Greed from alternative.me. The still-running bar of today is dropped.	We need history to learn from. Dropping the running bar means no "prediction" is ever checked against an intraday snapshot.	Tickers, start date 2018-01-01 → CSV files in `data/raw/` (frozen) or `data/raw_live/` (live)	`fetch_asset`, `fetch_yfinance_external`, `fetch_fear_greed_index`, `drop_incomplete_bars`
2. Data loading	Read the raw CSV into a DataFrame and parse timestamps.	One entry point per asset.	CSV → DataFrame	`DataCleaner.load_data`
3. Cleaning	Drop duplicate timestamps, sort, keep positive prices. Bitcoin: re-index to every calendar day and forward-fill gaps (none were found). Gold/Silver: keep exchange days only (no weekend rows).	Garbage or fake rows would create fake returns (an old version had 30 % fake zero-return weekend rows for metals).	Raw DataFrame → clean DataFrame	`DataCleaner.clean_data`
4. Feature engineering	Compute log return, RSI, ADX, ROC, EMA ratios, MACD/price, Bollinger %B and width, ATR/price, high-low range, lagged returns, 20-day momentum, rolling / HAR / EWMA volatility; Bitcoin: day-of-week, volume change.	The models need numbers that describe the recent past, in a scale-free (stationary) form.	Clean prices → 23 core features plus asset-specific ones	`add_moving_averages`, `add_rsi`, `add_macd`, `add_advanced_ta`, `add_relative_features`, `add_lag_returns`, `add_rolling_volatility`, `add_calendar_features`, `add_volume_change`
5. Close-time alignment	Add macro and sentiment returns using only values known when the asset's close is fixed: same-day for Bitcoin, previous-day for the metals; lag the High/Low-based features for the metals one session.	This is the **leakage guard**. Without it the metals showed a fake 62 % directional accuracy.	External series → `sp500_return`, `vix_return`, `dxy_return`, ...	`_align_external`, `add_external_features`, `lag_post_settlement_features`
6. Validation of features	Raise an error if any NaN or infinite value or fewer than 100 rows remain.	A corrupt download must never reach the models.	Feature table → same table or `ValueError`	`validate_features`
7. Split and scaling	Cut into train (to 2025-09-10), validation (to 2026-02-17) and test by calendar date; fit a MinMaxScaler on the training rows only and transform all three.	Time order must be respected; the scaler must never "see" validation or test values.	Feature table → three scaled CSVs + `*_scaler.pkl`	`normalize_and_split`, `save_data`
8. Dataset assembly and target	Build 30-day sliding windows over the scaled rows; compute the target $y = \ln(P_{t+1}/P_t)$; standardise it with the training mean and std; assign each sample to train/val/test by the date its *target* covers.	This is the one function used by tuning, training, stacking, evaluation and inference, so every stage sees identical data.	Scaled CSVs + features → arrays `X_*` (n,30,F), `Xt_*` (n,F), `y_*`	`build_dataset`, `_task_target`, `_make_windows`
9. Hyper-parameter tuning	Grid search per model; each configuration scored by 4-fold expanding walk-forward validation on train+val; the lowest mean RMSE wins.	Choose regularisation strength without ever touching the test set.	Grids → `results/tuning/best_params.json`	`tune_models.py`, `cv_evaluate`
10. Model training	Phase A: fit on train, monitor validation to find the stopping point. Phase B: refit on train+val with that stopping point fixed. Save to disk.	Honest validation metrics plus a final model that uses all pre-test data.	Arrays + params → `data/models/*.pkl|.cbm|.keras`	`train_asset` in `train_models.py`, `registry.py`
11. Stacking (experiment)	Fit a non-negative Ridge meta-model on out-of-fold predictions of Ridge, LightGBM, CatBoost, GRU.	Tested as an alternative to equal weights; **not served**.	OOF predictions → `*_stack.pkl`	`fit_stack` in `ensemble_model.py`

Stage	What happens	Why it is needed	In → out	Component
12. Validation and evaluation	Walk-forward table (baselines + models + Combined), pooled walk-forward tests, one pass over the untouched test set, band calibration, regime analysis; writes CSVs, `model_status.json` and `FINAL_RESULTS.md`.	The only place the test set is read. Produces all numbers shown to the user.	Models + arrays → `results/*.csv`, `model_status.json`	`main` in `backtesting.py`
13. Model loading	Load the saved scaler, target mean/std and each model artefact.	Inference must use the same preprocessing as training.	`data/models/*` → in-memory models	`load_trained`, `_load`, `_target_stats`
14. Prediction generation	Take the last complete day, scale the last 30 rows with the saved scaler, run every model, convert standardised outputs back to log returns.	This is what "Run prediction" does.	Live features → $\hat{y}$ from each model	`predict_next_day`, `_window`, `_predict_one`
15. Combination, UP/DOWN, band	Average the six returns; $\hat{P} = P_t \cdot \exp(\hat{y})$; direction = UP if $\hat{y} > 0$; band = $P_t \cdot \exp(\hat{y} \pm k \cdot \sigma_t)$.	One number for the user, plus honest uncertainty.	Six $\hat{y}$ → price, direction, band	`_predict_from_window`, `_band`
16. Dashboard / API	Format and display: final card, scorecard, charts, tables, methodology; or return JSON.	The user interface.	Result dict → HTML / Plotly / JSON	`app/streamlit_app.py`, `src/api/app.py`

2.3 Order of execution (Makefile)

`make pipeline` runs, in order: `preprocess eda tune train stack evaluate figures experiments test`. Before `preprocess`, the raw files must exist (`make collect-data`). The target names map to scripts:

Make target	Script it runs	Writes
`collect-data`	`src/data/data_collection.py`, `src/data/external_data.py`	`data/raw/*.csv`
`preprocess`	`src/data/preprocessing.py`	`data/processed/*_features.csv`, `*_{train,val,test}_scaled.csv`, `data/models/*_scaler.pkl`
`eda`	`src/data/eda.py`	`results/eda_summary.csv`, `results/figures/eda_*.png`
`tune`	`src/training/tune_models.py`	`results/tuning/*.csv`, `results/tuning/best_params.json`
`train`	`src/training/train_models.py`	`data/models/<asset>_<model>.*`, `results/train_val_metrics.csv`, `results/feature_importance.csv`, `results/histories/*.json`
`stack`	`src/models/ensemble_model.py`	`data/models/*_stack.pkl`, `results/stacking/*.json`
`evaluate`	`src/evaluation/backtesting.py`	`results/cv_results.csv`, `final_test_results.csv`, `walkforward_pooled.csv`, `band_calibration.csv`, `regime_analysis.csv`, `predictions/*.csv`, `FINAL_RESULTS.md`, `data/models/model_status.json`
`figures`	`src/evaluation/plots.py`	`results/figures/*.png`
`experiments`	`run_experiments.py`, `audit_improvements.py`, `before_after.py`	`results/experiments/*`

Make target	Script it runs	Writes
`serve` / `api` / `predict`	Streamlit, uvicorn, `prediction.py` (main)	(nothing; runtime only)

2.4 Two kinds of data: frozen and live

	Frozen evaluation data	Live data
Folder	`data/raw/`, `data/processed/<prefix>_features.csv`	`data/raw_live/`, `data/processed/<prefix>_live_features.csv`
Last bar	2026-09-12 (Bitcoin), 2026-09-11 (Gold, Silver)	Updated whenever Yahoo publishes a new complete bar (latest: 2026-10-08 on the day of writing)
Used for	Training, tuning, every reported metric, the "Predict a Day" demo on stored test days	Next-day forecasts ("Run prediction"), live scorecard, new demo days marked NEW
Changed by	Only `make collect-data` + `make preprocess`	`src/data/sync_live_data.py` (button or automatic check at most every 30 minutes)
Why separate	Results stay reproducible. The dashboard can never overwrite them.	The dashboard can show fresh prices without touching the evidence.

> **Why the scaler and the models are NOT retrained on live data**
> The deployed models were trained on data up to 2026-02-17. New live days only provide new *inputs*; the models keep their trained parameters. This is why the "Predict a Day" list can grow every day while the models stay fixed, and why all days after 2026-02-17 are genuinely unseen.

Chapter 3: Technologies and Libraries

Only technologies that are **actually imported or configured** in the repository are listed. Versions are those installed in the project's `venv` (Python 3.9.6) on 2026-10-09, matching `requirements.txt`. The "where" column was generated by scanning the source files' imports, not typed by hand.

3.1 Programming, tooling and automation

Technology	What it is / what it does	Why we use it	Where it appears
Python 3.9.6 (local venv); CI uses 3.11	General-purpose language; the entire project is Python.	The ML ecosystem (pandas, scikit-learn, TensorFlow) is Python-first.	Every `.py` file
GNU Make	Runs named commands from a `Makefile`.	One memorable command per pipeline stage (`make train`, `make evaluate`, `make serve`).	`Makefile`
Git	Version control: 45 commits from 2026-01-31 to 2026-10-04.	History, rollback, evidence of what changed and why.	`.git`, `.gitignore`, `CHANGELOG.md`
GitHub Actions	Continuous-integration service. The workflow runs on push / pull request to `main`: Python 3.11, `pip install -r requirements.txt`, four import checks, `pytest`.	Catches broken imports and failing tests automatically.	`.github/workflows/ci.yml`
VS Code	The editor. The repo only stores `.vscode/settings.json` with `makefile.configureOnOpen: false`; `.claude/launch.json` defines preview servers for Streamlit and the API.	Day-to-day development.	`.vscode/`, `.claude/launch.json`
argparse, logging, json, itertools, subprocess, time, datetime (Python standard library)	Command-line options, log messages, JSON files, grid combinations, running scripts, timing.	No extra dependency needed.	`train_models.py`, `tune_models.py`, `logging_config.py`, `retrain.py`, `prediction.py`

3.2 Data acquisition and data processing

Technology	What it does in our project	Where (generated from imports)	What depends on it
yfinance 1.2.0	Downloads daily OHLCV for BTC-USD, GC=F, SI=F and the macro tickers through `yf.Ticker(ticker).history(start=...)`.	`data_collection.py`, `external_data.py`	All data; the live sync; therefore every prediction after 2026-09-12
urllib + json (stdlib)	One HTTPS GET to `api.alternative.me/fng/?limit=0` for the Fear & Greed index (no API key).	`external_data.py` (`fetch_fear_greed_index`)	Bitcoin feature `fear_greed`
zoneinfo / tzdata	Converts "now" to America/New_York to decide whether a futures bar is complete (17:15 ET rule).	`market_calendar.py`	Complete-bar rule, dashboard warnings
pandas 2.3.3	Tables (DataFrame), CSV read/write, `rolling`, `ewm`, `shift`, date indexes, `TimeSeries` alignment.	`streamlit_app.py`, `data_collection.py`, `eda.py`, `external_data.py`, `market_calendar.py`, `preprocessing.py` +8 more	Essentially every stage
NumPy 1.26.4	Arrays, sliding windows, `exp`, `log`, quantiles; the format models consume.	`streamlit_app.py`, `eda.py`, `preprocessing.py`, `backtesting.py`, `cross_validation.py`, `plots.py` +9 more	Windows, targets, metrics, inference
ta 0.11.0	Technical-analysis library: Bollinger Bands, ATR, ADX, ROC (`add_advanced_ta`).	`preprocessing.py`	Features `bb_pctb`, `bb_width`, `atr_norm`, `ADX`, `ROC`
SciPy 1.13.1	`binomtest` (direction p-value), Student-t CDF (Diebold-Mariano), `norm`, `skew`, `kurtosis`.	`eda.py`, `metrics.py`	Metrics and EDA
joblib 1.5.3	Saves and loads Python objects (scaler, scikit-learn and LightGBM models, stack) as `.pkl`.	`preprocessing.py`, `prediction.py`, `ensemble_model.py`, `registry.py`	Model persistence and inference

3.3 Machine learning and statistics

Technology	What it is	Role in our project	Where
scikit-learn 1.6.1	Classical ML toolkit.	`MinMaxScaler` (scaling), `Ridge`, `RandomForestRegressor`, `TimeSeriesSplit` (walk-forward folds), the non-negative Ridge meta-model.	`preprocessing.py`, `cross_validation.py`, `ensemble_model.py`, `registry.py`
LightGBM 4.6.0	Fast gradient-boosted decision trees (leaf-wise growth, histogram splits).	Model 3 of 6 (`LightGBMModel`), early stopping on validation.	`registry.py`
CatBoost 1.2.10	Gradient boosting with symmetric ("oblivious") trees and ordered boosting.	Model 4 of 6 (`CatBoostModel`); the best single model by validation for Bitcoin and Gold.	`registry.py`
TensorFlow / Keras 2.16.2	Deep-learning framework; Keras API (`Sequential`, `GRU`, `LSTM`, `Dense`, `Dropout`, `Adam`, `EarlyStopping`).	Models 5 and 6 (`GRUModel`, `LSTMModel`); also seeds (`tf.random.set_seed`).	`model_gru.py`, `model_lstm.py`, `registry.py`, `reproducibility.py`
statsmodels 0.14.6	Statistical models and tests.	`ARIMA` baseline, Augmented Dickey-Fuller test, ACF / PACF plots.	`eda.py`, `registry.py`

3.4 Visualisation, dashboard and API

Technology	What it is	Role in our project	Where
Streamlit 1.50.0	Python framework that turns a script into an interactive web app.	The entire dashboard: sidebar controls, 4 tabs, metrics, tables, spinners, status boxes, `st.cache_data`, `st.session_state`.	`streamlit_app.py`
Plotly 6.7.0	Interactive charts.	All dashboard charts: price with split shading, forecast chart, comparison bars, scatter, importance.	`streamlit_app.py`
matplotlib 3.9.4	Static plotting library (`Agg` backend).	Report figures (`plots.py`) and EDA plots (`eda.py`). This guide's own charts were also drawn with it.	`eda.py`, `plots.py`
FastAPI 0.128.8 + **uvicorn** + **pydantic**	Web-API framework, ASGI server, and typed data models.	`/health`, `/models`, `/predict/{asset}`; `PredictionResponse` validates the JSON; CORS middleware enabled.	`app.py`
pytest 8.4.2	Test runner.	47 tests in `tests/` (look-ahead, alignment, split, metrics, inference, failure modes).	(see text)

3.5 Technologies that are NOT used (so you do not claim them)

> **Do not claim**
> There is **no** XGBoost, Transformer / PatchTST / TFT, Prophet, Williams %R or CCI indicator, database, MLflow, Docker or cloud deployment in the final project. An earlier README claimed some of these; the audit of 2026-09-12 removed the claims. Docker and Docker Compose files were removed in commit `9e7a83c` (2026-09-14).
>
> `requests` and `jupyter` are listed in `requirements.txt` but `requests` is **not imported anywhere** in the code and `jupyter` is only needed to open the notebooks. `reportlab` and `python-dotenv` were removed from the requirements after their uses were deleted.

3.6 How the technologies fit the workflow

Workflow stage	Technologies used
Acquire	yfinance, urllib/json, zoneinfo, pandas
Clean and feature-engineer	pandas, NumPy, `ta`
Scale and split	scikit-learn (`MinMaxScaler`), joblib
Tune, train, validate	scikit-learn, LightGBM, CatBoost, TensorFlow/Keras, statsmodels (ARIMA), NumPy, `itertools`
Evaluate	NumPy, pandas, SciPy (tests), matplotlib (figures)
Save / load	joblib (`.pkl`), CatBoost native (`.cbm`), Keras (`.keras`), JSON sidecars
Predict and serve	Streamlit + Plotly, FastAPI + uvicorn + pydantic
Protect	pytest, GitHub Actions, fixed seeds (`set_all_seeds`), pinned `requirements.txt` and `requirements-lock.txt`

Chapter 4: Dataset and Data Processing

All statistics below were computed from the actual files in `data/raw/` and `data/processed/` on 2026-10-09 (not copied from earlier documents).

4.1 Data source and how the data is obtained

- **Source:** Yahoo Finance for prices and macro series (through the `yfinance` library) and alternative.me for the Fear & Greed index. Both are free and need **no API key**.
- **Downloaded, then stored locally:** `src/data/data_collection.py` and `src/data/external_data.py` download the full history from `DATA_START_DATE = 2018-01-01` and write CSV files. All later stages read the CSVs, not the internet.
- **Frozen versus live copy:** `data/raw/` is the frozen copy (last bar 2026-09-12) behind every reported result; `data/raw_live/` is refreshed by the dashboard (Chapter 2.4).
- **Git status:** `data/raw/`, `data/processed/` and `data/models/` are in `.gitignore` (only `model_status.json` is tracked), so a fresh clone must run `make collect-data preprocess train stack evaluate`. Because Yahoo revises history, a re-download will differ slightly from the frozen files (the audit measured a maximum revision of 0.75 % on one Bitcoin day, mean below 0.005 %). The earlier 3-year files are tracked in `data/raw_3y_backup/`.
- **Complete-bar rule:** Yahoo returns today's still-running bar as if it were a close. `drop_incomplete_bars` removes rows whose bar is not finished (Bitcoin: the current UTC day; futures and US series: before 17:15 ET).

4.2 What data exists (assets, date range, frequency, records)

All series are **daily** (one row per day for Bitcoin, one row per exchange day for the metals and US market series).

Series	File	Rows	First day	Last day	Columns	Nulls	Dups
Bitcoin (BTC-USD)	`bitcoin_data.csv`	3,177	2018-01-01	2026-09-12	open, high, low, price, volume	0	0
Gold (GC=F)	`gold_data.csv`	2,186	2018-01-02	2026-09-11	open, high, low, price, volume	0	0
Silver (SI=F)	`silver_data.csv`	2,186	2018-01-02	2026-09-11	open, high, low, price, volume	0	0
US Dollar Index (DX-Y.NYB)	`dxy_data.csv`	2,187	2018-01-02	2026-09-11	price	0	0
WTI crude oil (CL=F)	`crude_oil_data.csv`	2,187	2018-01-02	2026-09-11	price	0	0
VIX (^VIX)	`vix_data.csv`	2,187	2018-01-02	2026-09-11	price	0	0
10-year yield (^TNX)	`tnx_data.csv`	2,185	2018-01-02	2026-09-11	price	0	0
S&P 500 (^GSPC)	`sp500_data.csv`	2,185	2018-01-02	2026-09-11	price	0	0
Fear & Greed (alternative.me)	`fear_greed_data.csv`	3,143	2018-02-01	2026-09-13	fear_greed	0	0

Row counts verified: Bitcoin 3,177 rows equals the 3,177 calendar days from 2018-01-01 to 2026-09-12 (no missing day). Gold and Silver have 2,186 rows versus 3,175 calendar days, i.e. 989 weekend and holiday days are *correctly absent*. The raw files contain **0 null values and 0 duplicate dates**.

4.3 Column dictionary

Column	Type	Meaning	Notes
`timestamp`	date (parsed to datetime)	Trading day of the bar	Index of all processed tables
`open`	float64	First traded price of the day	Chart-only; not a model input
`high`	float64	Highest price of the day	Used to build `hl_range`, `atr_norm`, `ADX`; for Yahoo futures it spans the full Globex session, which ends **after** the 13:30 ET settlement
`low`	float64	Lowest price of the day	Same remark as `high`
`price`	float64	Closing price P_t of the day (renamed from Yahoo's `Close`)	Bitcoin: 00:00 UTC bar close; metals: 13:25 to 13:30 ET COMEX settlement
`volume`	int64	Traded volume	Bitcoin: USD volume, usable. Metals: front-month contract volume (e.g. 73 to 275 recently) which jumps at contract rolls; 36 zero-volume rows in Gold and 138 in Silver. Therefore **only Bitcoin uses volume**
`price` (macro files)	float64	Close of DXY, WTI, VIX, 10-year yield or S&P 500	Converted to daily log returns in preprocessing
`fear_greed`	int64	Crypto Fear & Greed index 0 (extreme fear) to 100 (extreme greed)	Observed range 5 to 95, mean 45.5; starts 2018-02-01 (earlier rows are filled with 50)

4.4 How the three assets' pipelines differ

Aspect	Bitcoin (BTC-USD)	Gold (GC=F)	Silver (SI=F)
Instrument	Spot cryptocurrency price	COMEX front-month gold futures	COMEX front-month silver futures
Calendar	7 days a week; missing days would be forward-filled (none found)	Exchange days only (2,186 rows)	Exchange days only (2,186 rows)
Daily close	00:00 UTC bar, **after** every US market close	13:30 ET settlement, **before** S&P (16:00), VIX (16:15), DXY (17:00), WTI (14:30)	13:25 ET settlement
Macro inputs	S&P 500, VIX (same-day values)	DXY, WTI, 10-year yield, S&P 500, VIX (**previous** trading day)	Same as Gold plus Gold's same-day return
Sentiment / calendar / volume	Fear & Greed, day-of-week sin/cos, log volume change	none	none
High/Low-based features	Same day	Lagged one session (`hl_range`, `atr_norm`, `ADX`)	Lagged one session
Model features	**29**	**28**	**29**
Price range 2018 to 2026	$3,236.76 to $124,752.53	$1,176.20 to $5,318.40	$11.73 to $115.08
Processed rows (after 30-row warm-up)	3,147 (from 2018-01-31)	2,156 (from 2018-02-15)	2,156 (from 2018-02-15)

4.5 Missing values, duplicates and data cleaning

Check	Result on the frozen data	Code that handles it
Duplicate timestamps	0 in every raw file	`drop_duplicates(subset=["timestamp"])` in `clean_data`
Missing values in raw files	0	None needed; `validate_features` raises `ValueError` if any NaN remains after feature building
Non-positive prices	none	`self.df = self.df[self.df["price"] > 0]`
Missing calendar days (Bitcoin)	0	`reindex(full date range).ffill()` (a gap would be filled from the previous day and counted in the log)
Weekend rows for metals	not created (an early version created fake ones: 30 % of rows)	Metals keep their own exchange calendar; test `test_commodities_keep_trading_calendar`
Warm-up rows	30 rows removed per asset	`dropna()` after rolling windows (30-day volatility, 22-day RV, 20-day return) are computed
Missing macro value on a day	return filled with 0	`.fillna(0)` in `add_external_features`; Fear & Greed filled with 50
Infinite values	none	`np.isinf(...)` check in `validate_features`
Too little data	n/a	fewer than 100 rows raises `ValueError`

4.6 Preprocessing, scaling and normalisation

1. **Log return.** `log_return = ln(price / price.shift(1))` for the day (an input feature: today's return).
2. **Stationary features only.** Raw level columns (`open, high, low, price, volume, EMA_14, EMA_30, BB_Mid, BB_Upper, BB_Lower, ATR, MACD, MACD_Signal`, 13 in `LEVEL_COLUMNS`) are kept in the file for charts but are **never** model inputs, because their test-period values leave the training range (Bitcoin reached $124k).
3. **MinMaxScaler(0, 1)** is fitted on the **training rows only** and applied to validation and test: $x' = (x - \min_{train}) / (\max_{train} - \min_{train})$. Validation and test values may fall outside [0, 1] (this proves they did not influence the fit; a test checks it).
4. **Target standardisation.** The target is z-scored with the training mean and standard deviation (Bitcoin $\mu = 0.000852$, $\sigma = 0.0336$; Gold $\mu = 0.000541$, $\sigma = 0.00976$; Silver $\mu = 0.000496$, $\sigma = 0.0192$). Predictions are converted back with $\hat{y} = z \cdot \sigma + \mu$.
5. **Windows.** The recurrent models receive the last `SEQ_LEN = 30` days of scaled features (shape 30 × F); the tabular models receive only the last row (shape F).

> **Why scale at all?**
> Neural networks and Ridge regression are sensitive to the size of inputs; trees are not. Using one scaled input for all six models keeps the comparison fair and the code simple. Scaling statistics come from the training period only; otherwise information from the future would leak into the inputs.

4.7 Return statistics of the training window (from `results/eda_summary.csv`)

Asset	Train days	Mean daily return %	Std %	Skew	Excess kurtosis	ADF p (price)	ADF p (return)	ACF lag 1	|r| > 2 % of days
Bitcoin	2,780	0.087	3.42	-0.96	15.2	0.974	0.000	-0.053	39.4
Gold	1,904	0.052	0.97	-0.25	3.6	1.000	0.000	-0.027	4.7
Silver	1,904	0.047	1.91	-0.47	5.2	0.972	0.000	-0.021	19.9

Reading the table in simple words. (1) Prices are *not stationary* (ADF p near 1: they wander), but returns are (p = 0.000): this is why we predict the return. (2) Returns have almost **no autocorrelation** (lag-1 between −0.05 and −0.02): yesterday barely tells us about today, which is the first hint that forecasting is hard. (3) Returns are **fat-tailed** (excess kurtosis 15.2 for Bitcoin): large moves are far more common than a normal distribution implies, so a naive "±1σ" band over-covers; our band is calibrated for this. (4) Bitcoin moves more than 2 % on 39 % of days, Gold on only 5 %.

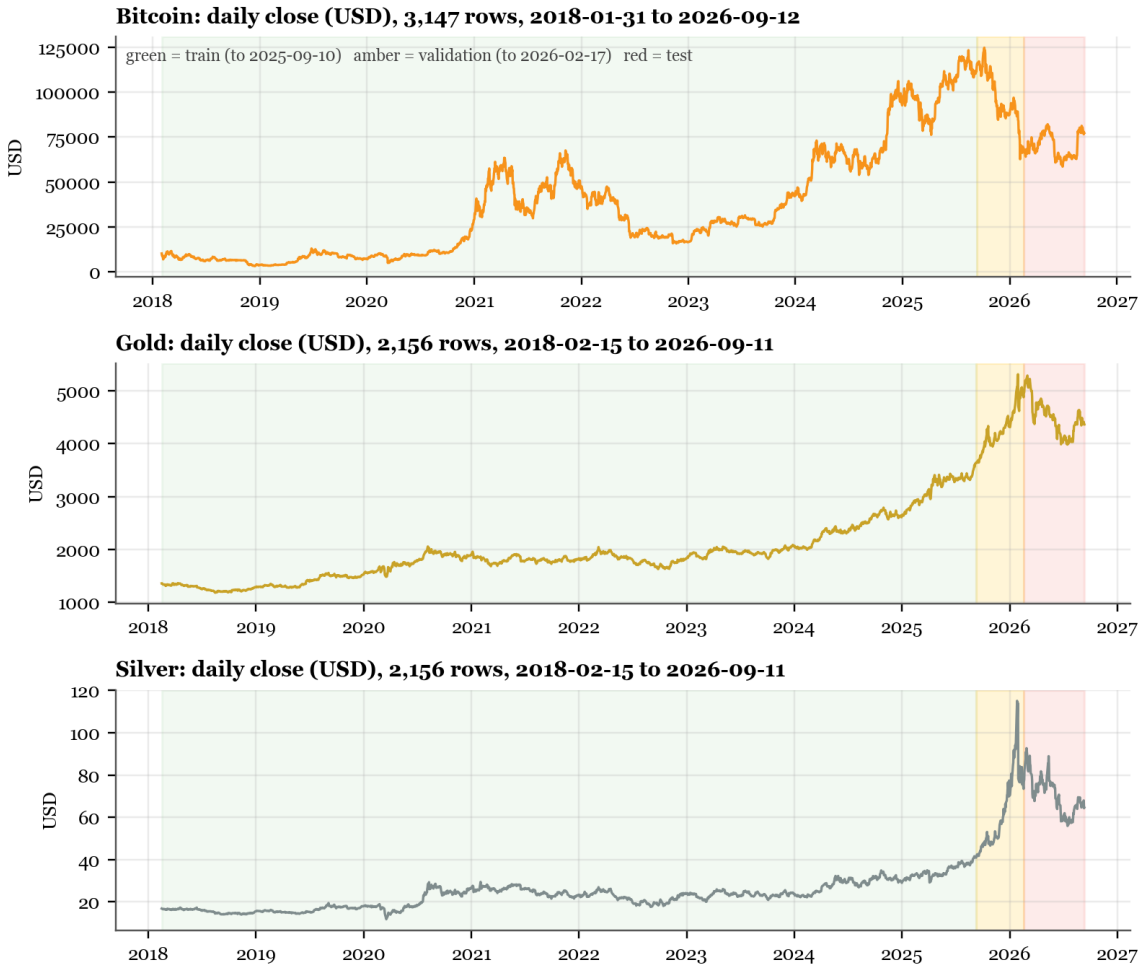


Figure 4.1: Daily close of the three assets with the frozen train / validation / test windows (drawn from `data/processed/*_features.csv`).

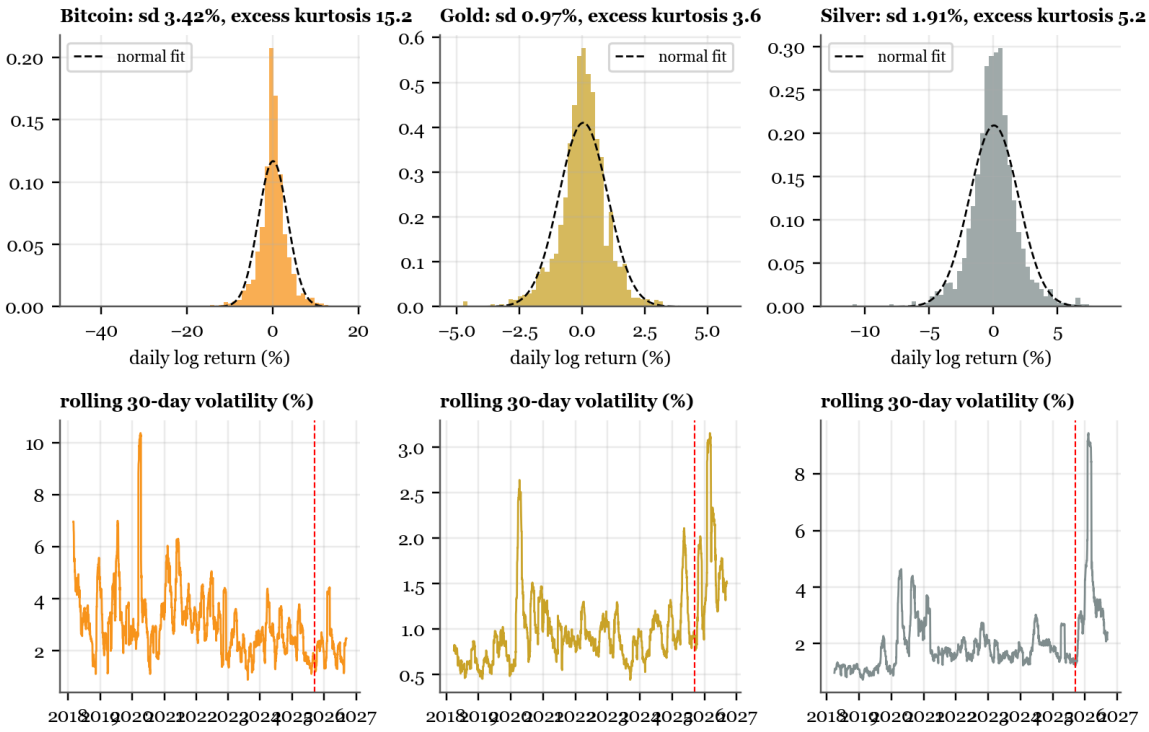


Figure 4.2: Fat-tailed daily returns (top) and volatility clustering (bottom, rolling 30-day standard deviation); red line = end of the training window.

4.8 Data quality limitations (honest list)

- **Free vendor data.** Yahoo can rate-limit, revise past closes, and publish Bitcoin's newest bar late (the dashboard shows a warning when the newest expected bar is missing).
- **Continuous futures series.** GC=F and SI=F follow the current front-month contract and can jump at a roll; the dashboard shows a caption for this on live data.
- **Futures volume is unusable** (dozens to hundreds of contracts, zero on some days), so it is excluded for the metals.
- **Weekend macro values for Bitcoin.** Because the S&P 500 and VIX do not trade on weekends, the aligned `sp500_return` and `vix_return` on Saturday and Sunday simply **repeat Friday's value** (verified: 100 % of Saturdays equal the previous day). This is not a leak (it is past information), but it is a stale input rather than a 0.
- **Only about eight years** of data and a short test window (143 to 207 days). Many market regimes (2020 crash, 2022 crypto winter, 2025 to 2026 metals rally) appear only once.
- **No news or fundamentals**: the models only see prices, their derived indicators, and a handful of daily macro series.

Chapter 5: Target Variable and Prediction Horizon

The code, not the project title, decides what is predicted. Everything below was read from `config.py`, `src/data/preprocessing.py` and `src/inference/prediction.py`.

5.1 What exactly is predicted

Question	Answer (verified in code)	Evidence
Target variable	The **next-day log return** of the asset's close	`config.TARGET_COL = "log_return"`, `_task_target`
Prediction horizon	**1 trading day** ahead (Bitcoin: next calendar day; Gold, Silver: next exchange day)	`PREDICTION_HORIZON_DAYS = 1`, `TASKS["return_1d"]["horizon"] = 1`
Prediction frequency	Once per day, after the day's bar is complete	daily bars; `next_trading_day()` labels the target date
Raw price, return or percentage?	A **return** (log). The price shown to the user is *reconstructed*: $\hat{P} = P_t \cdot \exp(\hat{y})$	`reconstruct_price`, `predict_next_day`
Classification?	No. UP / DOWN is only the **sign** of the predicted return	`direction = "UP" if r_hat > 0 else "DOWN"`
Is the horizon next-day in the final system?	Yes. 5-day return and volatility targets exist only as validation-only experiments	`TASKS` in `config.py`; `src/experiments/run_experiments.py`

5.2 The formula in simple language

> **Definition**
> $y_t = \ln(P_{t+1} / P_t)$
> **In words:** "how much (in natural-log terms) tomorrow's close is above or below today's close". For small moves a log return is almost the same as the percentage change: +0.43 % in price is a log return of +0.00429.

- **Positive** log return = tomorrow closes higher (UP). **Negative** = lower (DOWN). **Zero** = unchanged ("tomorrow = today").
- **Reverse the formula** to get a price: $P_{t+1} = P_t \cdot \exp(y_t)$. Our prediction is $\hat{P}_{t+1} = P_t \cdot \exp(\hat{y}_t)$, where P_t is the **actual** close of day t.
- **Why logarithms?** Log returns add up over days (the 5-day return is the sum of five daily ones), treat gains and losses symmetrically, and behave like a roughly stationary, mean-zero series even though prices trend.

5.3 A real worked example (Gold, test period)

Step	Value (real, from running `predict_for_date("Gold", "2026-05-12")`)
Day t (the day we stand on)	2026-05-12
Close P_t	$4,677.60
Models see	features up to 2026-05-12 only (30-day window starts 2026-03-31); 84 later rows are hidden
Predicted log return $\hat{y}_t$ (mean of six models)	+0.00076 (= +0.076 %)
Predicted close $\hat{P}_{t+1} = P_t \cdot \exp(\hat{y})$	$4,681.18
Actual close on 2026-05-13	$4,697.70
Actual log return $y_t = \ln(4{,}697.70 / 4{,}677.60)$	+0.00429 (= +0.429 %)
Price error (pred – actual)	$-16.52 (-0.35 %)
Direction	predicted UP, actual UP: a hit

This is one day, picked because it is the example used in the project's own docstring; it says nothing about skill. The "naive" forecast for the same day is $\hat{P} = \$4{,}677.60$ (error −0.43 %): the models were closer here, but over 143 test days they were not closer on average (Chapter 10).

5.4 How the target is created in the code

src/data/preprocessing.py (the target definition)

```
305 def _task_target(log_return, task):
306     """
307     Real-space target series indexed by the date of day t (the last observed day).
308         return_h : sum_{k=1..h} r_{t+k}
309         vol_h    : ln sqrt( mean_{k=1..h} r_{t+k}^2 )
310     Only FUTURE rows enter the target; features at t never see them.
311     """
312     h, kind = TASKS[task]['horizon'], TASKS[task]['kind']
313     r = log_return
314     if kind == 'return':
315         fwd = sum(r.shift(-k) for k in range(1, h + 1))
316     else:
317         fwd = np.log(np.sqrt(sum(r.shift(-k) ** 2 for k in range(1, h + 1)) / h).clip(lower=1e-6))
318     return fwd
```

- `r.shift(-k)` moves the series **backwards**: the value at row t becomes the return of day t+k. For `return_1d` (`h = 1`) the target at row t is therefore the return from t to t+1. This is the **only** place in the pipeline where future data is used, and it is used only as the label.
- `build_dataset` then drops the last row (no tomorrow yet) with `valid = target.notna()`, and uses `features["log_return"]` as the source series.
- A unit test (`test_create_sequences_target_is_next_row_and_never_in_window`) checks that the target is the next row and never inside the input window; another test (`test_true_target_reconstructs_actual_close_exactly`) checks that $P_t \cdot \exp(y_t)$ reproduces the actual close to 1e-9.

5.5 Standardisation of the target

```
src/data/preprocessing.py  (training mean and std only)
374     y_tr_real = target.values[masks['train']]
375     mu, sd = float(y_tr_real.mean()), float(y_tr_real.std() or 1.0)
376     fwd = lambda v: (np.ravel(v) - mu) / sd
377     inv = lambda v: np.ravel(v) * sd + mu
```

The models are trained on $z = (y - \mu) / \sigma$ with μ, σ computed **from the training targets only** (so no validation or test information leaks). `data["inv"]` converts model output back to a real log return. At inference time μ and σ are read from `model_status.json` (`target_mean`, `target_std`); a stale file without them raises an explicit `FileNotFoundError`.

5.6 Why this target is suitable

- **Stationary.** ADF tests (Chapter 4.7) reject a unit root for returns ($p = 0.000$) but not for prices ($p = 0.97$ to 1.00).
- **Makes the baseline explicit.** The random-walk forecast is $\hat{y} = 0$, so "does the model beat tomorrow = today?" becomes "is the model's error smaller than the error of predicting zero?".
- **Comparable across assets.** A return of 1 % means the same for Bitcoin, Gold and Silver; USD prices ($80,000 versus $4,000 versus $60) do not.
- **Avoids a misleading metric.** On price levels, "tomorrow = today" already gets $R^2 \approx 0.95$, so a price-level R^2 would make every model look excellent.
- **Contains both direction and size** in one number.

5.7 How the target relates to the input features

Time	What is known / what is predicted
Up to and including day t	**Inputs.** 30 days of features. The most recent row contains `log_return` = $r_t = \ln(P_t/P_{t-1})$, lagged returns r_{t-1}, r_{t-2}, r_{t-5}, r_{t-10}, volatility estimates, indicators, and macro returns (aligned by the close-time rule)
Day t+1	**Target.** $y_t = r_{t+1} = \ln(P_{t+1}/P_t)$. Never an input
Day t+2 and later	Hidden from the models at prediction time; in `predict_for_date` every row after the chosen day is cut off (`feats.loc[:as_of]`)

So today's return **is** an input and tomorrow's return is the output. Because daily returns have almost zero autocorrelation (Chapter 4.7), this input alone says very little about the output.

5.8 Other targets defined in the code (experiments only)

Task key	Definition	Used for	Outcome
`return_1d`	$y_t = r_{t+1}$	**Served** forecast	No skill over random walk
`return_5d`	$\Sigma_{k=1..5}\, r_{t+k}$ (overlapping targets, exact embargo)	Experiment E3	Walk-forward RMSE within about ±1.4 % of naive. Gold's small gains (−0.4 to −1.0 %) are matched by the historical-mean forecast (−0.7 %), i.e. drift, not model signal
`vol_5d`, `vol_22d`	ln √(mean of squared returns over the next 5 or 22 days)	Experiment E4 (`vol_22d`)	EWMA beats persistence by 4 to 8 %; some models beat persistence for Bitcoin and Gold, none for Silver (R^2 still negative)

Chapter 6: Feature Engineering

A **feature** is one number that describes the situation on day t. The models receive **29 features for Bitcoin, 28 for Gold and 29 for Silver**. This list was read from the processed files and verified against the saved scaler's `feature_names_in_`.

6.1 Policy: three rules every feature follows

1. **Stationary.** Prices, moving averages, Bollinger levels, ATR and MACD in dollars would drift outside the training range (Bitcoin rose from $3k to $124k), so they are converted to ratios or returns. The 13 level columns in `config.LEVEL_COLUMNS` stay in the file for charts only.
2. **Backward-looking.** Every feature at row t uses only rows ≤ t (`rolling`, `ewm`, `shift(+k)`). A unit test changes the last observation and checks that no earlier feature changes; a truncation test on the real data agreed.
3. **Close-time rule.** A feature may only use information that exists when the asset's own close is fixed. This differs for Bitcoin and the metals (6.2).

6.2 The close-time rule (the most important design decision)

Yahoo's daily close for Gold and Silver futures is the **13:25 to 13:30 ET COMEX settlement**, i.e. *before* the S&P 500 (16:00), VIX (16:15), DXY (17:00) and 10-year yield close. If a day-t feature used the same day's S&P return, it would contain news from *inside* the interval the model is asked to forecast. Yahoo's futures High and Low also cover the whole session until 17:00 ET, which is after the settlement. Bitcoin's daily bar closes at 00:00 UTC, after every US market has closed, so same-day macro values are legitimately known.

config.py (the two switches)

```
86 EXTERNAL_SAME_DAY = {'crypto': True, 'commodity': False}
87 POST_SETTLEMENT_FEATURES = ['hl_range', 'atr_norm', 'ADX']
```

`EXTERNAL_SAME_DAY["commodity"] = False` makes `_align_external` pick, for each metals row t, the latest external value from a date **strictly before** t. `POST_SETTLEMENT_FEATURES` lists the High/Low-based features that `lag_post_settlement_features` shifts by one session for the metals. Before this fix (2026-09-13) Silver appeared to reach 62.2 % directional accuracy (p = 0.002); after the fix the served Silver model scored 55.2 % (p = 0.12), not significant. Two dedicated tests keep the rule in place.

6.3 Complete feature dictionary

Return path (6 features, all assets)

Feature	Type	Formula (simple)	Source column(s)	Why it may help	Known at prediction time?
`log_return`	return	$\ln(P_t / P_{t-1})$	`price`	Today's move; tiny negative autocorrelation for Bitcoin (−0.05)	Yes (day t close)
`return_1d`	lag	r_{t-1} = `log_return.shift(1)`	`log_return`	Yesterday's move (momentum / reversal)	Yes
`return_2d`	lag	r_{t-2}	`log_return`	Two days ago	Yes
`return_5d`	lag	r_{t-5}	`log_return`	One trading week ago. **Not** the 5-day sum despite the name	Yes
`return_10d`	lag	r_{t-10}	`log_return`	Two weeks ago. **Not** a 10-day sum	Yes
`return_20d`	rolling sum	$\Sigma_{i=0..19}\, r_{t-i}$ (`rolling(20).sum()`)	`log_return`	Monthly momentum / reversal; top Gold CatBoost feature	Yes (includes day t)

Volatility state (9 features, all assets)

Feature	Type	Formula (simple)	Source column(s)	Why it may help	Known at prediction time?
`volatility_10d`	rolling stat	std of r over the last 10 days	`log_return`	Short-term risk level; volatility clusters	Yes
`volatility_30d`	rolling stat	std of r over the last 30 days	`log_return`	Medium-term risk level	Yes
`rv_1d`	realised vol (HAR)	$\sqrt{(r_t^2)} = \lvert r_t \rvert$	`log_return`	Today's absolute move	Yes
`rv_5d`	realised vol (HAR)	√(mean of r² over 5 days)	`log_return`	Weekly realised volatility (Corsi 2009 HAR model)	Yes
`rv_22d`	realised vol (HAR)	√(mean of r² over 22 days)	`log_return`	Monthly realised volatility	Yes
`ewma_vol`	EWMA vol	$\sigma_t^2 = 0.94 \cdot \sigma_{t-1}^2 + 0.06 \cdot r_t^2$, then √	`log_return`	RiskMetrics volatility; **also the σ_t of the served band**	Yes
`bb_width`	indicator	(Upper − Lower) / Middle of 20-day, 2σ Bollinger Bands	`price`	Relative width of the band = volatility state	Yes
`atr_norm`	indicator	ATR(14) / P_t	`high`, `low`, `price`	Average true range relative to price	Bitcoin: yes. Metals: **lagged one session**
`hl_range`	range	(High − Low) / P_t	`high`, `low`, `price`	Intraday range; volatility proxy	Bitcoin: yes. Metals: **lagged one session**

Momentum and trend (8 features, all assets)

Feature	Type	Formula (simple)	Source column(s)	Why it may help	Known at prediction time?
RSI	indicator	100 − 100 / (1 + RS), RS = Wilder EMA(gain, 14) / EMA(loss, 14)	price	Overbought / oversold level, 0 to 100	Yes
ADX	indicator	Average Directional Index, 14 days (ta)	high, low, price	Trend strength 0 to 100, direction-free	Bitcoin: yes. Metals: **lagged one session**
ROC	indicator	$(P_t / P_{t-12} - 1) \times 100$	price	12-day rate of change	Yes
ema14_ratio	ratio	$P_t / EMA_{14} - 1$	price	Distance from the short trend; top Silver feature	Yes
ema30_ratio	ratio	$P_t / EMA_{30} - 1$	price	Distance from the medium trend	Yes
macd_norm	ratio	$(EMA_{12} - EMA_{26}) / P_t$	price	Scale-free MACD line	Yes
macd_hist_norm	ratio	$(MACD - EMA_9(MACD)) / P_t$	price	Scale-free MACD histogram	Yes
bb_pctb	indicator	$(P_t$ − Lower) / (Upper − Lower)	price	Position inside the Bollinger Bands (about 0 to 1)	Yes

Macro and cross-asset returns

Feature	Type	Formula (simple)	Source column(s)	Why it may help	Known at prediction time?
sp500_return	external return	$\ln(S_t / S_{t-1})$ of ^GSPC	sp500_data.csv	Equity risk appetite. All three assets	Bitcoin: same day. Metals: **previous trading day**
vix_return	external return	ln of ^VIX day-over-day	vix_data.csv	Change in market fear. All three assets	Same rule
dxy_return	external return	ln of DX-Y.NYB day-over-day	dxy_data.csv	Dollar strength (inverse driver of metals). Gold, Silver	Previous trading day
oil_return	external return	ln of CL=F day-over-day	crude_oil_data.csv	Commodity co-movement. Gold, Silver	Previous trading day
tnx_return	external return	ln of ^TNX (10-year yield level) day-over-day	tnx_data.csv	Interest-rate change (opportunity cost of gold). Gold, Silver	Previous trading day
gold_return	cross-asset return	Gold's log return on the same date (forward-filled to Silver's calendar)	gold_data.csv	Gold and silver move together. **Silver only**	Same day (5-minute settlement gap, negligible)

Sentiment, calendar and volume (Bitcoin only, 4 features)

Feature	Type	Formula (simple)	Source column(s)	Why it may help	Known at prediction time?
`fear_greed`	sentiment	Fear & Greed index value 0 to 100 (filled with 50 before 2018-02-01)	`fear_greed_data.csv`	Crypto market mood	Yes (published at 00:00 UTC for the day)
`dow_sin`	calendar	$\sin(2\pi \cdot \text{weekday} / 7)$	index	Weekday seasonality (Bitcoin trades on weekends)	Yes (calendar)
`dow_cos`	calendar	$\cos(2\pi \cdot \text{weekday} / 7)$	index	Same, second coordinate of the circle	Yes (calendar)
`log_volume_change`	volume	$\text{clip}(\ln(V_t / V_{t-1}), -3, +3)$	`volume`	Change in activity	Yes

> **Counting check**
> Return path 6 + volatility 9 + momentum 8 = **23 core features** for every asset. Add `sp500_return` and `vix_return` = 25. Bitcoin adds `fear_greed`, `dow_sin`, `dow_cos`, `log_volume_change` = **29**. Gold adds `dxy_return`, `oil_return`, `tnx_return` = **28**. Silver adds those three and `gold_return` = **29**.

Naming trap for the viva: `return_5d` and `return_10d` are the returns of day t–5 and t–10 (single days, lagged), whereas `return_20d` is a 20-day rolling **sum**. The experiments' prediction *targets* of the same names (`return_5d` as a task) are different objects.

6.4 The technical indicators explained one by one

Only indicators that appear in `src/data/preprocessing.py` are listed. **Not implemented:** Williams %R, CCI, Stochastic, OBV, Ichimoku, Fibonacci levels (an older README claimed some of them; the claim was removed).

Indicator	Plain meaning	Calculation in our code	How to read it
EMA (exponential moving average)	A moving average that weights recent days more	`price.ewm(span=14 / 30, adjust=False).mean()`; feature = P / EMA – 1	Positive: price above its average (uptrend); negative: below
RSI(14)	Strength of recent gains versus losses	Wilder smoothing: `ewm(alpha=1/14)` of gains and of losses; RSI = 100 – 100/(1+RS)	Above 70 often called overbought, below 30 oversold (the dashboard draws both lines)
MACD(12, 26, 9)	Difference between a fast and a slow EMA	$EMA_{12} - EMA_{26}$; signal = EMA_9 of MACD; both divided by price	MACD above its signal: short-term momentum positive
Bollinger Bands(20, 2σ)	A band 2 standard deviations around the 20-day mean	`ta.volatility.BollingerBands(window=20, window_dev=2)`	%B near 1: price at the upper band; width: how volatile the market is
ATR(14)	Average daily trading range including gaps	`ta.volatility.average_true_range(high, low, close, 14)`; feature = ATR / price	Higher = more volatile
ADX(14)	How strong a trend is (any direction)	`ta.trend.ADXIndicator(...).adx()`	Below about 20: no clear trend; above 25: strong trend
ROC(12)	12-day percentage change	`ta.momentum.roc(close, window=12)` (in %)	Simple momentum
HAR realised volatility	Daily, weekly and monthly volatility measured from squared returns	`rv_1d`, `rv_5d`, `rv_22d` as in the table above	Captures that volatility is persistent

Indicator	Plain meaning	Calculation in our code	How to read it
EWMA volatility (RiskMetrics)	Volatility estimate that decays old shocks by 6 % per day	`r².ewm(alpha=0.06, min_periods=22, adjust=False).mean()` then sqrt	Used as σ_t of the uncertainty band

`src/data/preprocessing.py (RSI)`

```
59      def add_rsi(self, window=14):
60          """Wilder-smoothed RSI (0-100)."""
61          delta = self.df['price'].diff()
62          gain = delta.clip(lower=0)
63          loss = -delta.clip(upper=0)
64          ema_gain = gain.ewm(alpha=1 / window, min_periods=window).mean()
65          ema_loss = loss.ewm(alpha=1 / window, min_periods=window).mean()
66          rs = ema_gain / ema_loss
67          self.df['RSI'] = 100 - (100 / (1 + rs))
```

Mini example (RSI). If over 14 days the smoothed average gain is 0.8 and the smoothed average loss is 0.4, then RS = 2 and RSI = 100 − 100/3 = 66.7 (fairly strong recent buying). Equal gains and losses give RS = 1 and RSI = 50, which is the value Gold had in our worked example below (48.9).

6.5 One real feature row (Gold, 2026-05-12, frozen dataset)

Feature	Raw value	Scaled (0 to 1)	Feature	Raw value	Scaled (0 to 1)
log_return	-0.0087482	0.389	return_5d	0.0079997	0.543
RSI	48.924	0.423	return_10d	-0.018108	0.303
ADX	13.441	0.100	return_20d	-0.031026	0.265
ROC	-0.94657	0.385	volatility_10d	0.014187	0.351
ema14_ratio	0.0013991	0.524	volatility_30d	0.014808	0.474
ema30_ratio	-0.0044534	0.390	rv_1d	0.0087482	0.151
macd_norm	-0.0044274	0.333	rv_5d	0.013083	0.321
macd_hist_norm	0.0025669	0.712	rv_22d	0.012753	0.375
bb_pctb	0.47356	0.514	ewma_vol	0.015977	0.524
bb_width	0.075169	0.330	dxy_return	0.0010216	0.593
atr_norm	0.01773	0.504	oil_return	0.027393	0.514
hl_range	0.0022887	0.029	tnx_return	0.010486	0.476
return_1d	-0.00036014	0.466	sp500_return	0.0018782	0.593
return_2d	0.0043736	0.509	vix_return	0.066935	0.511

The **scaled** column is what the models receive: (raw − $\min_{train}$) / ($\max_{train}$ − $\min_{train}$). For example `hl_range` is 0.0023 raw (the previous session's high-low range was only 0.23 % of price, a small value for gold), which scales to 0.029. Tabular models receive exactly these 28 numbers; GRU and LSTM receive the 30 most recent such rows.

6.6 Which features matter, and what the evidence says

Asset	Model	Top-4 features (importance)
Bitcoin	CatBoost	`return_1d` (0.08), `dow_sin` (0.08), `log_return` (0.08), `sp500_return` (0.07)
Bitcoin	LightGBM	`atr_norm` (0.07), `rv_1d` (0.06), `sp500_return` (0.06), `log_return` (0.05)
Bitcoin	RandomForest	`atr_norm` (0.19), `sp500_return` (0.08), `RSI` (0.07), `fear_greed` (0.07)
Bitcoin	Ridge	`fear_greed` (0.09), `log_return` (0.08), `RSI` (0.08), `bb_pctb` (0.07)
Gold	CatBoost	`return_20d` (0.20), `tnx_return` (0.08), `dxy_return` (0.08), `return_1d` (0.05)
Gold	LightGBM	`return_5d` (0.09), `oil_return` (0.09), `atr_norm` (0.09), `macd_hist_norm` (0.09)
Gold	RandomForest	`return_20d` (0.13), `rv_5d` (0.09), `bb_width` (0.09), `return_5d` (0.07)
Gold	Ridge	`return_20d` (0.11), `macd_hist_norm` (0.10), `return_5d` (0.09), `ema14_ratio` (0.08)
Silver	CatBoost	`ema14_ratio` (0.44), `return_20d` (0.13), `gold_return` (0.11), `dxy_return` (0.05)
Silver	LightGBM	`macd_hist_norm` (0.39), `volatility_30d` (0.16), `bb_width` (0.11), `atr_norm` (0.10)
Silver	RandomForest	`bb_width` (0.15), `macd_norm` (0.12), `macd_hist_norm` (0.08), `rv_5d` (0.06)
Silver	Ridge	`log_return` (0.11), `hl_range` (0.08), `ema14_ratio` (0.07), `ADX` (0.07)

Importance is the model's normalised gain (trees) or absolute coefficient (Ridge) from `results/feature_importance.csv`. GRU and LSTM expose no importance. The lists **disagree between models** (for Bitcoin, `return_1d`, `dow_sin`, `atr_norm`, `fear_greed` each lead one model), which is what you expect when the inputs carry almost no signal.

- **Ablation (E2).** Dropping any one feature group (return path, volatility, momentum, macro, sentiment/calendar/volume) changes walk-forward RMSE by at most **0.39 %** in absolute value (Ridge and LightGBM, three assets). No group is indispensable.
- **Redundancy.** 15 (Bitcoin), 18 (Gold) and 17 (Silver) feature pairs have |correlation| > 0.85 on the training window (e.g. `ema30_ratio` with `RSI`, `ewma_vol` with `rv_22d`). The audit tested removing 10 redundant features (E5b); it was **not better**, so all features were kept.
- **Meaning.** The features are sensible and leakage-free, but they do not contain out-of-sample return signal; volatility features *are* informative about the size of tomorrow's move.

6.7 How features differ across assets and models

	Bitcoin	Gold	Silver
Number of features	29	28	29
Calendar / volume / sentiment	`dow_sin`, `dow_cos`, `log_volume_change`, `fear_greed`	none	none
Macro returns	`sp500_return`, `vix_return` (same day)	`sp500`, `vix`, `dxy`, `oil`, `tnx` (previous day)	same five plus `gold_return`
Lagged High/Low features	no	yes (3)	yes (3)
Window for GRU / LSTM	30 × 29	30 × 28	30 × 29

- **Tabular models** (Ridge, Random Forest, LightGBM, CatBoost) use only the **last row** of the window: `Xt = X[:, -1, :]`. Their lagged information is inside the lag and rolling features.
- **Recurrent models** (GRU, LSTM) use the **full 30-day window** and learn their own temporal patterns from the same columns.

- All six models receive the same scaled columns in the same order; inference re-orders columns by `scaler.feature_names_in_` and rejects a mismatch by name.

Chapter 7: Machine Learning Fundamentals Used in Our Project

Only concepts that the implementation actually relies on are covered. Each one has a plain definition, an example from **our** project, and the more technical view.

1. Machine learning (ML)

In simple words: Instead of writing rules, we show a program many examples and let it find the rules itself.

In our project: We show models about 1,900 (metals) to 2,750 (Bitcoin) days of "situation → what happened next" and they learn a rule that maps today's features to tomorrow's return.

Technically: ML fits a function $f_\theta(x)$ whose parameters θ minimise a loss on training data, hoping the function also works on new data.

2. Supervised learning

In simple words: Learning from examples that come with the right answer.

In our project: Every training example has inputs (28 or 29 features of day t) and a known label (the return of day t+1).

Technically: Given pairs (x_i, y_i), learn f so that $f(x_i) \approx y_i$. The labels here are not given by humans; they come from the future of the same series.

3. Regression

In simple words: Predicting a number (not a category).

In our project: We predict a number, the log return, e.g. +0.0764 %. UP / DOWN is derived afterwards from the sign.

Technically: The output is a real value; squared error (MSE) is the standard loss. Classification would predict a class label directly (not used here).

4. Time-series forecasting and stationarity

In simple words: Data in time order, where the order carries information and must be respected.

In our project: Prices are ordered by date; we always train on the past and test on the future. Prices wander (non-stationary); returns do not, so we predict returns.

Technically: A series is (weakly) stationary if its mean and variance do not drift. The ADF test rejects a unit root for returns ($p = 0.000$) but not for prices ($p \geq 0.97$).

5. Features and target

In simple words: Features are the inputs the model looks at; the target is what it must predict.

In our project: Features: `log_return`, `RSI`, `ewma_vol`, `sp500_return`, … (Chapter 6). Target: the next-day log return (Chapter 5).

Technically: $x \in R^F$ (tabular models) or $X \in R^{30 \times F}$ (recurrent models); $y \in R$ (standardised for training).

6. Training, validation and test data

In simple words: Training data teaches; validation data helps choose settings; test data gives the final honest grade, used once.

In our project: Train ≤ 2025-09-10, validation ≤ 2026-02-17, test = the last 143 to 207 days. Inside train+validation we also run 4-fold walk-forward validation.

Technically: Splits are by time, never random. The test set must not influence any decision (tuning, early stopping, model choice, ensemble weights), otherwise its score is optimistic.

7. Model fitting (training)

In simple words: Adjusting the model's internal numbers until its predictions on the training examples are good.

In our project: Ridge learns coefficients, trees learn split rules and leaf values, GRU/LSTM learn weight matrices by gradient descent (Adam).

Technically: Minimise the training loss $L(\theta) = (1/n)\ \Sigma\ (z_i - f_\theta(x_i))^2$ plus regularisation, by closed form (Ridge), greedy tree building (RF), boosting (LightGBM, CatBoost) or backpropagation (GRU, LSTM).

8. Predictions (inference)

In simple words: Using the trained model on new inputs.

In our project: `predict_next_day` scales the newest 30 rows, calls each model's `predict`, and converts the output back to a return and then a price.

Technically: No parameters change at inference time. The same scaler and the same column order as in training are mandatory.

9. Loss functions and error metrics

In simple words: A loss is a score for how wrong a prediction is; training tries to make it small.

In our project: All six models minimise **squared error** on the standardised return. We report RMSE, MAE, MAPE, R^2 and direction accuracy to describe the result (Chapter 10).

Technically: $MSE = (1/n)\ \Sigma\ (y - \hat{y})^2$; $RMSE = \sqrt{MSE}$; $MAE = (1/n)\ \Sigma\ |y - \hat{y}|$. Squared error punishes big misses more, and is the loss behind the Diebold-Mariano test.

10. Hyper-parameters

In simple words: Settings chosen **before** training (not learned by the model), such as tree depth or learning rate.

In our project: Ridge `alpha`, forest `max_depth`, boosting `learning_rate`, `num_leaves`, GRU `dropout`, etc. They come from `results/tuning/best_params.json`, found by walk-forward search on train+validation only.

Technically: Parameters θ are learned from data; hyper-parameters λ control capacity and regularisation and must be selected on data that is not the final test.

11. Overfitting and underfitting

In simple words: Overfitting: memorising noise, great on the training days and poor on new ones. Underfitting: too simple to learn anything.

In our project: LightGBM on Bitcoin reached train $R^2 = 0.335$ and 71 % train direction accuracy but validation $R^2 = 0.04$: memorisation. Heavy regularisation and early stopping controlled it.

Technically: Bias-variance trade-off: flexible models have low bias, high variance. With signal-to-noise near zero, extra capacity only adds variance, which is why our models are small and strongly regularised.

12. Generalisation

In simple words: Doing well on data the model has never seen.

In our project: The only honest evidence is performance on the untouched test days and on out-of-fold walk-forward days. Here every model generalises *as well as* the random walk and no better.

Technically: Estimated by out-of-sample error. A model that generalises beats a baseline on new data with statistical significance.

13. Regularisation and early stopping

In simple words: Gentle brakes that stop a model from fitting noise.

In our project: Ridge shrinks coefficients (`alpha = 100`); trees have minimum leaf sizes and shallow depth; boosting and the neural nets stop when validation error stops improving (GRU/LSTM often after **1 to 9 epochs**); dropout 0.2 to 0.4.

Technically: L2 penalty $\alpha\|w\|^2$, subsampling, min-child-samples, `early_stopping_rounds`, and Keras `EarlyStopping(monitor="val_loss", restore_best_weights=True)`.

14. Feature importance

In simple words: Which inputs the model leaned on most.

In our project: Reported for Ridge, Random Forest, LightGBM and CatBoost in `results/feature_importance.csv`; the lists disagree, a sign of weak signal.

Technically: Trees: normalised total split gain; Ridge: |coefficient|; neural networks: not available here.

15. Scaling and normalisation

In simple words: Putting every input on a similar scale (here 0 to 1).

In our project: `MinMaxScaler` fitted on the training rows. The target is separately standardised to mean 0, std 1.

Technically: $x' = (x - \min) / (\max - \min)$. The statistics must come from training data only.

16. Data leakage

In simple words: Accidentally letting the model see information it would not have in real life, which makes results look better than they are.

In our project: The classic case in this project: the metals' daily close is at 13:30 ET but same-day S&P and VIX closes happen later. Using them gave a **fake 62 % direction accuracy for Silver**. Fixed by the close-time rule; the true result is null.

Technically: Leakage types: target in inputs, future rolling windows, scaler fitted on all data, tuning on test, shuffling time series, vendor timing. Chapter 9 lists each check and where the code prevents it.

17. Walk-forward (time-series) cross-validation

In simple words: Repeatedly train on the past, validate on the next block, then move forward.

In our project: 4 expanding folds inside train+validation (582 or 396 samples per block); also pooled into one long out-of-sample record of 2,328 or 1,584 days.

Technically: Unlike random k-fold, the training set always precedes the validation block, mimicking real use (`sklearn.model_selection.TimeSeriesSplit`).

18. Ensembles

In simple words: Asking several models and combining their answers.

In our project: The served forecast is the **equal-weight mean of six models' predicted returns**. A stacked ensemble with learned weights was tested as an experiment.

Technically: Averaging reduces the variance of the errors if the members are not perfectly correlated. If all members carry zero signal, the average also carries none (Chapter 11).

19. Baselines and the random walk

In simple words: The simplest forecast you must beat to claim the model is useful.

In our project: "Tomorrow = today" (predicted return 0). Also "always predict the historical mean return" and ARIMA.

Technically: If asset returns are unpredictable (efficient-market hypothesis), the best forecast of the next price is the current price. Model skill = significantly lower error than this baseline.

20. Statistical significance (p-values, DM test, binomial test)

In simple words: A way to ask "could this difference just be luck?".

In our project: A 3 % lower error over 143 days may be luck. The **Diebold-Mariano test** compares two forecasts' squared errors; the **binomial test** asks if a hit-rate above 50 % is more than luck.

Technically: DM statistic = $\text{mean}(d_t) / \sqrt{(\text{Var}(d_t)/n)}$ with $d_t = e_{A,t}^2 - e_{B,t}^2$, small-sample corrected; $p < 0.05$ would indicate a real difference. Binomial: P(hits ≥ observed | p = 0.5).

21. Signal-to-noise ratio (why this problem is hard)

In simple words: How much of the movement is pattern versus random noise.

In our project: The Combined forecast moves by only 0.12 % (Bitcoin), 0.09 % (Gold), 0.09 % (Silver) standard deviation while actual returns move by 2.08 %, 1.67 %, 3.19 %. Correlation of forecast with outcome: +0.09, +0.16, +0.18.

Technically: If $R^2 \approx 0$, a model that outputs a nearly constant value is as good as any. Our models learned exactly that: small, almost constant predictions. The test-window correlations (computed while writing this guide) are weakly positive; only Silver's is nominally significant (p = 0.03, Gold p = 0.06, Bitcoin p = 0.19, unadjusted for testing several hypotheses), and on the 1,584 pooled out-of-fold days it falls to +0.018 (p = 0.47). With an amplitude 35 times smaller than the real moves it does not lower RMSE (Silver DM p = 0.46), so we do not call it skill.

Chapter 8: Every Machine Learning Model

The registry (`src/models/registry.py`) contains **nine models** behind one interface (`make_model(name, params)`; `fit`, `predict`, `save`, `load`), and `src/models/ensemble_model.py` adds a tenth, the stacked ensemble. Their roles are different, so the first table separates them.

Model	Type	Role in the final prediction pipeline	Trained and saved?
Naive (random walk)	Baseline, $\hat{y} = 0$	Reference only (evaluation, dashboard comparison)	No (nothing to learn)
Naive-Mean (drift)	Baseline, $\hat{y}$ = mean return	Reference only	Fitted inside each evaluation; not saved
ARIMA(p,0,q)	Statistical baseline	Reference only	Fitted inside each evaluation; not saved
Ridge	Linear regression with L2 penalty	**Member** of the Combined forecast	Yes, `<prefix>_ridge.pkl`
Random Forest	Bagged decision trees	**Member**	Yes, `<prefix>_randomforest.pkl`
LightGBM	Gradient-boosted trees	**Member**	Yes, `<prefix>_lightgbm.pkl`
CatBoost	Gradient-boosted symmetric trees	**Member**	Yes, `<prefix>_catboost.cbm` + `.json`
GRU	Recurrent neural network	**Member**	Yes, `<prefix>_gru.keras` + `.json`
LSTM	Recurrent neural network	**Member**	Yes, `<prefix>_lstm.keras` + `.json`
Stacked ensemble	Non-negative Ridge on 4 models' predictions	Experiment, **not** served	Yes, `<prefix>_stack.pkl`
Combined	Equal-weight mean of the six members	**The served forecast**	No file; computed at prediction time

Prefixes are `btc`, `gold`, `silver`; the 6 members x 3 assets give the **18 trained models** in `data/models/`. Where a number is shown for "test", "walk-forward" or "pooled", it is read from `results/final_test_results.csv`, `cv_results.csv` or `walkforward_pooled.csv`.

8.1 Naive (random walk): the baseline every model must beat

1. Type / what it does	Baseline. Predicts that tomorrow's return is **zero**, i.e. tomorrow's price equals today's.
2. Simple English	"Whatever the price is now, it will be the same tomorrow."
3. Technical	$\hat{y}_t = 0 \Rightarrow \hat{P}_{t+1} = P_t$. Its RMSE equals the root mean square of the true returns. Under the efficient-market hypothesis it is the optimal forecast.
4. Why selected	It is the most demanding fair opponent: on prices it already scores $R^2 \approx 0.95$ and MAPE ≈ 1.3 to 2.5 %.
5. Input / target	None (constant). Implemented as `NaiveModel` (constant 0.0); evaluation uses `data["naive_<split>"]`, an array of zeros for return tasks.
6. Hyper-parameters	None.
7. Training	None.
8. Evaluation	Appears in every table as the reference row. Direction accuracy is "—" because sign(0) is undefined.
9. Strengths	Cannot overfit; costs nothing; very hard to beat.
10. Weaknesses	Gives no direction and no information about tomorrow.
11. Role	The yardstick for "skill".
12. Comparison	Every other model is expressed as a percentage above or below its RMSE.

8.2 Naive-Mean (historical mean, "drift")

1. Type / what it does	Baseline. Predicts the **average training return** every day.
2. Simple English	"Prices rose on average in the past, so say they will rise a little every day."
3. Technical	$\hat{y}_t = \text{mean}(z_{\text{train}}) \cdot \sigma + \mu$, a constant (`NaiveMean.fit` stores `np.mean(y_tr)`). In walk-forward each fold uses the mean of its own training part; for the test it uses train+validation.
4. Why selected	It separates **drift** from **timing**: an asset that rose gets an "always UP" forecast, which already scores the share of up days as direction accuracy.
5. Input / target	None / standardised next-day return.
8. Evaluation	**Test** RMSE vs random walk: Bitcoin -0.05 % (DM p 0.81) · Gold +0.26 % (DM p 0.43) · Silver +0.10 % (DM p 0.63). **Walk-forward** (4 folds): Bitcoin +0.11 % · Gold -0.12 % · Silver -0.01 %. **Pooled walk-forward** DM p vs random walk: Bitcoin 0.025 · Gold 0.070 · Silver 0.507.
9-10. Strengths / weaknesses	Strength: honest benchmark for direction hit-rates. Weakness: no information about any particular day.
11. Role	Used in `walkforward_pooled.csv` as the "drift" forecast: a model's direction accuracy must beat **this**, not 50 %, to show timing skill (it does not for Gold or Silver).

8.3 ARIMA (statistical time-series baseline)

1. Type / what it does	Autoregressive integrated moving-average model, here **ARIMA(p, 0, q)** because returns are already the differenced series.
2. Simple English	"Tomorrow's return is a weighted mix of the last few returns and the last few surprises."
3. Technical	$y_t = c + \Sigma_{i=1..p} \varphi_i y_{t-i} + \Sigma_{j=1..q} \theta_j \varepsilon_{t-j} + \varepsilon_t$. Fitted by maximum likelihood (`statsmodels.tsa.arima.model.ARIMA`).
4. Why selected	The classical time-series benchmark; if linear autocorrelation existed, ARIMA would find it.
5. Input / target	Only the standardised return series itself (no other features).
6. Hyper-parameters	Order grid p, q ∈ {0,1,2,3}, d = 0, chosen by **lowest AIC** on the training window. Orders for the test run: Bitcoin (2, 0, 0), Gold (1, 0, 1), Silver (2, 0, 3).
7. Training / forecasting	`ARIMAModel.fit_series` then `forecast_walk_forward`: forecast one step, append the **true** value after forecasting, repeat (no refit, no look-ahead).
8. Evaluation	**Test** RMSE vs random walk: Bitcoin +0.56 % (DM p 0.35) · Gold +0.62 % (DM p 0.52) · Silver +1.14 % (DM p 0.50). **Walk-forward** (4 folds): Bitcoin +0.14 % · Gold +0.77 % · Silver +0.76 %.
9-10. Strengths / weaknesses	Strength: interpretable, fast. Weakness: linear, univariate; daily returns have ACF lag 1 of only −0.02 to −0.05.
11. Role	Baseline. It is the worst or nearly worst model in most tables (+0.6 to +1.1 % above naive on test).

8.4 Ridge regression

1. Type / what it does	Linear regression with an L2 penalty (`sklearn.linear_model.Ridge`).
2. Simple English	"Draw the best straight-line (weighted-sum) rule from the features to tomorrow's return, but keep the weights small so the rule does not chase noise."
3. Technical	$\hat{w} = \text{argmin}_w \Sigma_i (z_i - w \cdot x_i - b)^2 + \alpha \lVert w \rVert^2$. Closed form $\hat{w} = (X^T X + \alpha I)^{-1} X^T z$. Larger α shrinks weights towards 0, so the prediction approaches the training mean.
4. Why selected	The simplest learned model: if even a linear rule shows no skill, complex models are unlikely to either. Fast and stable.
5. Input / target	Last row `Xt` (28 or 29 scaled features) → standardised next-day return.
6. Hyper-parameters	`alpha` = 100.0 for all three assets (grid 0.1, 1, 10, 100: the **largest** value won, so the search wanted even more shrinkage, another sign that there is little signal).
7. Training	One fit; no stopping rule. Phase B refits on train+validation with the same `alpha`.
8. Evaluation	**Test** RMSE vs random walk: Bitcoin +0.04 % (DM p 0.91) · Gold +0.33 % (DM p 0.76) · Silver -0.27 % (DM p 0.48). **Walk-forward** (4 folds): Bitcoin +0.03 % · Gold -0.18 % · Silver +0.05 %. **Pooled walk-forward** DM p vs random walk: Bitcoin 0.682 · Gold 0.050 · Silver 0.737.
9. Strengths	Transparent, no randomness, cannot overfit much with α = 100.
10. Weaknesses	Linear only; 28 to 29 correlated inputs (15 to 18 highly correlated pairs).
11. Role	Member of Combined. Importance is its absolute coefficient.
12. Comparison	Statistically indistinguishable from the other members. Gold walk-forward: Ridge -0.18 % vs naive, the drift forecast -0.12 % (nearly the same, so the small gain is drift).

8.5 Random Forest

1. Type / what it does	Bagged ensemble of regression trees (`RandomForestRegressor`).
2. Simple English	"Grow 300 small decision trees on different random samples of the days and average their answers."
3. Technical	$\hat{y}(x) = (1/B)\,\Sigma_{b=1..B}\,T_b(x)$. Each tree is grown on a bootstrap sample by repeatedly choosing the split that most reduces the squared error; averaging reduces variance. `max_depth` and `min_samples_leaf` limit complexity.
4. Why selected	A non-linear model that is hard to over-tune and handles correlated features.
5. Input / target	Last row `Xt` → standardised target.
6. Hyper-parameters	`n_estimators`=300, `max_depth`=3, `min_samples_leaf`=30 for all assets (both at the edge of the grid: the shallowest depth and the largest leaf won), `random_state`=42, `n_jobs`=-1.
7. Training	One fit on train (phase A) or train+validation (phase B).
8. Evaluation	**Test** RMSE vs random walk: Bitcoin -0.23 % (DM p 0.62) · Gold -0.73 % (DM p 0.57) · Silver -1.36 % (DM p 0.17). **Walk-forward** (4 folds): Bitcoin +0.01 % · Gold +0.17 % · Silver +0.19 %. **Pooled walk-forward** DM p vs random walk: Bitcoin 0.996 · Gold 0.757 · Silver 0.649.
9. Strengths	Robust, little tuning, gives feature importance.
10. Weaknesses	Cannot extrapolate beyond training values; predictions of depth-3 trees on noise are almost constant.
11. Role	Member of Combined. Best single model on the *test* set for Gold and Silver (hindsight; it was not best on validation).
12. Comparison	File size 270 to 300 KB (largest of the saved models); the only member whose test ranking differs strongly from its validation ranking.

8.6 LightGBM (gradient-boosted trees)

1. Type / what it does	Gradient-boosted decision trees (`lightgbm.LGBMRegressor`), leaf-wise growth with histogram binning.
2. Simple English	"Build trees one after another; each new tree tries to fix the mistakes of the trees before it."
3. Technical	$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$, where h_m is fitted to the negative gradient of the squared loss (the residuals) and each leaf value is $w = -G / (H + \lambda)$ with λ = `reg_lambda`. η is `learning_rate`; `num_leaves` and `min_child_samples` bound complexity.
4. Why selected	State-of-the-art on tabular data; fast; strong regularisation controls available.
5. Input / target	Last row `Xt` → standardised target.
6. Hyper-parameters (tuned)	Bitcoin: `n_estimators`=600, `learning_rate`=0.03, `num_leaves`=31, `min_child_samples`=20, `reg_lambda`=0.0, `colsample_bytree`=0.8, `subsample`=0.8 Gold: `n_estimators`=600, `learning_rate`=0.01, `num_leaves`=31, `min_child_samples`=100, `reg_lambda`=5.0, `colsample_bytree`=0.8, `subsample`=0.8 Silver: `n_estimators`=600, `learning_rate`=0.01, `num_leaves`=8, `min_child_samples`=50, `reg_lambda`=5.0, `colsample_bytree`=0.8, `subsample`=0.8 `subsample`=0.8 is **inert** (LightGBM only bags rows when `subsample_freq` > 0, left at 0); documented in `registry.py`, the reported models were trained this way.
7. Training	Phase A: early stopping with 30 patience rounds on validation to find the tree count. Phase B: refit on train+validation with that count fixed. Trees used: Bitcoin 81; Gold 10; Silver 10 (10 is the code's floor `max(10, best_iteration)`, so the metals' models are almost constant).
8. Evaluation	**Test** RMSE vs random walk: Bitcoin -0.53 % (DM p 0.79) · Gold +0.05 % (DM p 0.88) · Silver -0.01 % (DM p 0.98). **Walk-forward** (4 folds): Bitcoin -0.05 % · Gold -0.12 % · Silver -0.08 %. **Pooled walk-forward** DM p vs random walk: Bitcoin 0.862 · Gold 0.061 · Silver 0.416.
9. Strengths	Flexible, fast, quick early stopping.
10. Weaknesses	Easily memorises noise: Bitcoin train R^2 0.335 and train direction accuracy 71.5 % versus validation R^2 0.04.
11. Role	Member of Combined.
12. Comparison	Best single model on the Bitcoin *test* set among the six (−0.53 %) but not significant (DM p 0.79) and not best on validation.

8.7 CatBoost (gradient boosting with symmetric trees)

1. Type / what it does	Gradient boosting (`catboost.CatBoostRegressor`) using **oblivious (symmetric) trees** and ordered boosting.
2. Simple English	Like LightGBM, but each tree asks the same question at each level, which makes trees simpler and the model harder to overfit.
3. Technical	Same additive scheme as LightGBM; all nodes at one depth share a split, so a depth-d tree has 2^d leaves. Ordered boosting computes each residual with a model that did not see that row, reducing target leakage inside training. `l2_leaf_reg` is the leaf-value L2 penalty.
4. Why selected	Often the most robust boosting library on small, noisy data.
5. Input / target	Last row `Xt` → standardised target.
6. Hyper-parameters (tuned)	Bitcoin: `iterations`=800, `learning_rate`=0.03, `depth`=7, `l2_leaf_reg`=3.0 Gold: `iterations`=800, `learning_rate`=0.01, `depth`=3, `l2_leaf_reg`=10.0 Silver: `iterations`=800, `learning_rate`=0.01, `depth`=3, `l2_leaf_reg`=10.0 `random_seed`=42, `verbose`=0.
7. Training	Phase A: `early_stopping_rounds`=50 on validation with `use_best_model=True`; Phase B: refit on train+validation with the best iteration count. Iterations used: Bitcoin 50, Gold 606, Silver 82.
8. Evaluation	**Test** RMSE vs random walk: Bitcoin +0.13 % (DM p 0.74) · Gold +0.62 % (DM p 0.71) · Silver +0.10 % (DM p 0.70). **Walk-forward** (4 folds): Bitcoin -0.13 % · Gold -0.35 % · Silver -0.02 %. **Pooled walk-forward** DM p vs random walk: Bitcoin 0.448 · Gold 0.283 · Silver 0.612.
9. Strengths	Best single model by walk-forward validation for Bitcoin and Gold (selected as `primary_model`).
10. Weaknesses	The advantage is tiny (Bitcoin CV RMSE 0.03156 vs naive 0.03160) and not significant.
11. Role	Member of Combined; reported as "best single model by validation" next to the served forecast.
12. Comparison	File format `.cbm` (CatBoost native) plus a `.json` sidecar with params and iteration count.

8.8 GRU (Gated Recurrent Unit)

`src/models/model_gru.py` **(the whole architecture)**

```
14 def build_gru_model(seq_len, n_features, units=32, dropout=0.2, learning_rate=0.001):
15     model = Sequential([
16         tf.keras.layers.Input(shape=(seq_len, n_features)),
17         GRU(units),
18         Dropout(dropout),
19         Dense(16, activation='relu'),
20         Dense(1),
21     ])
22     model.compile(optimizer=Adam(learning_rate=learning_rate), loss='mse')
23     return model
```

1. Type / what it does	Recurrent neural network (deep learning). Reads the last 30 days **in order** and produces one number.
2. Simple English	A network with a memory that moves day by day through the 30-day window and decides, with "gates", what to keep and what to forget; the final memory is turned into the prediction.
3. Technical	$z_t = \sigma(W_z x_t + U_z h_{t-1})$; $r_t = \sigma(W_r x_t + U_r h_{t-1})$; $\tilde{h}_t = \tanh(W x_t + U(r_t \odot h_{t-1}))$; $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$. Then Dropout, Dense(16, ReLU), Dense(1, linear). Loss: MSE; optimiser: Adam.
4. Why selected	The standard deep-learning model for sequences; includes the possibility of non-linear temporal patterns that tabular models cannot see.

5. Input / target	Window `Xseq` of shape (30, F) → standardised next-day return.
6. Hyper-parameters (tuned)	Bitcoin: `units`=32, `dropout`=0.4, `learning_rate`=0.001, `batch_size`=64, `epochs`=40, `patience`=6 Gold: `units`=32, `dropout`=0.4, `learning_rate`=0.001, `batch_size`=64, `epochs`=40, `patience`=6 Silver: `units`=32, `dropout`=0.4, `learning_rate`=0.001, `batch_size`=64, `epochs`=40, `patience`=6 Trainable parameters: Bitcoin 6593, Gold 6497, Silver 6593 (counted from the saved models).
7. Training	`shuffle=False` (time order kept), seeds set to 42 before every fit, `EarlyStopping(monitor="val_loss", patience=6, restore_best_weights=True)`. Phase A finds the best epoch; Phase B retrains on train+validation for exactly that many epochs. **Epochs used: Bitcoin 1, Gold 8, Silver 9.**
8. Evaluation	**Test** RMSE vs random walk: Bitcoin +0.44 % (DM p 0.35) · Gold -0.44 % (DM p 0.58) · Silver -0.01 % (DM p 0.99). **Walk-forward** (4 folds): Bitcoin +0.24 % · Gold -0.01 % · Silver -0.18 %. **Pooled walk-forward** DM p vs random walk: Bitcoin 0.037 · Gold 0.622 · Silver 0.423.
9. Strengths	Can model order and persistence in the window; Silver's best single model by validation (–0.17 % walk-forward).
10. Weaknesses	With 1 to 9 useful epochs and a near-zero signal it behaves like a constant predictor; for Bitcoin it is significantly **worse** than the random walk on the pooled record (DM p = 0.037).
11. Role	Member of Combined.
12. Comparison	Not better than the simple models; deep learning gives no advantage here.

8.9 LSTM (Long Short-Term Memory)

1. Type / what it does	Recurrent neural network with a separate cell state; same shape as the GRU for a fair comparison (`build_lstm_model`).
2. Simple English	Like the GRU but with a longer-term memory lane and three gates (input, forget, output).
3. Technical	$i_t = \sigma(W_i x_t + U_i h_{t-1})$, $f_t = \sigma(\ldots)$, $o_t = \sigma(\ldots)$, $g_t = \tanh(\ldots)$; $c_t = f_t \odot c_{t-1} + i_t \odot g_t$; $h_t = o_t \odot \tanh(c_t)$. Then Dropout, Dense(16, ReLU), Dense(1).
4. Why selected	The other classic sequence model; GRU vs LSTM tests whether architecture matters.
5. Input / target	Same window and target as the GRU.
6. Hyper-parameters (tuned)	Bitcoin: `units`=32, `dropout`=0.4, `learning_rate`=0.001, `batch_size`=64, `epochs`=40, `patience`=6 Gold: `units`=32, `dropout`=0.2, `learning_rate`=0.001, `batch_size`=64, `epochs`=40, `patience`=6 Silver: `units`=32, `dropout`=0.2, `learning_rate`=0.001, `batch_size`=64, `epochs`=40, `patience`=6 Trainable parameters: Bitcoin 8481, Gold 8353, Silver 8481.
7. Training	Identical procedure to the GRU. **Epochs used: Bitcoin 2, Gold 1, Silver 4.**
8. Evaluation	**Test** RMSE vs random walk: Bitcoin +0.13 % (DM p 0.35) · Gold +0.96 % (DM p 0.28) · Silver +0.27 % (DM p 0.63). **Walk-forward** (4 folds): Bitcoin +0.31 % · Gold +0.12 % · Silver +0.05 %. **Pooled walk-forward** DM p vs random walk: Bitcoin 0.004 · Gold 0.541 · Silver 0.798.
9-10. Strengths / weaknesses	Strength: longest memory. Weakness: the weakest member for Bitcoin (pooled DM p = 0.004, significantly worse than the random walk); the validation loss is smallest after 1 or 2 epochs, so it cannot learn anything useful.
11. Role	Member of Combined.

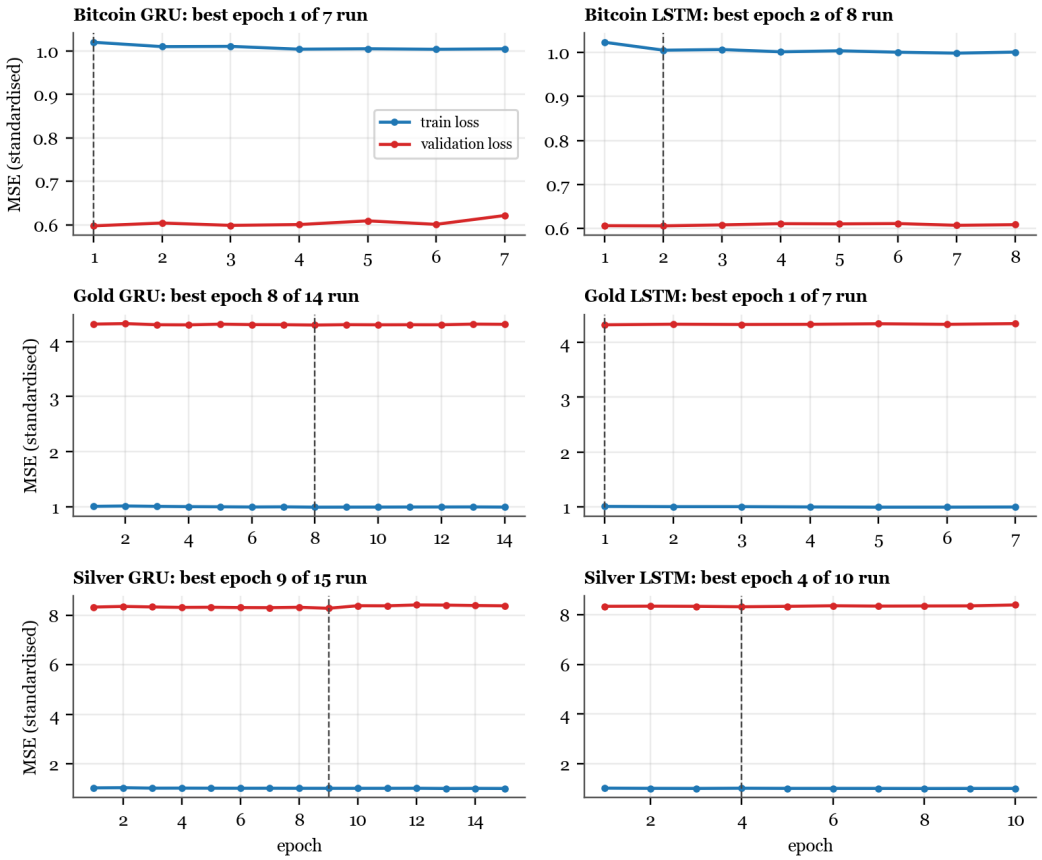


Figure 8.1: GRU and LSTM training versus validation loss (phase A, from `results/histories/*.json`). The best epoch (dashed) comes after very few epochs: the networks have almost nothing to learn.

8.10 Stacked ensemble (experiment, not served)

1. Type / what it does	Meta-model: a non-negative Ridge regression (`Ridge(alpha=1.0, positive=True)`) fitted on the **out-of-fold** predictions of Ridge, LightGBM, CatBoost and GRU.
2. Simple English	Instead of averaging equally, learn how much to trust each model, but never let a weight go negative.
3. Technical	$\hat{y} = w_0 + \Sigma_k w_k \hat{y}_k$, $w_k \geq 0$. If the base models carry no out-of-sample signal, the weights fit noise.
6. Fitted weights	Bitcoin: LightGBM 0.39, CatBoost 0.31, others 0. Gold: Ridge 0.08, CatBoost 1.18, GRU 0.59. Silver: LightGBM 0.55, GRU 0.99. (Weights above 1 mean the stack amplifies a member; for Gold it is the **worst** model on test.)
8. Evaluation	**Test** RMSE vs random walk: Bitcoin -0.72 % (DM p 0.44) · Gold +2.02 % (DM p 0.44) · Silver +0.06 % (DM p 0.91). (no walk-forward row: it is built from the same out-of-fold predictions).
11. Role	Kept as a reported experiment; excluded from Combined because it is itself a blend of some of its members.

8.11 Side-by-side comparison

Model	Bitcoin: test RMSE vs naive (DM p)	Gold	Silver
Combined (served)	-0.34 % (p 0.49)	-0.04 % (p 0.97)	-0.29 % (p 0.46)
Ridge	+0.04 % (p 0.91)	+0.33 % (p 0.76)	-0.27 % (p 0.48)
RandomForest	-0.23 % (p 0.62)	-0.73 % (p 0.57)	-1.36 % (p 0.17)
LightGBM	-0.53 % (p 0.79)	+0.05 % (p 0.88)	-0.01 % (p 0.98)
CatBoost	+0.13 % (p 0.74)	+0.62 % (p 0.71)	+0.10 % (p 0.70)
GRU	+0.44 % (p 0.35)	-0.44 % (p 0.58)	-0.01 % (p 0.99)
LSTM	+0.13 % (p 0.35)	+0.96 % (p 0.28)	+0.27 % (p 0.63)
Stacked (experiment)	-0.72 % (p 0.44)	+2.02 % (p 0.44)	+0.06 % (p 0.91)
Naive-Mean (drift)	-0.05 % (p 0.81)	+0.26 % (p 0.43)	+0.10 % (p 0.63)
ARIMA	+0.56 % (p 0.35)	+0.62 % (p 0.52)	+1.14 % (p 0.50)

Negative = lower error than "tomorrow = today". Every |difference| is at most 2.0 % and **no DM p-value is below 0.05**. The spread between the best and the worst model is 1.3 / 2.8 / 2.5 percentage points (Bitcoin / Gold / Silver), which is noise for 143 to 207 days.

> **Why did we choose these particular models?**
> To cover **families**, not to find a winner: linear (Ridge), bagged trees (RF), two boosted-tree libraries (LightGBM, CatBoost), and two recurrent networks (GRU, LSTM), plus three baselines. If *any* family had skill, one of these would show it. ExtraTrees, HistGradientBoosting and validation-weighted ensembles were tried on validation on 2026-09-21 and landed within ±0.2 % of the random walk, so they were not added; XGBoost was judged redundant with LightGBM and CatBoost.

Chapter 9: Training, Validation and Testing

9.1 Dataset splitting (actual dates and record counts)

The split is **frozen in** `config.py`: `TRAIN_END = "2025-09-10"` and `VAL_END = "2026-02-17"`, identical for all assets. A sample is one forecast: inputs up to day t, target = the return from t to t+1. Each sample is assigned to a split by the **date its target covers**, so no training target reaches past 2025-09-10 and no validation target past 2026-02-17.

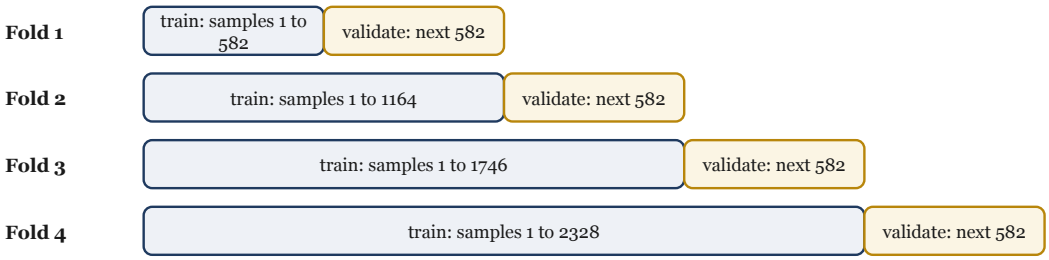
Split	Bitcoin samples	Gold = Silver samples	Bitcoin: day t range	Bitcoin: target dates	Metals: day t range
Train	2,750 (88.2 %)	1,874 (88.1 %)	2018-03-01 to 2025-09-09	up to 2025-09-10	2018-03-29 to 2025-09-09
Validation	160 (5.1 %)	109 (5.1 %)	2025-09-10 to 2026-02-16	2025-09-11 to 2026-02-17	2025-09-10 to 2026-02-13
Test	207 (6.6 %)	143 (6.7 %)	2026-02-17 to 2026-09-11	2026-02-18 to 2026-09-12	2026-02-17 to 2026-09-10
Total	3,117	2,126			

Gold and Silver test targets end on 2026-09-11, Bitcoin's on 2026-09-12. The first sample starts on 2018-03-01 (Bitcoin) or 2018-03-29 (metals) because the first 30 feature rows are needed to fill one 30-day window. Test sizes: 207, 143 and 143 samples; for Bitcoin that is about 7 months, for the metals about 7 months of exchange days.

A. Frozen chronological split (Bitcoin sample counts; the dates are identical for all assets)



B. Walk-forward validation inside train + validation: 4 expanding folds (Bitcoin: 582 samples per block)



Metals (Gold, Silver): 396 samples per block, training windows of 399 / 795 / 1,191 / 1,587 samples. The test block is never inside any fold.

9.2 Why time-series validation differs from ordinary random splitting

	Random split (wrong for time series)	Our time-ordered split
Which days are in the test set	Random days from anywhere in 2018 to 2026	Only the last 143 to 207 days
Training sees	Days both **before and after** each test day	Only the **past**
Neighbouring days	Yesterday and tomorrow of a test day are in training: near-copies leak the answer	Test days come after every training target
What the score means	Interpolation: filling in gaps in the past	Forecasting: what would have happened if we had used the model live
Typical outcome	Impressive but fake accuracy	An honest (here: null) result

No `train_test_split`, `KFold`, or `shuffle=True` exists anywhere in the code; Keras training uses `shuffle=False`; the folds come from `TimeSeriesSplit`.

9.3 Walk-forward (expanding window) validation

To compare models and tune hyper-parameters we need *several* out-of-sample scores, not just one. `walk_forward_splits` cuts train+validation (2,910 samples for Bitcoin, 1,983 for the metals) into **4 consecutive validation blocks**; fold k trains on everything before block k.

Asset	Fold	Train samples	Training window (day t)	Validation samples	Validation window (day t)
Bitcoin	1	582	2018-03-01 to 2019-10-03	582	2019-10-04 to 2021-05-07
	2	1,164	2018-03-01 to 2021-05-07	582	2021-05-08 to 2022-12-10
	3	1,746	2018-03-01 to 2022-12-10	582	2022-12-11 to 2024-07-14
	4	2,328	2018-03-01 to 2024-07-14	582	2024-07-15 to 2026-02-16
Gold	1	399	2018-03-29 to 2019-10-28	396	2019-10-29 to 2021-05-25
	2	795	2018-03-29 to 2021-05-25	396	2021-05-26 to 2022-12-19
	3	1,191	2018-03-29 to 2022-12-19	396	2022-12-20 to 2024-07-19
	4	1,587	2018-03-29 to 2024-07-19	396	2024-07-22 to 2026-02-13

- Silver's folds are the same as Gold's (same calendar). The test block is **never** inside a fold; the four validation blocks together cover **2,328 (Bitcoin) and 1,584 (metals) out-of-fold days**, 2019-10 to 2026-02, which the project pools into one long record (Chapter 10).
- **Inside each fold** the last 15 % of that fold's training window serves as the early-stopping monitor for models that need one (`inner_val_frac = 0.15`); the fold's validation block is only ever predicted, never fitted on.
- For multi-day targets (experiments) the last h−1 training samples are purged so no training target overlaps a validation target.

src/evaluation/cross_validation.py (the fold generator)

```
25 def walk_forward_splits(n_samples, n_splits=CV_FOLDS, min_train=None):
26     """Yield (train_idx, val_idx) with growing training windows and equal validation blocks."""
27     tscv = TimeSeriesSplit(n_splits=n_splits)
28     for tr, va in tscv.split(np.arange(n_samples)):
29         if min_train and len(tr) < min_train:
30             continue
31         yield tr, va
```

9.4 Hyper-parameter selection

Model	Searched values (grid)	Configs
Ridge	alpha: [0.1, 1.0, 10.0, 100.0]	4
RandomForest	max_depth: [3, 6], min_samples_leaf: [10, 30]	4
LightGBM	learning_rate: [0.01, 0.03], num_leaves: [4, 8, 15, 31], min_child_samples: [20, 50, 100], reg_lambda: [0.0, 5.0]	48
CatBoost	learning_rate: [0.01, 0.03], depth: [3, 5, 7], l2_leaf_reg: [3.0, 10.0]	12
GRU	dropout: [0.2, 0.4]	2
LSTM	dropout: [0.2, 0.4]	2

For **each asset** every configuration is scored by the 4 walk-forward folds (`cv_evaluate`), and the one with the **lowest mean RMSE in return space** is written to `results/tuning/best_params.json`. That is 72 configurations x 4 folds x 3 assets. The grids are deliberately small and strongly regularised: with about 1,900 to 2,750 noisy samples the search is over *how much to hold the model back*, not over capacity. Notice that several winners sit on the **edge** of the grid (Ridge `alpha` = 100, forest `max_depth` = 3, `min_samples_leaf` = 30): the search keeps asking for more regularisation, which again says there is little signal to learn. The test set is never read by the tuner (`tune_models.py` only touches train+validation arrays).

9.5 Model fitting: two phases for every model with a stopping rule

Phase	Data used	Purpose	Output
A (validation fit)	Fit on **train**, monitor **validation** (trees: early-stopping rounds; Keras: `val_loss` with patience)	Find the stopping point (number of trees / iterations / epochs) and produce honest train and validation metrics	`results/train_val_metrics.csv`, `results/histories/*.json` (loss curves)
B (deployed fit)	Refit on **train + validation** with the stopping point **fixed**	The deployed model uses all pre-test data without needing a monitor	`data/models/<prefix>_<model>.*`

The phase split prevents the old CatBoost bug (early stopping on its own training data) and keeps validation out of its own monitor. Models without a stopping rule (Ridge, Random Forest) simply fit once per phase. Reproducibility: `set_all_seeds(42)` before each Keras fit, `random_state=42` for Random Forest and LightGBM, `random_seed=42` for CatBoost.

9.6 Saved model artefacts

File pattern (in `data/models/`)	Content	Written by	Read by
`<prefix>_scaler.pkl` (about 2.4 KB)	Fitted `MinMaxScaler` incl. `feature_names_in_`	`DataCleaner.save_data`	`build_dataset`, `_window`, `recent_track_record`
`<prefix>_ridge.pkl`, `_randomforest.pkl`, `_lightgbm.pkl`	joblib dict {model, params, fit_info}	`BaseModel.save`	`BaseModel.load` via `load_trained`
`<prefix>_catboost.cbm` + `.cbm.json`	Native CatBoost model + params and iteration count	`CatBoostModel.save`	`CatBoostModel.load`
`<prefix>_gru.keras`, `_lstm.keras` + `.keras.json`	Keras model + params, epochs used, loss history	`_KerasSequenceModel.save`	`_KerasSequenceModel.load`
`<prefix>_stack.pkl`	Meta-model, base list, weights (experiment)	`fit_stack`	`StackedModel`
`model_status.json` (about 16 KB; **the only tracked file** in `data/models/`)	Per asset: best single model, test metrics, Combined metrics and members, target mean/std, features list, band `k` and coverage	`backtesting.main`	`config.load_model_status`, `prediction.py`, dashboard, API

Together: 18 member models (6 per asset), 3 stacks, 3 scalers and `model_status.json`; sizes range from 0.8 KB (Ridge) to 302 KB (Bitcoin Random Forest).

9.7 How data leakage can occur, and what the code does about it

Leakage risk	What our code does	Where / how verified
Target inside the inputs	Window = rows t−29..t; target = return t→t+1 taken from a shifted copy and used only as the label	`build_dataset`, `_task_target`; tests `test_create_sequences_target_is_next_row_and_never_in_window`, `test_true_target_reconstructs_actual_close_exactly`
Future data in a feature	Only `rolling`, `ewm`, `shift(+k)` and backward `ta` indicators	Test `test_no_lookahead_in_any_feature` (changes the last bar, no earlier feature may change) for crypto and futures paths; truncation test on the real data (2026-09-21)
Vendor close-time leak (macro and High/Low)	Metals: macro value from a date strictly before t; High/Low features lagged one session	`_align_external`, `lag_post_settlement_features`; tests `test_commodity_external_features_are_previous_day_values`, `test_commodity_high_low_features_are_lagged_one_session`
Scaler sees test data	`MinMaxScaler().fit(train_df)` only	Test `test_scaler_fitted_on_train_only`
Target standardisation sees test data	Mean and std from `y_real_train` only	`build_dataset`; test `test_target_standardisation_round_trip`
Random shuffling	None; `TimeSeriesSplit`; `shuffle=False`	Code search, no `train_test_split`
Split by feature date instead of target date	Masks use the date the target covers	Test `test_split_is_chronological_and_disjoint`
Hyper-parameters tuned on test	Tuner and trainer never read `X_test`; only `backtesting.py` does	`tune_models.py`, `train_models.py` source
Early-stopping monitor = final test	Phase A uses validation; Phase B has no monitor; walk-forward uses an inner 15 % tail	`registry.py`, `cross_validation.py`
Ensemble weights chosen on test	Combined has no weights; the stack is fitted on out-of-fold predictions of train+val	`backtesting.test_predictions`, `ensemble_model.fit_stack`
Running the model on a partial bar	Complete-bar rule drops today's running bar at download	`market_calendar.py`; test in `test_market_calendar.py`
Inference sees the future	`predict_for_date` cuts the frame at the chosen day; ARIMA appends truth only after forecasting	Test `test_predict_for_date_uses_only_past_data_and_reveals_actual` (perturbs every future row, prediction unchanged)

9.8 Weaknesses, and how the methodology was corrected over time

Date	Problem found	Correction
2026-09-12	`reconstruct_price` used the previous day's **open** instead of the close (every USD metric wrong); hyper-parameters chosen on the test set; synthetic weekend rows for metals (30 % of rows); price-level features; CatBoost early stopping on its own training data	Explicit previous close; walk-forward tuning on train+val only; exchange calendar for metals; stationary features only; two-phase training
2026-09-13	**Close-time leak** for Gold/Silver: same-day macro data and session High/Low came from after the 13:30 ET settlement; Silver showed a fake 62.2 % direction accuracy	Close-time rule (`EXTERNAL_SAME_DAY`, `POST_SETTLEMENT_FEATURES`), full re-run; the apparent skill vanished
2026-09-14	Bitcoin's last frozen row was an intraday snapshot	Complete-bar rule; frozen file replaced
2026-09-21	"68 %" band was a constant that covered 87 % of validation days for Bitcoin and 44 to 53 % for the metals	Conditional band with EWMA volatility
2026-10-04	Band still over-covered (73 to 75 %); inference relied on column order; one 143-to-207-day test window has little statistical power	Calibrated multiplier k; column-order guard; pooled walk-forward significance tests; E5 candidates tested and rejected

Remaining weaknesses (say them before the examiner does)

The test window is not a pristine one-shot holdout. It was looked at in several development rounds (leak fix, Combined forecast, band). No parameter was fitted on it, but design decisions were made after seeing it. That is why the 6-year pooled walk-forward record exists; it agrees with the test result.

Hyper-parameters were tuned on the same walk-forward folds that are pooled as evidence, so that record is, if anything, biased in favour of the models. It still shows no skill.

LightGBM's `subsample` = 0.8 is inert (`subsample_freq` is 0). The reported models were trained that way; the code comment says so. It would change served models without evidence of benefit, so it was documented, not changed.

GRU and LSTM stop after 1 to 9 epochs, i.e. they are nearly constant predictors; they are retained because the question was whether *any* family finds signal.

Validation metrics are optimistic because stopping points were chosen on them (phase A).

Chapter 10: Model Evaluation and Actual Results

This is the most important chapter. Every number comes from `results/final_test_results.csv`, `cv_results.csv`, `walkforward_pooled.csv`, `band_calibration.csv`, `regime_analysis.csv`, `train_val_metrics.csv` or `data/models/model_status.json`, all written by the project's own evaluation scripts. Where a number was **calculated during the preparation of this guide** (not by the project), it is labelled so.

10.1 The metrics we use

Metric	What it measures	Better	Formula (n days)	Limitation
RMSE (return)	Typical size of the error of the predicted return, large misses weighted more	Lower	$\sqrt{(1/n)\,\Sigma\,(y-\hat{y})^2}$	Scale depends on the asset; compare to the baseline's RMSE, never alone
MAE (return, USD)	Average absolute error	Lower	$(1/n)\,\Sigma\,\lvert y-\hat{y}\rvert$	In USD it mixes assets with very different prices
MAPE (USD)	Average error as a % of the true price	Lower	$(100/n)\,\Sigma\,\lvert P-\hat{P}\rvert / P$	**Not accuracy**: "tomorrow = today" scores the same (1.52 / 1.29 / 2.49 %)
R^2 (return)	Share of the variance of the return that the model explains	Higher (0 = predicting the mean)	$1-\Sigma(y-\hat{y})^2 / \Sigma(y-\bar{y})^2$	Negative means worse than the mean. R^2 on **price levels** is deliberately not reported (a random walk scores about 0.95)
Directional accuracy (DA)	Share of non-flat days on which sign(ŷ) = sign(y)	Higher (50 % = coin flip)	hits / $n_{\text{non-flat}}$; one-sided binomial p-value vs 50 %	A forecast that says UP on almost every day scores the **share of up days**, which is drift, not timing
UP calls %	Share of days the model predicted a rise	Context	mean($\hat{y} > 0$)	Reported next to DA and next to the share of actual up days
Diebold-Mariano (DM) p	Whether two forecasts' squared errors differ more than luck allows (here: vs the random walk)	$p < 0.05$ = real difference	$DM = \bar{d} / \sqrt{(\mathrm{Var}(d)/n)}$, $d_t = e_{\text{model},t}^2 - e_{\text{naive},t}^2$; HLN small-sample correction; Student-t	Low power with 143 to 207 days; hence the pooled test
Strategy return	Long when $\hat{y} > 0$, flat otherwise, 10 bps cost per position change, versus buy-and-hold	Context only	Σ pos·y – costs, exponentiated	Not a trading claim; noise at this sample size
Band coverage	Share of days the actual close fell inside the 68 % band	Close to 68.3 %	mean($\lvert y-\hat{y}\rvert \le k\cdot\sigma_t$)	Tests honesty of the uncertainty, not accuracy

How to read "vs naive". RMSE of a model divided by RMSE of the zero-return forecast, minus 1. Negative = better than "tomorrow = today". Tolerance: differences below roughly 5 to 10 % cannot be detected with 143 to 207 test days, which is why the DM test and the pooled record are used.

10.2 Which result is which (train, validation, walk-forward, pooled, test)

Label in this guide	What it is	Data it uses	File
Train / Validation (phase A)	Model fitted on train, scored on train (in-sample) and on validation; used for stopping points and the over-fitting gap	train, validation	`train_val_metrics.csv`
Walk-forward validation	Mean ± sd over 4 expanding folds; **used for model selection and tuning**	train + validation only	`cv_results.csv`
Pooled walk-forward	All out-of-fold predictions of the 4 folds in one record (2,328 or 1,584 days) tested at once	train + validation only	`walkforward_pooled.csv`
Test (untouched)	Deployed models predicting 2026-02-18 to 2026-09-11/12, evaluated by `backtesting.py`	test only	`final_test_results.csv`, `predictions/*.csv`

10.3 Training versus validation (over-fitting check)

Asset	Model	Train RMSE (ret)	Val RMSE (ret)	Train R^2	Val R^2	Train DA %	Val DA %
Bitcoin	Ridge	0.03352	0.02605	+0.005	-0.012	52.3	50.6
	RandomForest	0.03299	0.02610	+0.036	-0.016	56.0	51.2
	LightGBM	0.02740	0.02539	+0.335	+0.038	71.5	54.4
	CatBoost	0.03209	0.02604	+0.088	-0.011	58.5	44.4
	GRU	0.03361	0.02597	-0.001	-0.006	50.0	53.8
	LSTM	0.03358	0.02617	+0.001	-0.021	50.4	46.2
	random walk	0.03361	0.02610				
Gold	Ridge	0.00973	0.02025	+0.006	-0.008	54.2	56.9
	RandomForest	0.00953	0.02031	+0.047	-0.014	57.8	54.1
	LightGBM	0.00973	0.02028	+0.005	-0.011	54.3	63.3
	CatBoost	0.00932	0.01995	+0.088	+0.021	57.4	59.6
	GRU	0.00974	0.02022	+0.004	-0.006	54.3	63.3
	LSTM	0.00975	0.02028	+0.000	-0.011	54.3	63.3
	random walk	0.00977	0.02035				
Silver	Ridge	0.01915	0.05543	+0.005	-0.012	52.9	59.6
	RandomForest	0.01867	0.05541	+0.055	-0.011	60.2	59.6
	LightGBM	0.01919	0.05532	+0.001	-0.008	52.4	62.4
	CatBoost	0.01904	0.05528	+0.017	-0.006	53.4	64.2
	GRU	0.01917	0.05528	+0.003	-0.006	51.3	63.3
	LSTM	0.01920	0.05543	+0.001	-0.012	48.9	56.0
	random walk	0.01921	0.05536				

Read the **random-walk row** first: its RMSE is much higher in validation than in training for the metals (Gold 0.0098 → 0.0203, Silver 0.0192 → 0.0554) because the validation months were far more volatile, not because models overfit. Real overfitting appears only for LightGBM on Bitcoin (train R^2 0.335, DA 71.5 % versus validation R^2 0.038): memorisation, controlled by early stopping at 81 trees. GRU and LSTM stop after 1 to 9

epochs with train R^2 near zero: they have nothing to memorise.

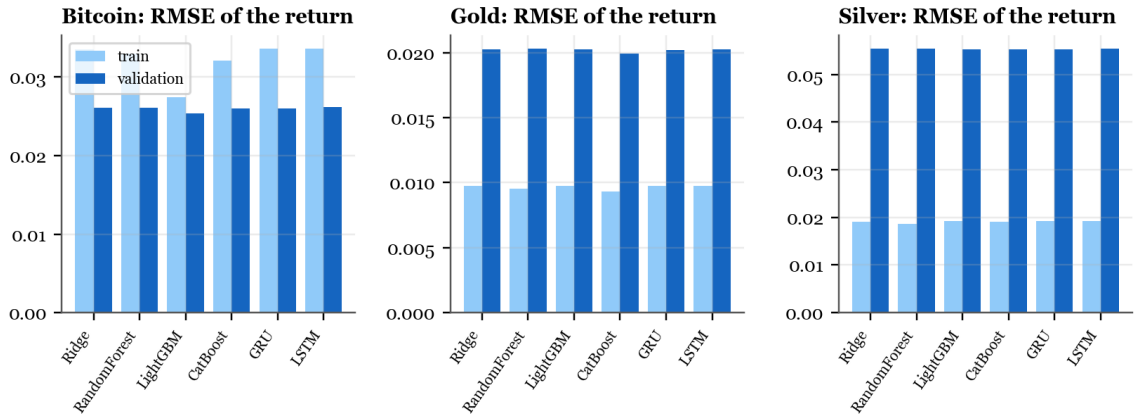


Figure 10.1: Train versus validation RMSE of the return (phase A), from `results/train_val_metrics.csv`.

10.4 Untouched test set, per asset

Bitcoin: 207 test days (2026-02-18 to 2026-09-12), 49 % of them up days

Model	MAE ($)	RMSE ($)	MAPE %	RMSE (return)	R^2 (return)	Dir. acc. % (p)	UP calls %	DM p
Combined (served)	1,063.94	1,447.37	1.52	0.02072	+0.006	46.9 (0.83)	37	0.49
Ridge	1,071.97	1,452.59	1.53	0.02080	-0.002	47.8 (0.76)	39	0.91
RandomForest	1,064.48	1,448.81	1.52	0.02075	+0.004	48.3 (0.71)	57	0.62
LightGBM	1,076.49	1,446.18	1.54	0.02068	+0.010	51.2 (0.39)	45	0.79
CatBoost	1,071.07	1,453.76	1.53	0.02082	-0.004	46.9 (0.83)	60	0.74
GRU	1,075.86	1,458.31	1.54	0.02089	-0.010	46.9 (0.83)	16	0.35
LSTM	1,068.06	1,453.44	1.53	0.02082	-0.004	46.4 (0.87)	26	0.35
Stacked (experiment)	1,061.35	1,442.94	1.52	0.02065	+0.013	50.2 (0.50)	64	0.44
Naive-Mean (drift)	1,067.32	1,451.38	1.53	0.02078	-0.000	49.3 (0.61)	100	0.81
ARIMA	1,067.96	1,459.44	1.53	0.02091	-0.012	46.9 (0.83)	72	0.35
Naive (random walk)	1,066.42	1,451.73	1.52	0.02079	-0.001	—	—	—

Gold: 143 test days (2026-02-18 to 2026-09-11), 50 % of them up days

Model	MAE ($)	RMSE ($)	MAPE %	RMSE (return)	R² (return)	Dir. acc. % (p)	UP calls %	DM p
Combined (served)	58.20	75.73	1.29	0.01671	-0.001	50.3 (0.50)	97	0.97
Ridge	58.51	76.05	1.30	0.01677	-0.009	50.3 (0.50)	94	0.76
RandomForest	58.08	75.10	1.29	0.01659	+0.012	49.0 (0.63)	87	0.57
LightGBM	58.25	75.81	1.29	0.01672	-0.003	49.7 (0.57)	100	0.88
CatBoost	58.43	76.22	1.30	0.01681	-0.015	51.7 (0.37)	92	0.71
GRU	58.00	75.42	1.29	0.01664	+0.007	51.7 (0.37)	97	0.58
LSTM	58.82	76.58	1.31	0.01687	-0.022	49.7 (0.57)	100	0.28
Stacked (experiment)	58.95	77.28	1.31	0.01705	-0.043	51.0 (0.43)	97	0.44
Naive-Mean (drift)	58.39	75.99	1.30	0.01675	-0.007	49.7 (0.57)	100	0.43
ARIMA	58.57	76.27	1.30	0.01681	-0.015	50.3 (0.50)	80	0.52
Naive (random walk)	58.26	75.76	1.29	0.01671	-0.002	—	—	—

Silver: 143 test days (2026-02-18 to 2026-09-11), 52 % of them up days

Model	MAE ($)	RMSE ($)	MAPE %	RMSE (return)	R² (return)	Dir. acc. % (p)	UP calls %	DM p
Combined (served)	1.78	2.37	2.47	0.03180	+0.005	53.8 (0.20)	91	0.46
Ridge	1.79	2.37	2.48	0.03181	+0.005	54.5 (0.16)	94	0.48
RandomForest	1.76	2.33	2.44	0.03146	+0.026	56.6 (0.07)	66	0.17
LightGBM	1.79	2.37	2.48	0.03189	-0.001	58.7 (0.02)	86	0.98
CatBoost	1.79	2.38	2.49	0.03193	-0.003	51.7 (0.37)	100	0.70
GRU	1.79	2.37	2.48	0.03189	-0.001	55.2 (0.12)	88	0.99
LSTM	1.80	2.38	2.49	0.03198	-0.006	51.7 (0.37)	100	0.63
Stacked (experiment)	1.79	2.38	2.48	0.03191	-0.002	52.4 (0.31)	94	0.91
Naive-Mean (drift)	1.79	2.38	2.49	0.03192	-0.003	51.7 (0.37)	100	0.63
ARIMA	1.81	2.37	2.52	0.03226	-0.024	52.4 (0.31)	55	0.50
Naive (random walk)	1.79	2.37	2.49	0.03189	-0.001	—	—	—

Rows are sorted as in the dashboard's legend (served Combined first, baselines last). MAE / RMSE in USD; return metrics in log-return units; DA = direction accuracy on non-flat days with the one-sided binomial p in brackets; DM p = Diebold-Mariano test against the random walk. The Stacked row is an experiment; **Stacked is the single best model for Bitcoin in RMSE terms (−0.7 %) and the single worst for Gold (+2.0 %)**: that sign flip is what noise looks like.

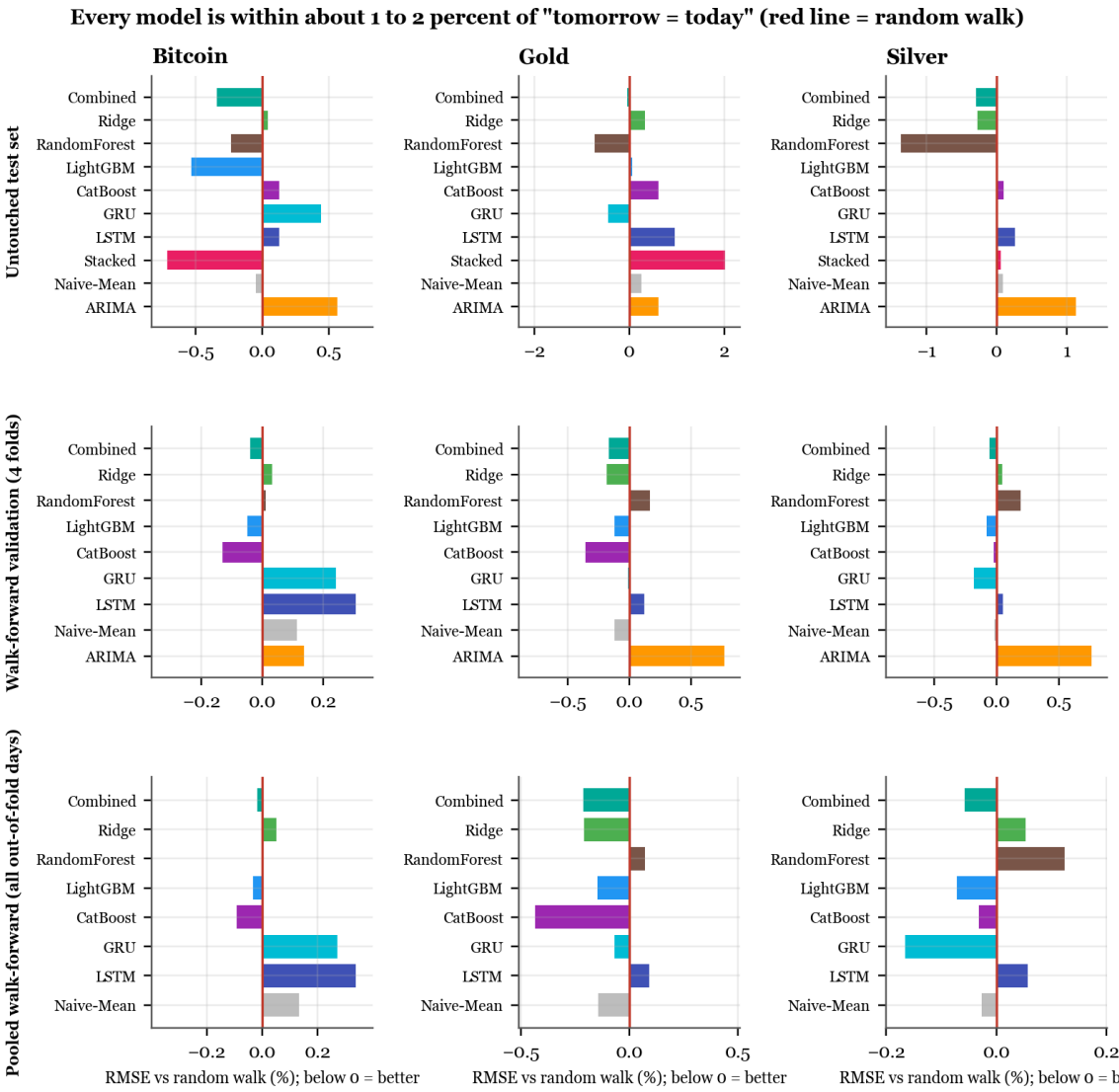


Figure 10.2: RMSE of every model relative to the random walk (red line) on the test set, in walk-forward validation, and on the pooled record. Note that the horizontal scale differs per panel and is zoomed to a few percent.

10.5 Walk-forward validation (model-selection evidence)

Bitcoin

Model	RMSE (return), mean ± sd over 4 folds	MAE (return)	R^2 (return)	Dir. acc. %	RMSE ($)	vs naive
CatBoost	0.03156 ± 0.00695	0.02136	-0.001	50.2	1,427.07	-0.13 %
LightGBM	0.03158 ± 0.00692	0.02140	-0.003	51.2	1,430.52	-0.05 %
Combined (served)	0.03159 ± 0.00693	0.02133	-0.004	50.8	1,429.51	-0.04 %
Naive (random walk)	0.03160 ± 0.00690	0.02130	-0.005	—	1,429.08	+0.00 %
RandomForest	0.03160 ± 0.00688	0.02140	-0.005	50.0	1,432.90	+0.01 %
Ridge	0.03161 ± 0.00693	0.02135	-0.005	49.4	1,430.34	+0.03 %
Naive-Mean (drift)	0.03163 ± 0.00694	0.02133	-0.007	48.8	1,430.95	+0.11 %
ARIMA	0.03164 ± 0.00695	0.02135	-0.007	48.1	1,431.80	+0.14 %
GRU	0.03168 ± 0.00697	0.02136	-0.009	49.4	1,432.61	+0.24 %
LSTM	0.03170 ± 0.00697	0.02142	-0.010	48.8	1,433.09	+0.31 %

Gold

Model	RMSE (return), mean ± sd over 4 folds	MAE (return)	R^2 (return)	Dir. acc. %	RMSE ($)	vs naive
CatBoost	0.01110 ± 0.00219	0.00786	-0.001	54.8	28.29	-0.35 %
Ridge	0.01112 ± 0.00223	0.00787	-0.003	54.8	28.40	-0.18 %
Combined (served)	0.01113 ± 0.00222	0.00788	-0.004	52.5	28.39	-0.16 %
Naive-Mean (drift)	0.01113 ± 0.00223	0.00787	-0.004	55.8	28.43	-0.12 %
LightGBM	0.01113 ± 0.00223	0.00787	-0.004	55.8	28.42	-0.12 %
GRU	0.01114 ± 0.00222	0.00791	-0.007	51.1	28.45	-0.01 %
Naive (random walk)	0.01114 ± 0.00225	0.00790	-0.006	—	28.46	+0.00 %
LSTM	0.01116 ± 0.00223	0.00793	-0.010	47.0	28.49	+0.12 %
RandomForest	0.01116 ± 0.00220	0.00791	-0.012	51.9	28.45	+0.17 %
ARIMA	0.01123 ± 0.00219	0.00796	-0.025	50.5	28.64	+0.77 %

Silver

Model	RMSE (return), mean ± sd over 4 folds	MAE (return)	R^2 (return)	Dir. acc. %	RMSE ($)	vs naive
GRU	0.02387 ± 0.00625	0.01592	+0.002	52.3	1.00	-0.18 %
LightGBM	0.02389 ± 0.00625	0.01592	-0.000	53.3	1.00	-0.08 %
Combined (served)	0.02390 ± 0.00624	0.01592	-0.001	52.7	1.00	-0.06 %
CatBoost	0.02391 ± 0.00623	0.01594	-0.002	53.5	1.00	-0.02 %
Naive-Mean (drift)	0.02391 ± 0.00623	0.01594	-0.002	53.5	1.00	-0.01 %
Naive (random walk)	0.02391 ± 0.00624	0.01596	-0.002	—	1.00	+0.00 %
Ridge	0.02393 ± 0.00625	0.01593	-0.003	53.3	1.00	+0.05 %
LSTM	0.02393 ± 0.00625	0.01595	-0.003	51.2	1.00	+0.05 %
RandomForest	0.02396 ± 0.00619	0.01596	-0.007	51.1	1.01	+0.19 %
ARIMA	0.02410 ± 0.00613	0.01619	-0.021	50.2	1.00	+0.76 %

Every model lies within about ±0.8 % of the random walk. **Selection rule** (`backtesting.py`): the single ML model with the lowest mean walk-forward RMSE is called `primary_model` (CatBoost for Bitcoin and Gold, GRU for Silver) and is shown for comparison; the served forecast is the equal-weight Combined, because the ranking is not statistically decisive.

10.6 Pooled walk-forward evidence (about six years, train+validation only)

Bitcoin: 2,328 out-of-fold days, 2019-10-04 to 2026-02-16, 50.6 % up days

Model	RMSE vs naive	DM p vs naive	DM p vs drift	Dir. acc. % (p)	UP calls %	corr(ŷ, y)
CatBoost	-0.091 %	0.448	0.039	50.2 (0.426)	68	+0.038
LightGBM	-0.034 %	0.862	0.363	51.2 (0.136)	58	+0.039
Combined (served)	-0.018 %	0.866	0.090	50.8 (0.234)	67	+0.018
RandomForest	-0.001 %	0.996	0.523	50.0 (0.525)	53	+0.040
Ridge	+0.050 %	0.682	0.490	49.4 (0.740)	60	+0.008
Naive-Mean (drift)	+0.134 %	0.025	—	48.8 (0.881)	75	-0.064
GRU	+0.271 %	0.037	0.136	49.4 (0.726)	70	-0.038
LSTM	+0.337 %	0.004	0.013	48.8 (0.881)	66	-0.047

Gold: 1,584 out-of-fold days, 2019-10-29 to 2026-02-13, 55.6 % up days

Model	RMSE vs naive	DM p vs naive	DM p vs drift	Dir. acc. % (p)	UP calls %	corr(ŷ, y)
CatBoost	-0.436 %	0.283	0.488	54.8 (0.000)	87	+0.079
Combined (served)	-0.213 %	0.085	0.570	52.5 (0.023)	74	+0.048
Ridge	-0.208 %	0.050	0.410	54.8 (0.000)	90	+0.033
LightGBM	-0.147 %	0.061	0.900	55.8 (0.000)	100	-0.002
Naive-Mean (drift)	-0.142 %	0.070	—	55.8 (0.000)	100	-0.017
GRU	-0.068 %	0.622	0.545	51.1 (0.203)	51	+0.035
RandomForest	+0.072 %	0.757	0.344	51.9 (0.069)	70	+0.013
LSTM	+0.090 %	0.541	0.153	47.0 (0.993)	28	+0.018

Silver: 1,584 out-of-fold days, 2019-10-29 to 2026-02-13, 53.4 % up days

Model	RMSE vs naive	DM p vs naive	DM p vs drift	Dir. acc. % (p)	UP calls %	corr(ŷ, y)
GRU	-0.166 %	0.423	0.459	52.3 (0.033)	75	+0.047
LightGBM	-0.072 %	0.416	0.547	53.3 (0.005)	83	+0.025
Combined (served)	-0.058 %	0.691	0.805	52.7 (0.016)	78	+0.018
CatBoost	-0.031 %	0.612	0.901	53.5 (0.003)	94	-0.006
Naive-Mean (drift)	-0.027 %	0.507	—	53.5 (0.003)	100	-0.025
Ridge	+0.053 %	0.737	0.564	53.3 (0.005)	78	-0.027
LSTM	+0.057 %	0.798	0.680	51.2 (0.176)	60	-0.008
RandomForest	+0.124 %	0.649	0.564	51.1 (0.203)	57	+0.013

DM p vs drift compares a model with the "always predict the training mean" forecast. Gold's and Silver's 52 to 55 % direction hit-rates are *statistically significant against 50 %* but **below what the drift forecast ("always UP") scores** (Gold 55.8 %, Silver 53.5 %), so they reflect the rising average price, not timing skill. The pooled record has about ten times more days than the test window, so small real effects would show up; none does.

10.7 Do the models beat the baseline? (the central question)

Asset	Test: Combined RMSE vs naive	Test DM p	Test DA % (p)	UP calls / up days %	Pooled: RMSE vs naive	Pooled DM p	Share of test days where Combined was closer than naive*
Bitcoin	-0.34 %	0.49	46.9 (p 0.83)	37 / 49	-0.018 %	0.87	46 %
Gold	-0.04 %	0.97	50.3 (p 0.50)	97 / 50	-0.213 %	0.08	47 %
Silver	-0.29 %	0.46	53.8 (p 0.20)	91 / 52	-0.058 %	0.69	52 %

*Calculated while writing this guide from `results/predictions/*_test_predictions.csv` (not a project metric): Combined's absolute return error was smaller than "predict zero" on 45.9 %, 46.9 % and 52.4 % of the days (Bitcoin, Gold, Silver). A real edge would show clearly above 50 %.

> **Verdict**
> **The models do not outperform the naive baseline.** On the test set the Combined forecast is 0.04 to 0.34 % below the random walk's RMSE (nothing; p = 0.49, 0.97, 0.46). Over six pooled years it is −0.02, −0.21 and −0.06 % (p = 0.87, 0.09, 0.69). GRU and LSTM are significantly **worse** for Bitcoin (pooled p = 0.037 and 0.004). Do not describe any model as "better": the differences are smaller than the noise.

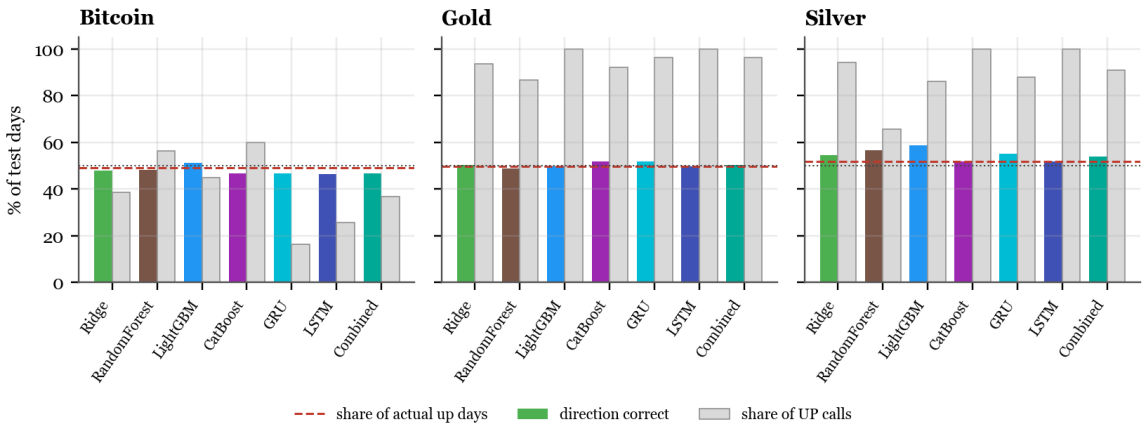


Figure 10.3: Direction accuracy (coloured) versus the share of UP calls (grey) and the share of actual up days (red dashed) on the test set. For the metals the models say UP almost every day, so their hit-rate equals the share of up days.

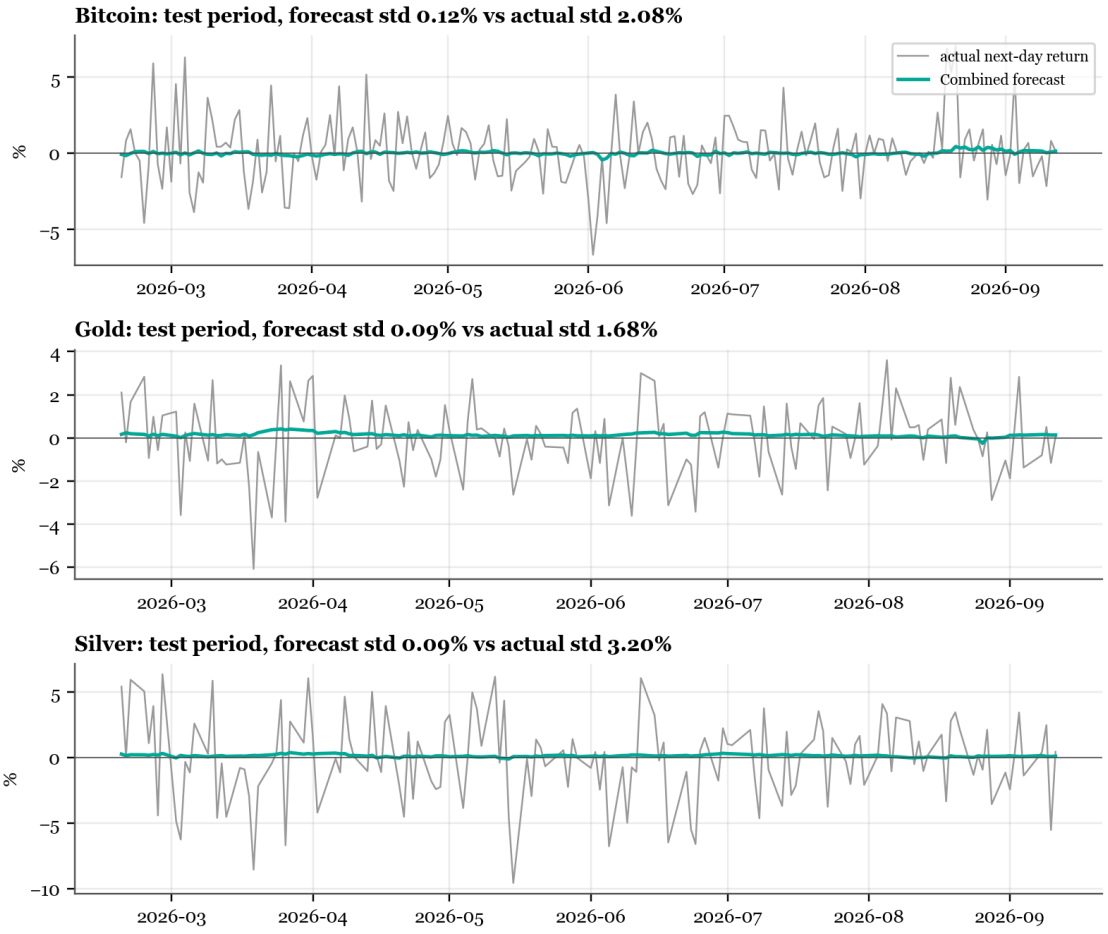


Figure 10.4: Actual next-day returns (grey) versus the Combined forecast (teal) on the test period: the forecast barely moves (standard deviation 0.09 to 0.12 % against 1.7 to 3.2 % for reality).

10.8 The uncertainty band: the one useful output

Asset	k (served)	Validation coverage % (served)	Test coverage % (served)	Test: plain ±1σ_t %	Test: fixed width %	Mean half-width %
Bitcoin	0.825	60.6	**65.7**	73.9	79.2	1.82
Gold	0.909	66.1	**69.2**	75.5	42.0	1.68
Silver	0.870	54.1	**69.2**	75.5	44.8	3.47

The served 68 % band is $P_t \cdot \exp(\hat{y} \pm k \cdot \sigma_t)$, σ_t = EWMA volatility at day t and k the 68.27 % quantile of |error| / σ_t over the walk-forward errors of the **training** split only (k < 1 because returns are fat-tailed). On the unseen test days it contained the actual close on 65.7 / 69.2 / 69.2 % of days (nominal 68.3 %); the uncalibrated ±1σ band covered 73.9 / 75.5 / 75.5 % and a fixed-width band 79.2 / 42.0 / 44.8 % (Bitcoin / Gold / Silver). On the shorter validation window the calibrated band covered 60.6 / 66.1 / 54.1 %: coverage is approximately, not exactly, honest.

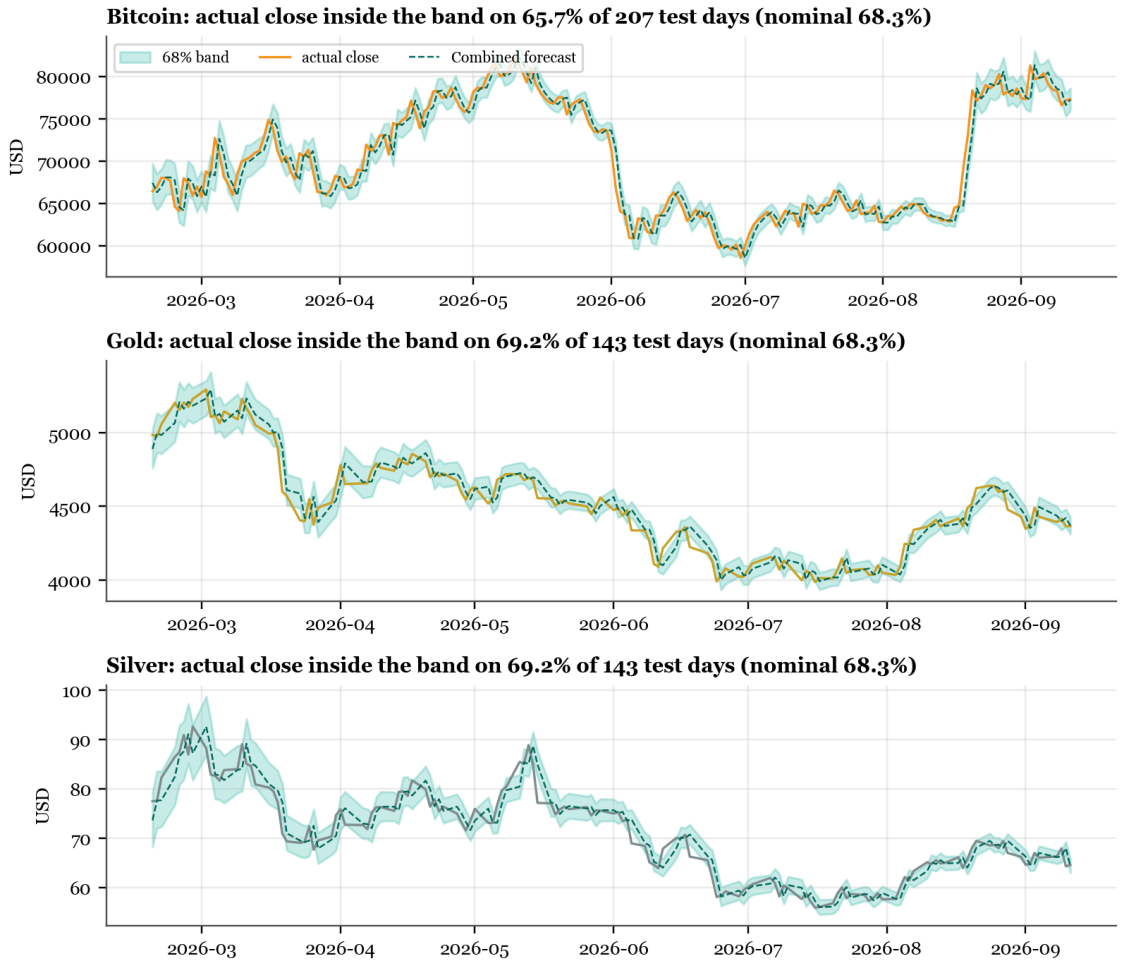


Figure 10.5: Actual close, Combined forecast and the served 68 % band on the test period (from `results/predictions/*.csv`*).*

10.9 Performance by market regime (test period)

Asset	Regime (defined on day t)	Days	MAE % (Combined)	MAE % (random walk)	Direction correct %
Bitcoin	high volatility	69	1.65	1.66	50.7
Bitcoin	low volatility	69	1.30	1.29	47.8
Bitcoin	medium volatility	69	1.62	1.62	42.0
Bitcoin	down-trend (20d)	96	1.62	1.62	47.9
Bitcoin	up-trend (20d)	111	1.43	1.44	45.9
Gold	high volatility	48	1.34	1.35	50.0
Gold	low volatility	48	1.27	1.27	50.0
Gold	medium volatility	47	1.26	1.26	51.1
Gold	down-trend (20d)	86	1.35	1.35	52.3
Gold	up-trend (20d)	57	1.21	1.21	47.4
Silver	high volatility	48	2.92	2.93	52.1
Silver	low volatility	48	2.11	2.14	64.6
Silver	medium volatility	47	2.39	2.38	44.7
Silver	down-trend (20d)	81	2.55	2.58	58.0
Silver	up-trend (20d)	62	2.37	2.36	48.4

Regimes are volatility terciles (30-day volatility) and the sign of the 20-day return. In every regime Combined and the random walk have nearly identical errors; the forecast is not better in calm or turbulent markets.

10.10 Strategy backtest (context only)

Asset	Long/flat strategy (Combined)	Buy and hold	Sharpe (strategy)	Sharpe (buy & hold)	Trades	Time in market
Bitcoin	-5.1 %	+14.5 %	-0.36	0.60	57	37 %
Gold	-7.9 %	-10.6 %	-0.56	-0.74	3	97 %
Silver	-6.0 %	-12.1 %	-0.22	-0.45	15	91 %

The simple rule "hold the asset when the forecast is positive" loses to buy-and-hold for Bitcoin (-5.1 % vs +14.5 %). For Gold (-7.9 % vs -10.6 %) and Silver (-6.0 % vs -12.1 %) it is long on 97 % and 91 % of days, so it mostly just follows the asset. Do **not** present this as profitability.

10.11 Design experiments (validation only)

Experiment	Question	Finding
E1 data size	3 years of data versus the full 2018+ history?	Mixed signs, changes of −1.7 % to +0.7 % in validation RMSE; the longer history did not clearly help
E2 feature ablation	Does dropping any feature group hurt?	No group moves walk-forward RMSE by more than 0.4 % (Ridge, LightGBM)
E3 horizon	Is a 5-day return easier than a 1-day return?	No clear gain; Gold's small gains equal the drift forecast's
E4 volatility target	Is the 22-day volatility predictable?	Yes, partly: EWMA beats persistence by 4.4 to 7.7 %, Ridge by 11.5 % for Bitcoin; R^2 of log-volatility is still negative
E5 audit candidates	Combine only tabular models; remove 10 redundant features; subtract the training drift	None met the rule "better for every asset with DM $p < 0.05$ vs the served forecast": rejected. Point forecasts unchanged

10.12 What the results demonstrate (and what they do not)

- **Demonstrated:** a fair, leakage-audited pipeline gives *the same answer* on the untouched test set and on six pooled years: none of nine models beats "tomorrow = today".
- **Demonstrated:** the size of tomorrow's move is partly predictable from recent volatility, and the calibrated band is roughly honest on unseen days.
- **Not demonstrated:** any direction skill (hit-rates are drift or chance), any profitability, any benefit of deep learning.
- **Always say:** MAPE of 1.3 to 2.5 % is **not** "98 % accuracy".

Chapter 11: Ensemble and Final Prediction

11.1 Which models contribute to the final result

The served forecast is called **Combined**. Its members are the **six trained models**: Ridge, Random Forest, LightGBM, CatBoost, GRU and LSTM (`BASE_MODELS = list(TABULAR) + list(RECURRENT)` in `prediction.py`). The members are discovered from the files on disk (`combined_members` checks `artefact_exists`), so an asset with a missing model file would be combined from the remaining models. The baselines (Naive, Naive-Mean, ARIMA) are not members, and the **stacked ensemble is deliberately excluded** because it is itself a blend of some of the same models.

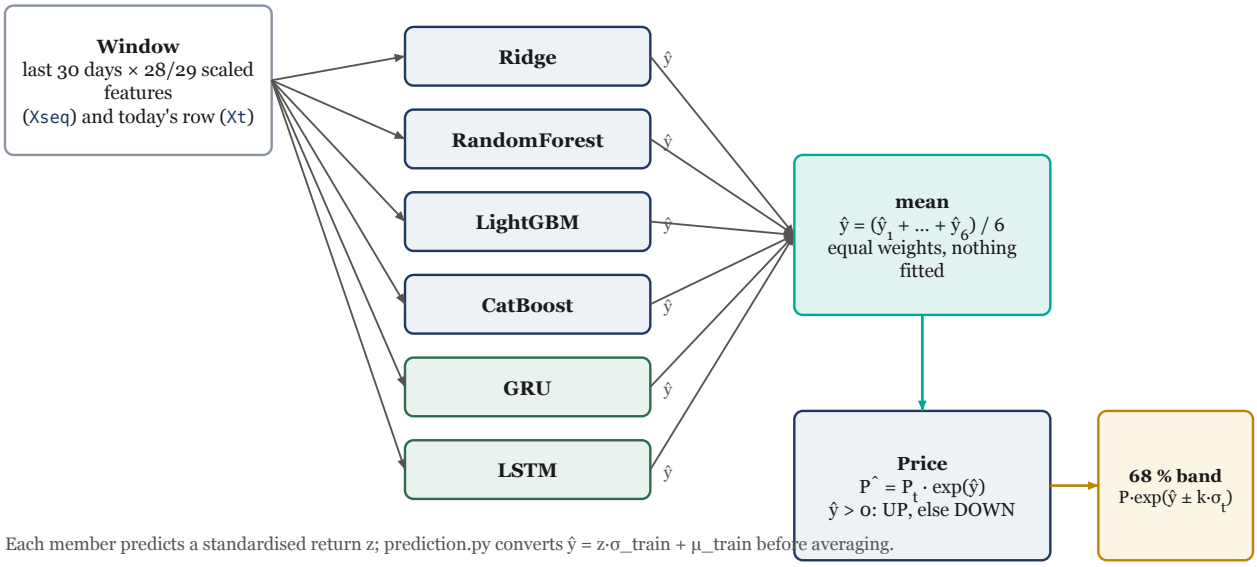


Figure 11.1: One prediction request. All six members receive the same scaled window; their standardised outputs are converted to log returns, averaged, and turned into a price and a band.

11.2 How each prediction is generated and combined (the real code)

src/inference/prediction.py (one member, then the combination)

```
151 def _predict_one(asset, model_name, Xseq, Xt):
152     mu, sd = _target_stats(asset)
153     r = float(np.ravel(_load(asset, model_name).predict(Xseq, Xt))[0] * sd + mu)
154     if not np.isfinite(r):
155         raise ValueError(f"{model_name} returned a non-finite prediction for {asset}")
156     return r
    ...
159 def _predict_from_window(asset, model_name, Xseq, Xt, progress=None):
160     """Real log-return prediction r^ and, for the combined forecast, every member's r^.
161     `progress(model, r_hat, seconds)` is called after each member has run (the dashboard shows it
        live)."""
162     if model_name != COMBINED:
163         return _predict_one(asset, model_name, Xseq, Xt), None
164     members = {}
165     for m in combined_members(asset):
166         t0 = time.perf_counter()
167         members[m] = _predict_one(asset, m, Xseq, Xt)
168         if progress:
169             progress(m, members[m], time.perf_counter() - t0)
170     if not members:
171         raise FileNotFoundError(f"No trained models found for {asset}. Run `make train` first.")
172     return float(np.mean(list(members.values()))), members
```

- `_predict_one` loads the artefact (`_load` → `load_trained`), calls `.predict(Xseq, Xt)`, and converts the standardised output z to a real log return: $\hat{\mathbf{y}} = \mathbf{z} \cdot \boldsymbol{\sigma}_{\mathbf{train}} + \boldsymbol{\mu}_{\mathbf{train}}$ (`_target_stats` reads σ and μ from `model_status.json`). A non-finite output raises `ValueError`.
- `_predict_from_window` runs `_predict_one` for each member (reporting progress to the dashboard) and returns `np.mean` **of the six values** plus the dictionary of individual predictions.
- The same equal-weight mean is used when the project **evaluates** the Combined forecast: `test_predictions` in `backtesting.py` averages the members' stored test predictions, and `combined_cv_row` averages their out-of-fold predictions per fold. A test (`test_combined_test_row_is_the_mean_of_its_members`) checks that the stored `pred_return_Combined` column is exactly that mean.

11.3 Weighting: none

> **Are weights used? How are they determined?**
> **No weights are fitted.** Each member has weight 1/6. This is a deliberate choice: (1) the members' walk-forward errors are statistically tied, so any fitted weights would only chase noise; (2) nothing is selected on the test set; (3) equal-weight averaging is the standard robust way to combine forecasts of similar quality. The one weighted alternative, the stacked ensemble (non-negative Ridge on out-of-fold predictions), was built and evaluated as an experiment; it is the best model for Bitcoin and the worst for Gold on the test set, which illustrates why it was not trusted.

11.4 The mathematical formula

Quantity	Formula	Code
Combined return	$\hat{y}_t = (1/6) \sum_{m \in \{\text{Ridge, RF, LGBM, CatBoost, GRU, LSTM}\}} \hat{y}_t^{(m)}$	`np.mean(list(members.values()))`
Predicted price	$\hat{P}_{t+1} = P_t \cdot \exp(\hat{y}_t)$	`last_close * np.exp(r_hat)`
Direction	UP if $\hat{y}_t > 0$, otherwise DOWN	`"UP" if r_hat > 0 else "DOWN"`
68 % band	$[\, P_t \cdot \exp(\hat{y}_t - k \cdot \sigma_t),\ P_t \cdot \exp(\hat{y}_t + k \cdot \sigma_t) \,]$	`_band`
Reconstruction of a member	$\hat{y}^{(m)} = z^{(m)} \cdot \sigma_{\text{train}} + \mu_{\text{train}}$	`_predict_one`

Worked example, real numbers: Gold, standing on 2026-05-12 (close $4,677.60), frozen dataset

Member	Model output z	Return $\hat{y} = z \cdot \sigma + \mu$ (σ = 0.00976, μ = 0.00054)	In %	Says
Ridge	+0.03107	+0.00084	+0.0844 %	UP
RandomForest	+0.00919	+0.00063	+0.0630 %	UP
LightGBM	+0.00473	+0.00059	+0.0587 %	UP
CatBoost	+0.01058	+0.00064	+0.0644 %	UP
GRU	-0.00978	+0.00045	+0.0445 %	UP
LSTM	+0.09172	+0.00144	+0.1436 %	UP
Mean (Combined)		**+0.00076**	**+0.0764 %**	UP

Predicted close $\hat{P}$ = 4,677.60 · exp(+0.00076) = **$4,681.18**; band $4,613.69 to $4,749.65 (k = 0.909, σ_t = 1.60 %). The actual next close was $4,697.70. All numbers are the output of `predict_for_date("Gold", "2026-05-12")`, re-run on 2026-10-09 against the frozen data.

11.5 Does the ensemble perform better than the individual models?

Asset	Rank on test (RMSE)	Rank in walk-forward	Rank in pooled record	Test RMSE vs naive: Combined / best member	Mean pairwise correlation of the members' test forecasts*
Bitcoin	2 of 7	3 of 7	3 of 7	-0.34 % / -0.53 %	0.41 (0.11 to 0.74)
Gold	3 of 7	3 of 7	2 of 7	-0.04 % / -0.73 %	0.58 (0.27 to 0.81)
Silver	2 of 7	3 of 7	3 of 7	-0.29 % / -1.36 %	0.25 (-0.27 to 0.70)

*Calculated while writing this guide from `results/predictions/*.csv`. Rank 1 = lowest error among the six members and Combined.

- **What the table shows:** Combined ranks **2nd or 3rd of 7** in all nine comparisons (three assets x test, walk-forward, pooled): never the single best, never worse than third. That is the practical benefit of averaging: low regret. The best single model changes with the sample (CatBoost in walk-forward and pooled for Bitcoin and Gold, LightGBM, RandomForest or GRU elsewhere), and the gaps are far inside the noise, so Combined is **not** claimed to be better than any single model.
- **Is Combined better than the baseline?** No (Chapter 10.7).
- The members' forecasts are only moderately correlated (mean pairwise correlation 0.25 to 0.58), so averaging does reduce their idiosyncratic noise, but there is no common signal to keep.

11.6 Why combining can help, and why it can also perform worse

Why an ensemble can help	Why an ensemble can perform worse
Errors that are not perfectly correlated partly cancel, so the variance of the average is lower than the typical member's	If every member is equally bad, the average is equally bad: it cannot create signal that none of them has
No single "lucky" model dominates; protects against picking the validation winner by chance	A member that is systematically worse (GRU / LSTM for Bitcoin, significantly worse than the random walk on the pooled record) drags the average down: tested in E5a (4 tabular models only), which helped Bitcoin by 0.09 % but hurt Silver, so was rejected
Robust: no weights to overfit	Equal weights ignore real quality differences when they exist; learned weights (stack) can capture them, but overfit when they do not (the stack was the worst Gold model on test)
Easy to explain and to audit	Averaging shrinks predictions towards zero: the Combined forecast has the smallest amplitude (0.09 to 0.12 % std)

11.7 The actual end-to-end flow when a user requests a prediction

1. The user presses **Run prediction** in the sidebar (or calls `GET /predict/{asset}` on the API).
2. `predict_next_day(asset, "Combined")` reads `model_status.json` and loads the **live** feature file if present, otherwise the frozen file (`load_features`).
3. `_window` loads the saved scaler, re-orders the feature columns to the scaler's order, scales the history, and keeps the **last 30 rows** (shape 1×30×F) and the last row (1×F).
4. `_predict_from_window` runs each of the six members and averages their returns.
5. The price, UP/DOWN, the 68 % band (`_band`, needs `band.k` from `model_status.json`) and the held-out test metrics (`_context`) are assembled in one dictionary, with the disclaimer.
6. The Streamlit page formats it (final card, tables, charts); the API returns it as JSON through the `PredictionResponse` model.

The dashboard's **Predict a Day** uses `predict_for_date` instead: identical computation, but the frame is cut at the chosen day and the actual close is revealed afterwards. A full code-level trace is in Chapter 22.

Chapter 12: Streamlit Application and User Experience

The dashboard is one file, `app/streamlit_app.py` (761 lines). Streamlit **re-runs the whole script from top to bottom on every interaction**, so the file reads like a long page: sidebar, data loading, overview, four tabs, footer. Every feature below was checked in the running app on 2026-10-09 (Gold with `?asset=Gold&run=1`, then each tab), and the Bitcoin page was opened.

12.1 Entry point, layout and navigation

- **Start:** `streamlit run app/streamlit_app.py` (Makefile: `make serve`; the preview server in `.claude/launch.json` uses port 8501). The file puts the project root on `sys.path` and imports `config`, `src.inference.prediction` and `src.data.sync_live_data`.
- **Page setup:** `st.set_page_config(layout="wide", initial_sidebar_state="expanded")`; a dark theme with custom CSS classes (`.overview`, `.final`, `.live`).
- **Navigation:** a left sidebar (controls) and four tabs: **Forecast**, **Predict a Day (unseen test)**, **Model Performance**, **Methodology**. A button press re-runs the script, so `st.session_state["open_tab"]` (set by each button's `on_click`) keeps the user on the tab they were using.
- **Deep links:** `?asset=Gold` selects the asset; `?run=1` presses Run prediction automatically.

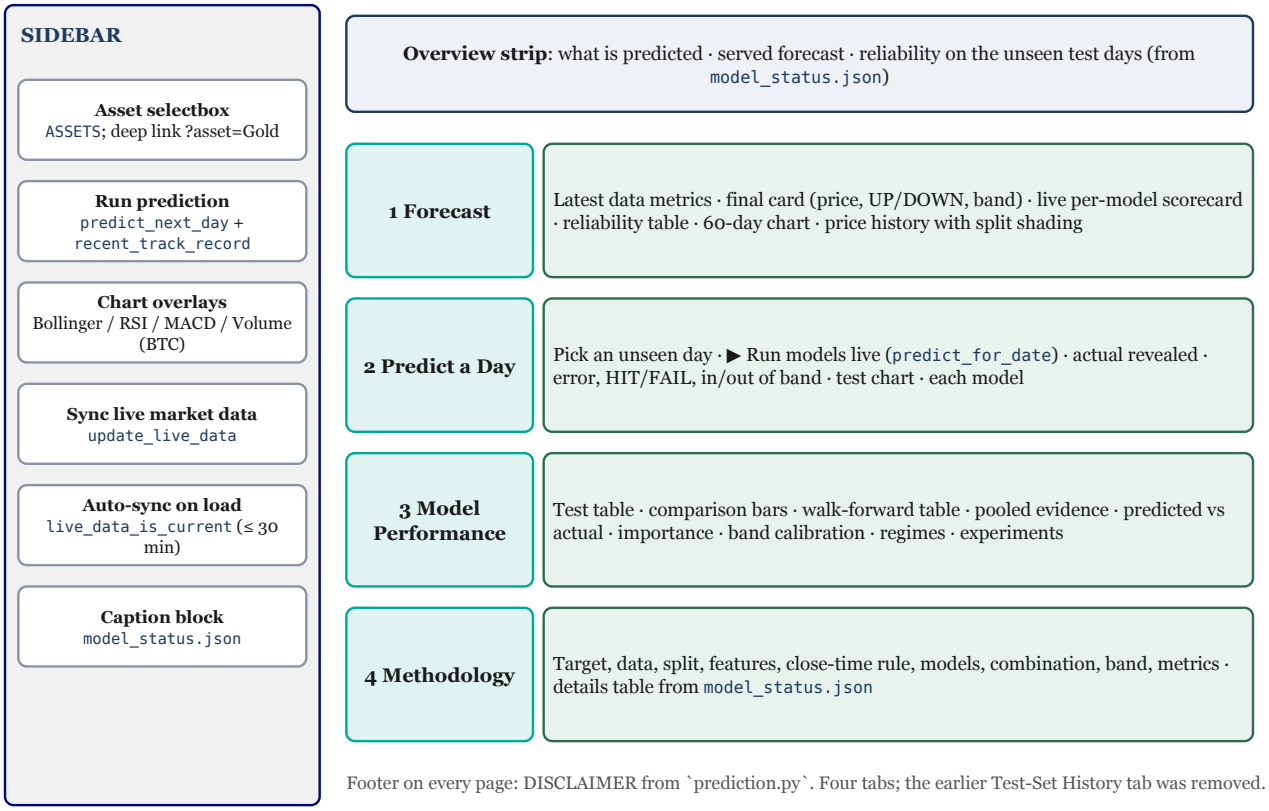


Figure 12.1: Dashboard map: sidebar controls on the left, the overview strip and the four tabs on the right, with the backend function behind each part.

12.2 Sidebar controls and what they call

Control	What it does	Backend logic	Status
Asset selectbox	Chooses Bitcoin, Gold or Silver for every tab	`ASSETS` from `config.py`; reads `status_all[asset]` and `combined_members(asset)`	Working
Run prediction (primary button)	Runs all trained models on the latest complete bar and shows the Combined forecast; also computes the live scorecard	Sets `session_state["run_prediction"]`; the Forecast tab then calls `predict_next_day` and `recent_track_record`	Working
Chart overlays (multiselect)	Adds Bollinger Bands, RSI(14), MACD(12,26,9) or (Bitcoin only) Volume under the price chart	Plain Plotly subplots from columns `BB_*`, `RSI`, `MACD`, `MACD_Signal`, `volume` in the loaded features	Working (display only; not model inputs)
Sync live market data	Downloads the latest bars into `data/raw_live/` and rebuilds the live feature file	`update_live_data(asset, refresh_external=True)`; clears `st.cache_data`; warns if Yahoo returns nothing	Working
(automatic) live check on load	If the live file does not end on the newest expected complete bar, downloads it; at most every 30 minutes per asset	`live_data_is_current`, file age check, `session_state["live_check_<asset>"]`	Working
Sidebar caption	Shows served forecast, best single model by validation, horizon, data source and last complete bar	`model_status.json`, `df.attrs["source"]`	Working

12.3 Tab 1: Forecast

Element	What the user sees	Backend
Overview strip (above all tabs)	What is predicted, the served forecast and its reliability on the unseen test days, including "The forecast does not beat the random walk (Diebold-Mariano p = ...)"	Numbers read from `model_status.json` (`combined_test`, `test_naive`), never typed in
Latest market data	Last complete close with change, 30-day high, 30-day low, 30-day volatility	`df = load_features(asset)` (live file if present); pandas `tail(30)`
Final prediction card	Predicted close for the next trading day, **UP ▲ / DOWN ▼**, % and USD change	`predict_next_day(asset, COMBINED)` → `predicted_price`, `direction`, `predicted_return_pct`
68 % band caption	Price interval, its ± %, σ and k, and its coverage on the unseen test period	`_band`; coverage from `band_calibration` in the result
Live per-model scorecard	A slider (20, 30, 60, 90 days), a progress box listing each model as it finishes, a table with each model's prediction for tomorrow, UP/DOWN, direction correct on the last N days, average price error, "better / same / worse" than "tomorrow = today", a green LIVE badge and a day-by-day expander	`recent_track_record(asset, n_days, progress)`: every model re-run on each of the last N days using only data up to each day; scored against the next close
Forecast chart	Last 60 days, the dashed forecast and the band	Plotly from `df.tail(60)` and the result
"How reliable is this forecast?" table	Combined versus random walk on the test set: MAE, RMSE, MAPE, direction, UP calls, DM verdict; a caption when the model says UP on 80 % or more of days	`final_test_results.csv` rows for Combined and Naive
Price history with split shading	Green train, amber validation, red test, range buttons (6M, 1Y, 2Y, All), optional indicator panes	`make_subplots`, `add_vrect`

Warnings shown: a yellow box if Yahoo has not yet published the newest expected bar (Bitcoin's previous-day bar is often a few hours late), a caption that continuous futures can differ after a contract roll, and

a caption when the forecast says UP on most test days (drift).

12.4 Tab 2: Predict a Day (unseen test)

- **What it is:** "stand on a day the models never saw, predict the next trading day, then reveal the truth".
- **Date list:** the 143 stored test days (frozen dataset) **plus every newer day from the live download**, newest first, marked `NEW`; the newest has "next close not published yet". The list therefore grows by itself.
- **▶ Run models live:** calls `predict_for_date(asset, day, COMBINED, progress=...)`. A status box shows four steps (hide the future → run each of six models with its prediction and milliseconds → average → reveal), then the green LIVE badge (time, seconds, data range, rows hidden).
- **Result:** five metrics (close on the day, predicted close, actual close, prediction error %, direction HIT / FAIL), the band caption with "inside / outside", a chart of the whole test period with the band and this prediction marked, and a per-model table.
- **Data used:** stored test days use the frozen dataset so they reproduce `results/predictions/*.csv` exactly (verified: maximum difference 2.7e-10 on 9 days); newer days use the live download.

12.5 Tab 3: Model Performance

Section	Content	Source file
Unseen test set table	All models plus Combined, sorted by RMSE (return), with "Honest reading" text generated from the numbers	`final_test_results.csv`
Comparison bars	RMSE of the return (lower is better) with the random-walk line; direction correct with the 50 % line	same
Walk-forward validation table	Mean ± sd over 4 folds, vs naive	`cv_results.csv`
Long-run evidence	Pooled walk-forward table	`walkforward_pooled.csv`
Predicted vs actual returns	Line chart and equal-axes scatter on the test period; caption on forecast amplitude	`results/predictions/*.csv`
Feature importance	Selectbox: CatBoost / LightGBM / RandomForest / Ridge; top 15 features	`feature_importance.csv`
Band calibration table	Coverage and half-width of served, ±1σ and fixed bands	`band_calibration.csv`
Expanders	Regime analysis; design experiments E1 to E4; training diagnostics (over-fitting gap and loss curves as images)	`regime_analysis.csv`, `experiments/*.csv`, `results/figures/*.png`

The training-diagnostics images are the PNGs saved earlier by `plots.py`; they are older than the final retrain of 2026-10-04 (see Appendix C), but their content (small train-versus-validation gaps, early-stopped loss curves) matches the current CSVs.

12.6 Tab 4: Methodology

A formatted text block generated from `model_status.json` and `config`: target, data and split, number of features, close-time rule (different wording for Bitcoin and the metals), models, how the combination works, the band, metrics. Below it, a "details" table: served forecast, members, combination rule, walk-forward RMSE of Combined versus random walk, best single model, its stopping point and hyper-parameters. This tab reads only stored values, so it never disagrees with the evaluation files.

12.7 Errors, warnings and messages

Situation	Message / behaviour	Where
Evaluation files missing	"No evaluation found. Run `make evaluate`..." (overview), "Run `make evaluate` first..." (Predict a Day), "Run `make evaluate` to generate results." (Model Performance)	tab code
Processed data missing	"No processed data found. Run `make preprocess` first."	Forecast tab
Yahoo returns nothing on Sync	Sidebar warning "Yahoo Finance did not return data (rate limit). Using the stored dataset."	Sidebar
Newest bar not yet published	Yellow warning naming the expected bar date and the date the forecast starts from	Forecast tab
Any exception during a prediction	`st.error("Prediction failed: ...")` and `logger.exception(...)`; the page keeps working	both prediction blocks
Stale `model_status.json`	Explicit `FileNotFoundError` text from `_band_k` or `_target_stats`, shown through the error box	prediction.py

12.8 Working, not connected, or not implemented?

> **Result of the check**
> **Working and connected:** every control, tab and table listed above. Each was exercised in the running app (Run prediction for Gold, ▶ Run models live for 2026-10-07, all four tabs read); no exception appeared in the page.
>
> **UI elements without backend:** none found. Streamlit's own "Deploy" toolbar button is framework chrome, not project code.
>
> **Not implemented:** model selection by the user (removed on purpose: the user never chooses a model), a Test-Set History tab (removed at the team's request, still reflected in the CHANGELOG), prediction logging, downloads, login, alerts, multi-day forecasts.

12.9 The user journey, from opening the page to the final result

1. Open `http://localhost:8501`. The app downloads a newer bar from Yahoo if one should exist (spinner "Checking Yahoo Finance for new Bitcoin data...").
2. The overview strip states the target, the served forecast and its reliability: direction correct 46.9 % (p = 0.83) for Bitcoin, and **"The forecast does not beat the random walk"**.
3. Pick an asset; read the latest price metrics.
4. Press **Run prediction**: after a few seconds the final card appears (e.g. for Gold on 2026-10-09 the app showed a predicted close for the next day, UP, with a ±1.09 % band), then the live scorecard of the six models and the chart.
5. Open **Predict a Day**, choose a day, press **Run models live**, watch each model, and compare predicted versus actual close, the HIT / FAIL and in-band mark.
6. Open **Model Performance** to see the evidence (tables, pooled record, band calibration) and **Methodology** to see how it was built.

Screenshots of these pages, captured from the running app on the day of writing, are in Appendix D.

Chapter 13: Strengths, Limitations and Future Improvements

13.1 Genuine technical strengths

Strength	Evidence in the repository
Leakage-audited pipeline	Close-time rule found and fixed a real leak (fake 62 % Silver result); target never in inputs; scaler fitted on train only; every guard has a test (Chapter 9.7)
Time-ordered validation with two layers of evidence	4-fold expanding walk-forward, one untouched test window, and the pooled 6-year out-of-fold record
Proper baselines and significance tests	Naive, Naive-Mean, ARIMA; Diebold-Mariano (HLN corrected), binomial tests, UP-call share to expose drift
Honest reporting	The dashboard itself says "The forecast does not beat the random walk" and shows the held-out metrics beside every number
Reproducibility	Fixed seeds, pinned versions, one config, one data loader; the 2026-10-04 audit re-ran everything from raw data and reproduced all results to 1e-13; today the live inference path reproduced stored test predictions to 2.7e-10
Engineering quality	Modular `src/` packages, 47 tests, CI workflow, explicit failure modes (stale files, NaN rows, missing artefacts raise clear errors), atomic writes of live files
Calibrated uncertainty	Test coverage 65.7 / 69.2 / 69.2 % for a nominal 68.3 %; derived only from information known on day t
Complete delivery	Dashboard with live demonstration modes, API, experiments, before/after comparison, documentation

13.2 Limitations by area

Area	Limitation	Consequence
Dataset quality and coverage	Free Yahoo data, about 8.7 years, one source; futures are continuous front-month series revised after rolls; Bitcoin's newest bar can be delayed	Small risk of silent data differences; a fresh download will not match the frozen files exactly
Historical financial data	Few regimes (one 2020 crash, one 2022 crypto winter, one 2025-26 metals rally); 1,874 to 2,750 training samples	Hard for 6 to 8 models to learn stable patterns; deep models see only a few hundred effective windows
Market volatility	Fat tails (Bitcoin excess kurtosis 15.2); validation and test periods are much more volatile than training for the metals (Silver validation RMS 0.055 vs 0.019 in training)	Errors cannot be compared across periods; the plain ±1σ band over-covered, hence the calibrated k
Model generalisation	No model beats the random walk; GRU and LSTM stop after 1 to 9 epochs; LightGBM memorises Bitcoin training data	The deployed models are effectively small-amplitude, near-constant predictors
Feature limitations	Only prices, indicators, five macro series, one sentiment index; heavy redundancy (15 to 18 correlated pairs); weekend macro values for Bitcoin repeat Friday's value	No news, no fundamentals; more of the same indicators adds variance, not signal (E2, E5b)
Validation reliability	Test window is only 143 to 207 days and was inspected during development; hyper-parameters were tuned on the folds that are pooled as evidence; validation metrics are optimistic	The test result alone could not detect a true effect smaller than about 5 to 10 % RMSE; the pooled record is stronger but not independent of tuning
Baseline performance	"Tomorrow = today" is as good as every model	The project's predictive claim is null; the contribution is the method and the measurement
Prediction uncertainty	The band describes size, not direction; coverage on the short validation window was 60.6 / 66.1 / 54.1 %, not exactly 68 %	Use it as a range of plausible prices, not a guarantee
External market events	News, policy decisions and shocks are not inputs	Any such event produces an error no model could avoid
Computational requirements	TensorFlow is a heavy dependency; full tuning about 30 minutes, training and evaluation about 15 minutes on a laptop; first dashboard prediction 15 to 20 s while models load; every prediction reloads models from disk	Fine for a demo; not designed for high-throughput serving

13.3 Future improvements (each with a reason)

Improvement	Justification from our own evidence	Effort
Make **volatility / range forecasting** the main task and the band a first-class output	E4: EWMA beats persistence by 4 to 8 %, Ridge by 11.5 % for Bitcoin; the band is the one calibrated, useful output. Candidate models: HAR-RV, GARCH family	Medium
Evaluate on a **fresh forward window with decisions frozen in advance** (tag a commit, then score only later days, e.g. with the existing Predict a Day tab)	The current test window was seen during development; a clean holdout would settle whether any weak effect exists	Low
Add genuinely new information (news sentiment, options-implied volatility, on-chain data) **with a close-time check for each series**	E2 and E5b show that more price-derived features add nothing; new series are the only plausible source of new signal, but each is a leak risk	High
Cache loaded models in the dashboard (`st.cache_resource`)	`_load` re-reads each artefact from disk on every prediction; first run takes 15 to 20 s	Low
Regenerate figures and refresh notebook text (`make figures`)	Dashboard shows `results/figures` PNGs older than the final retrain; notebook 01 still says "three years" (Appendix C)	Low
Add API tests and tighten CORS / add authentication if ever exposed beyond localhost	No test calls `src/api/app.py` directly; `allow_origins=["*"]` with credentials is permissive	Low
Complete `requirements-lock.txt` (CatBoost, FastAPI, uvicorn, ta are missing) and run CI on the same Python as local development (3.9 vs 3.11)	Fresh-clone reproducibility depends on it; the audit noted CI was not re-run	Low
Roll-adjusted futures series for the metals	Contract rolls create jumps in the continuous series; the live and frozen series can differ	Medium

> **What we do NOT claim**
> Perfect predictions, guaranteed profit, professional trading reliability, that deep learning helps, or that a directional hit-rate of 52 to 55 % is skill (for the metals it is below the "always UP" forecast).

Chapter 14: Project Security, Reliability and Reproducibility

14.1 What happens when something goes wrong

Failure	Actual behaviour in the code	Remaining limitation
Yahoo download fails or is rate-limited	`fetch_asset` catches exceptions, prints the error and returns an empty frame (existing CSV untouched); an empty response is reported as a possible rate limit. `update_live_data` returns `False` and never raises (test `test_live_sync_survives_network_failure`). The dashboard warns "Using the stored dataset".	No retry or back-off. `fetch_yfinance_external` has no `try` of its own (a failure inside `make collect-data` would raise); inside the dashboard the outer `try` catches it
Fear & Greed API fails	Warning printed, empty frame returned; "BTC model will train without it" (but the saved models expect the feature, so the file is needed for inference)	Missing `fear_greed_data.csv` makes preprocessing skip the feature and the scaler check then reports a mismatch
Missing or corrupt values	`validate_features` raises `ValueError` for NaN, infinite values or fewer than 100 rows; `update_live_data` then refuses to write the live file; inference raises on a NaN in the newest row (test `test_corrupt_live_data_raises_instead_of_returning_nan`)	—
Missing model artefacts	`combined_members` skips absent members; `_predict_from_window` raises `FileNotFoundError("No trained models found ... Run make train")` if none exist; the API returns **503**; the dashboard shows the error text	If only some members are missing, Combined is silently built from fewer models (the dashboard states the count)
Stale `model_status.json`	`_band_k` and `_target_stats` raise `FileNotFoundError` asking to re-run `make evaluate` (tests `test_missing_band_calibration_is_reported_clearly`, `test_missing_evaluation_is_reported_clearly`)	—
Non-finite model output	`_predict_one` raises `ValueError`	—
Unknown asset or model in the API	404 with the list of valid names	—

14.2 Dependencies, configuration and secrets

- **Installation:** `pip install -r requirements.txt` (main packages pinned with `==`; `scipy`, `ta`, `catboost`, `fastapi`, `uvicorn`, `pytest`, `tzdata` with `>=`). `requirements-lock.txt` is a full `pip freeze` last changed on 2026-07-25 and **does not list catboost, fastapi, uvicorn or ta**, so it should not be treated as complete.
- **Python:** the local venv is Python 3.9.6; the CI workflow uses 3.11 (not re-run during the audit). TensorFlow 2.16.2 pins the NumPy 1.26.4 line.
- **Configuration:** one file, `config.py` (paths, tickers, split dates, feature policy, default hyper-parameters, `SEQ_LEN`, `CV_FOLDS`). Importing it **creates** the data, results and tuning folders (a side effect worth knowing).
- **Environment variables:** none required. `.env.example` contains only `TF_CPP_MIN_LOG_LEVEL=3`; `.env` is in `.gitignore`. The code sets `TF_CPP_MIN_LOG_LEVEL` itself with `os.environ`.
- **API keys:** none. Yahoo Finance (yfinance) and alternative.me are unauthenticated; no secret is stored or committed.

14.3 Security observations

Observation	Detail	Risk level for a local prototype
API CORS	`allow_origins=["*"]`, `allow_credentials=True`, all methods and headers; no authentication or rate limiting	Low on localhost; must be tightened before any public deployment
Model files are pickles	`joblib.load` and Keras loading can execute code if a file is malicious	Low: models are generated locally and git-ignored; never load model files from untrusted sources
HTML in Streamlit	`unsafe_allow_html=True` is used with strings built only from the app's own numbers and result files	Low; no user-typed text is rendered as HTML
Subprocess use	`scripts/retrain.py` calls `subprocess.run([sys.executable] + fixed_list)`, no shell, no user input	Low
Network use	HTTPS downloads from Yahoo (via yfinance) and one `urllib` request with a browser User-Agent to alternative.me	Low

14.4 Reproducibility, serialisation and preprocessing consistency

- **Seeds:** `set_all_seeds(42)` (Python, NumPy, `PYTHONHASHSEED`, TensorFlow) before each Keras fit; `random_state=42` (Random Forest, LightGBM), `random_seed=42` (CatBoost).
- **Result:** the audit of 2026-10-04 re-ran preprocess, train, stack and evaluate from the raw files: every result file reproduced to ≤ 1e-13 and the scaled splits byte-for-byte. For this guide the live inference path was re-run on nine stored test days: maximum difference to `results/predictions` 2.7e-10 (Bitcoin), 4.8e-11 (Gold), 1.9e-10 (Silver). GPU non-determinism was not tested (CPU only).
- **Serialisation:** scikit-learn and LightGBM models and the scaler by `joblib` (`.pkl`), CatBoost natively (`.cbm`), Keras models as `.keras`; each with a JSON sidecar or inside the pickle that stores `params` and `fit_info`. Pickles depend on library versions (scikit-learn 1.6.1, LightGBM 4.6.0), so keep the pinned versions.
- **One preprocessing path:** the same `DataCleaner.clean_data` builds frozen and live features; `build_dataset` is the only loader for tuning, training, stacking and evaluation; inference uses the same saved `MinMaxScaler`, re-orders columns by `scaler.feature_names_in_` and rejects a different feature set **by name**. Tests cover look-ahead, column-order independence and failure modes.
- **Fresh clone:** `data/raw`, `data/processed` and `data/models` are git-ignored (only `model_status.json` is tracked), so run `make collect-data preprocess train stack evaluate` (about 15 minutes, using the committed `results/tuning/best_params.json`). Because Yahoo revises history, a fresh download gives slightly different numbers than the frozen files.

Chapter 15: Complete Project Summary (revision sheet)

What the project does	Forecasts the **next trading day's log return** of Bitcoin, Gold and Silver, shows it as a price with UP/DOWN and a 68 % band, and measures honestly whether this beats "tomorrow = today".
Data	Yahoo Finance daily bars 2018-01-01 to 2026-09-12 (frozen) for BTC-USD, GC=F, SI=F; DXY, WTI, VIX, 10-year yield, S&P 500; Fear & Greed. No nulls, no duplicates; complete bars only; 3,177 (Bitcoin) and 2,186 (metals) raw rows.
Features	29 / 28 / 29 stationary, backward-looking features: returns and lags, 20-day momentum, rolling / HAR / EWMA volatility, RSI, ADX, ROC, EMA ratios, MACD/price, Bollinger %B and width, ATR/price, high-low range, macro returns; Bitcoin adds Fear & Greed, weekday, volume change; Silver adds gold's return. Close-time rule for the metals.
Models	Six trained: Ridge, Random Forest, LightGBM, CatBoost, GRU, LSTM. Baselines: Naive (random walk), Naive-Mean (drift), ARIMA. Experiment: Stacked. Served: **Combined** (equal-weight mean of the six).
Split and validation	Train ≤ 2025-09-10 · validation ≤ 2026-02-17 · test 2026-02-18 to 2026-09-12/11 (207 / 143 / 143 days); scaler on train only; 4-fold expanding walk-forward for tuning and selection; pooled 2,328 / 1,584 out-of-fold days.
Evaluation	RMSE, MAE, R^2 (return); MAE, RMSE, MAPE (USD); direction accuracy with binomial p; UP-call share; Diebold-Mariano vs random walk and vs drift; strategy backtest; band coverage; regimes.
Final prediction	`predict_next_day`: scale last 30 rows → six models → $\hat{y}$ = mean → $\hat{P} = P_t \cdot \exp(\hat{y})$ → UP if $\hat{y} > 0$ → band $P_t \cdot \exp(\hat{y} \pm k \cdot \sigma_t)$.
What the results show	Combined vs random walk on test: -0.34 / -0.04 / -0.29 % RMSE (DM p 0.49 / 0.97 / 0.46); direction 46.9 / 50.3 / 53.8 % (p 0.83 / 0.50 / 0.20); pooled six years: no significant improvement. Band coverage on test 65.7 / 69.2 / 69.2 %.
Most important limitation	No predictive skill over the random walk; short test window that was inspected during development; metals' direction hit-rate is drift.
What we can confidently claim	(1) A leakage-audited, reproducible, tested pipeline. (2) A fair comparison of 9 models and 2 ensembles on identical days. (3) Time-series-correct validation with three layers of evidence. (4) The finding that next-day returns of these assets are statistically indistinguishable from a random walk with this data. (5) That volatility is partly predictable and our band is roughly calibrated. (6) That we found and removed a real data leak.
What we must not claim	Accurate price prediction; "98 % accuracy"; profit; deep-learning superiority; that 52 to 59 % direction is skill; that the test set was never looked at.

> **One-line answer for any "so what?" question**
> "We showed, with a fair and reproducible test, that these three markets are not predictable one day ahead from public daily data, and we built the tooling that makes that claim checkable."

PART 2
Complete VS Code Folder, File and Source Code Explanation

The actual repository, every folder, every meaningful file, and how the parts connect

8 chapters + glossary and appendices | file lists, import graphs and call tables were generated by analysing the real source files

---

Chapter 16: Complete Folder Structure

16.1 The actual folder and file tree

Generated from the file system on 2026-10-09 (commit `83323b4`). Folders and files marked `[generated]` or `[cache]` are not source code.

```
repository tree
    crypto-forex-prediction-system/
    (not shown: .git/, venv/, __pycache__/, .DS_Store, and the empty __init__.py file of each src package)
    ├── .claude/
    │   └── launch.json   # preview-server definitions
    ├── .github/
    │   └── workflows/
    │       └── ci.yml   # CI: imports + pytest
    ├── .pytest_cache/   # [cache] pytest
    │   └── (generated files)
    ├── .vscode/
    │   └── settings.json   # editor setting
    ├── app/   # Streamlit dashboard
    │   └── streamlit_app.py   # dashboard entry point (761 lines)
    ├── catboost_info/   # [generated] CatBoost training logs
    │   └── (generated files)
    ├── data/   # datasets and saved models
    │   ├── models/   # saved scalers and models (generated)
    │   │   ├── <p>_catboost.cbm   (p = btc, gold, silver)
    │   │   ├── <p>_catboost.cbm.json   (p = btc, gold, silver)
    │   │   ├── <p>_gru.keras   (p = btc, gold, silver)
    │   │   ├── <p>_gru.keras.json   (p = btc, gold, silver)
    │   │   ├── <p>_lightgbm.pkl   (p = btc, gold, silver)
    │   │   ├── <p>_lstm.keras   (p = btc, gold, silver)
    │   │   ├── <p>_lstm.keras.json   (p = btc, gold, silver)
    │   │   ├── <p>_randomforest.pkl   (p = btc, gold, silver)
    │   │   ├── <p>_ridge.pkl   (p = btc, gold, silver)
    │   │   ├── <p>_scaler.pkl   (p = btc, gold, silver)
    │   │   ├── <p>_stack.pkl   (p = btc, gold, silver)
    │   │   └── model_status.json
    │   ├── processed/   # features + scaled splits (generated)
    │   │   ├── <p>_features.csv   (p = btc, gold, silver)
    │   │   ├── <p>_live_features.csv   (p = btc, gold, silver)
    │   │   ├── <p>_test_scaled.csv   (p = btc, gold, silver)
    │   │   ├── <p>_train_scaled.csv   (p = btc, gold, silver)
    │   │   └── <p>_val_scaled.csv   (p = btc, gold, silver)
    │   ├── raw/   # FROZEN raw downloads (git-ignored)
    │   │   └── 9 CSVs: bitcoin gold silver dxy crude_oil vix tnx sp500 fear_greed
    │   ├── raw_3y_backup/   # old 3-year files (tracked)
```

```
repository tree  (continued)
     │   │   └── 9 CSVs: bitcoin gold silver dxy crude_oil vix tnx sp500 fear_greed
     │   └── raw_live/   # live downloads (git-ignored)
     │       └── 9 CSVs: bitcoin gold silver dxy crude_oil vix tnx sp500 fear_greed
     ├── docs/   # PDF documents only
     │   ├── FYP_Complete_Study_Guide.pdf
     │   └── FYP_Technical_Report.pdf
     ├── notebooks/   # 4 companion notebooks (display saved results)
     │   ├── 01_data_exploration.ipynb
     │   ├── 02_feature_engineering.ipynb
     │   ├── 03_model_training.ipynb
     │   └── 04_evaluation.ipynb
     ├── results/   # all evaluation output
     │   ├── archive_3y_final/   # archived 3-year system
     │   │   └── 11 archived files (README or FINAL_RESULTS inside)
     │   ├── archive_pre_fix/   # archived pre-audit results
     │   │   └── 35 archived files (README or FINAL_RESULTS inside)
     │   ├── archive_sameday_macro_leak/   # archived results with the leak
     │   │   └── 14 archived files (README or FINAL_RESULTS inside)
     │   ├── experiments/   # E1 to E5 outputs
     │   │   └── E1..E5 CSVs, before_after.csv, E5_summary.md, experiments_summary.md, run.log (9 files)
     │   ├── figures/   # PNG figures (older than last retrain)
     │   │   └── 24 PNG files (price_history, model_comparison_*, overfitting_gap, eda_*, per-asset plots)
     │   ├── histories/   # GRU/LSTM loss curves (json)
     │   │   └── 6 files: bitcoin_gru.json, bitcoin_lstm.json, gold_gru.json, ...
     │   ├── predictions/   # one row per unseen test day
     │   │   └── 3 files: btc_test_predictions.csv, gold_test_predictions.csv, silver_test_predictions.csv
     │   ├── stacking/   # stack weights
     │   │   └── 3 files: bitcoin_stack_weights.json, gold_stack_weights.json, silver_stack_weights.json
     │   ├── thumbnail/   # [empty folder] tooling leftover
     │   │   └── cutouts/
     │   ├── tuning/   # grid-search logs + best_params.json
     │   │   └── best_params.json, tuning_summary.csv, <asset>_<model>.csv  (20 files)
     │   ├── band_calibration.csv
     │   ├── cv_results.csv
     │   ├── eda_summary.csv
     │   ├── feature_importance.csv
     │   ├── FINAL_RESULTS.md
     │   ├── final_test_results.csv
     │   ├── regime_analysis.csv
     │   ├── train_val_metrics.csv
     │   └── walkforward_pooled.csv
     ├── scripts/   # one orchestration script
     │   └── retrain.py   # runs the whole pipeline end to end
     ├── src/   # all importable project code
     │   ├── api/   # FastAPI service
     │   │   └── app.py
     │   ├── data/   # download, clean, features, split, live sync, EDA
     │   │   ├── data_collection.py
     │   │   ├── eda.py
     │   │   ├── external_data.py
     │   │   ├── market_calendar.py
     │   │   ├── preprocessing.py
     │   │   └── sync_live_data.py
     │   ├── evaluation/   # walk-forward CV, final evaluation, figures
     │   │   ├── backtesting.py
     │   │   ├── cross_validation.py
     │   │   └── plots.py
     │   ├── experiments/   # validation-only design experiments
     │   │   ├── audit_improvements.py
     │   │   ├── before_after.py
     │   │   └── run_experiments.py
     │   ├── inference/   # prediction used by dashboard and API
     │   │   └── prediction.py
     │   ├── models/   # model registry + architectures + stack
     │   │   ├── ensemble_model.py
     │   │   ├── model_gru.py
     │   │   ├── model_lstm.py
     │   │   └── registry.py
     │   ├── training/   # tuning and two-phase training
     │   │   ├── train_models.py
```

repository tree (continued) (continued)

```
│   │   └── tune_models.py
│   ├── utils/   # metrics, seeds, logging, price reconstruction
│   │   ├── inverse_transform.py
│   │   ├── logging_config.py
│   │   ├── metrics.py
│   │   └── reproducibility.py
│   └── README.md
├── tests/   # 47 pytest tests
│   ├── conftest.py
│   ├── test_features.py
│   ├── test_market_calendar.py
│   ├── test_metrics.py
│   ├── test_models_and_inference.py
│   └── test_sequences_and_split.py
├── .env.example   # only TF log level, no secrets
├── .gitignore   # ignores data, models, venv, caches
├── CHANGELOG.md   # history of fixes and audits
├── config.py   # central configuration
├── FINAL_AUDIT.md   # Phase-2 audit report + 110 viva Q&A
├── Makefile   # one command per stage
├── README.md   # project overview
├── requirements-lock.txt   # full freeze (incomplete, see Ch. 14)
└── requirements.txt   # pinned dependencies
```

16.2 Every folder: purpose, contents, connections

Folder	Purpose	Contains	Why it exists / how it connects
app/	The user interface: one Streamlit script.	streamlit_app.py	Needed to present results. Imports config, src.inference.prediction, src.data.sync_live_data, src.data.market_calendar.
src/	All importable project code, organised by pipeline stage.	Seven sub-packages (data, models, training, evaluation, inference, experiments, utils) plus api	Core of the project. src/README.md summarises it.
src/data/	Acquire, clean, engineer, split, scale, live-sync, explore.	6 modules	First stage of the pipeline; build_dataset (preprocessing) is used by almost everything downstream.
src/models/	One interface for all models; the two network architectures; the stacked experiment.	registry.py, model_gru.py, model_lstm.py, ensemble_model.py	Used by training, evaluation and inference.
src/training/	Tune hyper-parameters; train the final models.	tune_models.py, train_models.py	Writes best_params.json and data/models/*.
src/evaluation/	Walk-forward CV, the one test evaluation, report figures.	cross_validation.py, backtesting.py, plots.py	Writes results/* and model_status.json.
src/inference/	Turn saved models into a forecast.	prediction.py	The only module the dashboard and the API call for forecasts.
src/experiments/	Validation-only design experiments and the before/after comparison.	run_experiments.py, audit_improvements.py, before_after.py	Not needed to run the product; they document why the design is what it is.
src/utils/	Shared helpers.	metrics.py, inverse_transform.py, reproducibility.py, logging_config.py	Imported across the package.
src/api/	FastAPI service.	app.py	Alternative front end; calls prediction.py.
scripts/	Orchestration.	retrain.py	Runs the whole pipeline with subprocesses.
tests/	Automated tests (47).	5 test modules + conftest.py	Run by make test and CI.
notebooks/	Four companion notebooks that load and display saved results.	01_data_exploration … 04_evaluation	Presentation aids; the scripts in src/ are the source of truth.
data/raw/	Frozen downloaded CSVs (9 files).	*_data.csv	Input of preprocessing; git-ignored.
data/raw_live/	Live downloads for the dashboard.	9 CSVs	Written by sync_live_data; git-ignored.
data/raw_3y_backup/	The earlier 3-year raw files.	9 CSVs	Kept (tracked) for the before/after comparison.
data/processed/	Engineered features and scaled splits.	<p>_features.csv, <p>_{train,val,test}_scaled.csv, <p>_live_features.csv	Generated; read by build_dataset and the dashboard.
data/models/	Scalers, trained models, stacks and model_status.json.	34 files	Generated; only model_status.json is tracked.
results/	Every evaluation output.	CSVs, FINAL_RESULTS.md, subfolders	Read by the dashboard; evidence for the report.

Folder	Purpose	Contains	Why it exists / how it connects
`docs/`	Generated PDF documents only.	`FYP_Technical_Report.pdf`, `FYP_Complete_Study_Guide.pdf`	Documentation; this guide is a newer, larger document.
`.github/workflows/`	CI definition.	`ci.yml`	GitHub Actions.
`.vscode/`, `.claude/`	Editor and preview settings.	`settings.json`, `launch.json`	Convenience only.
`catboost_info/`, `.pytest_cache/`	Caches created by CatBoost and pytest.	logs / node ids	Generated; safe to delete (`make clean`).

16.3 Every Python file at a glance

Path	Lines	Purpose	Project modules it imports	Run by / imported by	Stage
app/streamlit_app.py	761	The dashboard	config.py, market_calendar.py, sync_live_data.py, prediction.py	make serve	Serve
config.py	136	Central configuration: paths, tickers, split dates, feature policy, task definitions, default and tuned hyper-parameters	—	streamlit_app.py, app.py, data_collection.py, eda.py +14 more + 3 test file(s)	any
scripts/retrain.py	54	End-to-end orchestration with subprocess	—	python scripts/retrain.py	Run
src/api/app.py	94	FastAPI service: /health, /models, /predict/{asset}	config.py, prediction.py	make api	Serve
src/data/data_collection.py	65	Download daily OHLCV for the 3 assets with the complete-bar rule	config.py, market_calendar.py	make collect-data; sync_live_data.py	Acquire
src/data/eda.py	85	ADF tests, ACF/PACF, return distributions, volatility clustering	config.py	make eda	Explore
src/data/external_data.py	174	Download macro series and Fear & Greed	config.py, market_calendar.py	make collect-data; sync_live_data.py	Acquire
src/data/market_calendar.py	60	Complete-bar rule, next trading day, expected last bar	—	streamlit_app.py, data_collection.py, external_data.py, prediction.py + 1 test file(s)	Acquire / inference
src/data/preprocessing.py	412	DataCleaner (cleaning + features + split + scaler) and build_dataset (the single loader)	config.py	make preprocess; sync_live_data.py, backtesting.py, plots.py, audit_improvements.py +4 more + 2 test file(s)	Preprocess
src/data/sync_live_data.py	57	Refresh live data and write <p>_live_features.csv atomically	config.py, data_collection.py, external_data.py, preprocessing.py	dashboard / python src/data/sync_live_data.py; streamlit_app.py + 1 test file(s)	Live
src/evaluation/backtesting.py	386	The one test evaluation, pooled tests, band calibration, model_status.json, FINAL_RESULTS.md	config.py, preprocessing.py, cross_validation.py, prediction.py, ensemble_model.py, registry.py, logging_config.py, metrics.py	make evaluate	Evaluate
src/evaluation/cross_validation.py	101	Walk-forward folds and cv_evaluate	config.py, registry.py, metrics.py	backtesting.py, audit_improvements.py, run_experiments.py, ensemble_model.py +1 more	Validate
src/evaluation/plots.py	231	Report figures	config.py, preprocessing.py	make figures	Report

Path	Lines	Purpose	Project modules it imports	Run by / imported by	Stage
src/experiments/audit_improvements.py	104	Experiment E5 (audit candidates)	config.py, preprocessing.py, cross_validation.py, registry.py, logging_config.py, metrics.py	make experiments	Experiment
src/experiments/before_after.py	62	Compare the 3-year system with the final one on identical days	config.py, metrics.py	make experiments	Experiment
src/experiments/run_experiments.py	187	Experiments E1 to E4	config.py, preprocessing.py, cross_validation.py, registry.py, logging_config.py, metrics.py	make experiments	Experiment
src/inference/prediction.py	357	Next-day / predict-for-date / live scorecard / history	config.py, market_calendar.py, ensemble_model.py, registry.py, logging_config.py	make predict; streamlit_app.py, app.py, backtesting.py + 1 test file(s)	Predict
src/models/ensemble_model.py	83	Stacked ensemble: fit on out-of-fold predictions; StackedModel wrapper	config.py, preprocessing.py, cross_validation.py, registry.py, logging_config.py	make stack; backtesting.py, prediction.py	Models
src/models/model_gru.py	23	Keras GRU architecture builder	—	registry.py	Models
src/models/model_lstm.py	19	Keras LSTM architecture builder	—	registry.py	Models
src/models/registry.py	365	make_model, nine model classes, save/load, artefact helpers	config.py, model_gru.py, model_lstm.py, reproducibility.py	backtesting.py, cross_validation.py, audit_improvements.py, run_experiments.py +3 more + 1 test file(s)	Models
src/training/train_models.py	108	Two-phase training, saving, train/val metrics, importances	config.py, preprocessing.py, registry.py, logging_config.py, metrics.py	make train	Train
src/training/tune_models.py	97	Grid search scored by walk-forward CV	config.py, preprocessing.py, cross_validation.py, logging_config.py	make tune	Tune
src/utils/inverse_transform.py	21	reconstruct_price: P·exp(r)	—	tests	Evaluate / tests
src/utils/logging_config.py	39	setup_logging, get_logger	—	backtesting.py, audit_improvements.py, run_experiments.py, prediction.py +3 more	any
src/utils/metrics.py	165	RMSE, MAE, MAPE, R², directional accuracy, Diebold-Mariano, backtest, band coverage, QLIKE	—	backtesting.py, cross_validation.py, audit_improvements.py, before_after.py +2 more + 1 test file(s)	Evaluate
src/utils/reproducibility.py	28	set_all_seeds	—	registry.py	Train

Path	Lines	Purpose	Project modules it imports	Run by / imported by	Stage
`tests/conftest.py`	38	Shared fixtures (synthetic prices, sequences, project root)	—	—	Test
`tests/test_features.py`	105	Look-ahead, close-time alignment, calendar, indicator bounds (9 tests)	`config.py`, `preprocessing.py`	—	Test
`tests/test_market_calendar.py`	25	Complete-bar rule and next trading day (3 tests)	`market_calendar.py`	—	Test
`tests/test_metrics.py`	54	Metric definitions (7 tests)	`metrics.py`	—	Test
`tests/test_models_and_inference.py`	172	Registry contract, inference, band, failure modes, scorecard (18 test cases)	`config.py`, `sync_live_data.py`, `prediction.py`, `registry.py`	—	Test
`tests/test_sequences_and_split.py`	73	Target alignment, split integrity, reconstruction, scaler (10 test cases)	`config.py`, `preprocessing.py`, `inverse_transform.py`	—	Test

The "imports" and "imported by" columns were generated by parsing the `import` statements of all 33 Python files (plus 9 tiny `__init__.py` files that mark packages). Each file is explained in depth in Chapter 17.

16.4 Non-Python files and artefacts

Path	Kind	Purpose and contents	Written by	Read by
`Makefile`	config	Targets: install, test, collect-data, preprocess, eda, tune, train, stack, evaluate, figures, experiments, pipeline, predict, serve, api, clean	hand	developer
`requirements.txt`	config	20 requirement lines (pandas, NumPy, scikit-learn, TensorFlow, LightGBM, CatBoost, statsmodels, yfinance, Streamlit, Plotly, FastAPI, ...)	hand	`pip`, CI
`requirements-lock.txt`	config	164-line `pip freeze` from 2026-07-25; lacks CatBoost, FastAPI, uvicorn, ta	frozen	optional
`.env.example`	config	Documents the only optional variable (`TF_CPP_MIN_LOG_LEVEL`); no secrets	hand	developer
`.gitignore`	config	Ignores `venv`, `.env`, `__pycache__`, `data/raw*`, `data/processed`, `data/models` (except `model_status.json`), caches	hand	git
`.github/workflows/ci.yml`	config	Python 3.11, install, 4 import checks, `pytest`	hand	GitHub Actions
`README.md`	doc	Overview, results at a glance, quick start, limitations, API	hand	readers
`CHANGELOG.md`	doc	Dated entries: fixes, audits, removals	hand	readers
`FINAL_AUDIT.md`	doc	26-phase audit of 2026-10-04 with verdict and 110 viva questions	hand	team
`src/README.md`	doc	Short guide to the package layout	hand	readers
`docs/FYP_Technical_Report.pdf`, `docs/FYP_Complete_Study_Guide.pdf`	doc	Earlier generated PDFs; their uncertainty-band paragraph predates the 2026-10-04 calibration (FINAL_AUDIT.md §23). **This guide supersedes the study guide.**	generators (not in repo)	readers
`data/raw/*.csv`	data	Frozen downloads, 9 files (Chapter 4)	`data_collection.py`, `external_data.py`	`DataCleaner`
`data/processed/<p>_features.csv`	data	Unscaled features, chart columns and `log_return` (3,147 / 2,156 / 2,156 rows)	`DataCleaner.save_data`	`build_dataset`, dashboard, `eda.py`, `plots.py`
`data/processed/<p>_{train,val,test}_scaled.csv`	data	Scaled model features of each split	`save_data`	`build_dataset`
`data/processed/<p>_live_features.csv`	data	Latest live features	`sync_live_data`	`prediction.py`, dashboard
`data/models/*`	artefact	Scalers, 18 models, 3 stacks, `model_status.json` (Chapter 9.6)	training, evaluation	inference, dashboard, API
`results/final_test_results.csv`	result	Test metrics per asset and model (33 rows)	`backtesting.py`	dashboard, `plots.py`
`results/cv_results.csv`	result	Walk-forward mean ± sd per model	`backtesting.py`	dashboard, `plots.py`
`results/walkforward_pooled.csv`	result	Pooled out-of-fold tests	`backtesting.py`	dashboard
`results/band_calibration.csv`, `regime_analysis.csv`, `eda_summary.csv`, `train_val_metrics.csv`, `feature_importance.csv`	result	Band coverage, regimes, EDA statistics, phase-A metrics, importances	evaluation / training / EDA	dashboard, notebooks

Path	Kind	Purpose and contents	Written by	Read by
`results/FINAL_RESULTS.md`	result	Auto-written readable tables	`backtesting.write_markdown`	readers
`results/predictions/*_test_predictions.csv`	result	One row per unseen test day with each model's prediction, error, hit and band	`backtesting.py`	dashboard (`prediction_history`)
`results/tuning/*`	result	Per-configuration CV scores, `best_params.json`, `tuning_summary.csv`	`tune_models.py`	`config.get_params`, `backtesting.py`
`results/experiments/*`	result	E1 to E5 tables and summaries	experiments	dashboard expander, docs
`results/histories/*.json`, `results/stacking/*.json`	result	Loss curves; stack weights	training / stacking	`plots.py`
`results/figures/*.png`	figure	24 PNGs (EDA, comparisons, per-asset plots)	`eda.py`, `plots.py`	dashboard expander, notebooks
`notebooks/*.ipynb`	notebook	Load and display saved results (they do not train)	hand	Jupyter

16.5 Generated files, caches and files that are not application source

Item	What it is	Treat as
`__pycache__/` (in every package)	Python byte-code caches	Cache: delete freely (`make clean`)
`.pytest_cache/`	pytest bookkeeping	Cache
`catboost_info/`	CatBoost training logs and TensorBoard events, rewritten at every CatBoost fit (also by the tests)	Generated; ignored by git
`.DS_Store`	macOS Finder metadata	Noise
`venv/`	The local Python environment	Not part of the repository
`data/processed/`, `data/models/` (except `model_status.json`), `data/raw/`, `data/raw_live/`	Generated by the pipeline	Regenerate with `make`; not in git
`results/archive_*`	Frozen results of earlier versions (3-year system, pre-audit, same-day-macro leak) with explanations	History; useful for the before/after story, not for running the product
`results/thumbnail/cutouts/`	Empty folder	Leftover, safe to delete
`data/raw_3y_backup/`	Old 3-year raw files (tracked)	History; used by nothing at run time
`docs/*.pdf`	Previously generated PDFs	Documentation, partly superseded
`results/figures/*.png`	Generated by `plots.py` and `eda.py` before the final retrain on 2026-10-04	Regenerate with `make figures` before publishing

Chapter 17: Every Important Source File Explained

One subsection per Python file, in pipeline order. "Used by" is generated from the real import statements. Line numbers in the code excerpts are the real line numbers in VS Code. Chapter 18 gives the function-by-function table, Chapter 19 the connection diagrams and Chapter 20 the code you must know best.

config.py

Purpose	The single place where "where is this decided?" is answered: paths, data sources, the frozen split, the target, the feature policy, default and tuned hyper-parameters, and which model status file to read.
Imports	`os`, `json` only. (Everything else imports *from* it: 21 imports across the project.)
Functions and classes	`get_prefix(asset_name)` → `"btc" / "gold" / "silver"`. `get_params(model_name, asset_name=None, task="return_1d")` → dict: defaults updated with the tuned values from `results/tuning/best_params.json`. `load_model_status()` → dict from `data/models/model_status.json`, or an "untrained" placeholder.
Important variables	`ASSET_CONFIG` (ticker, type crypto/commodity, filename, prefix), `ASSETS`, `DATA_START_DATE="2018-01-01"`, `TARGET_COL="log_return"`, `PREDICTION_HORIZON_DAYS=1`, `TASKS` (return_1d, return_5d, vol_5d, vol_22d), `TRAIN_END="2025-09-10"`, `VAL_END="2026-02-17"`, `LEVEL_COLUMNS` (13), `EXTERNAL_SAME_DAY`, `POST_SETTLEMENT_FEATURES`, `SEQ_LEN=30`, `DEFAULT_PARAMS`, `CV_FOLDS=4`, `MODEL_STATUS` (a snapshot read once at import).
Key processing steps	On import: computes absolute paths and **creates** the data/results/tuning/experiments folders. `get_params` copies the defaults and overlays the best params for (task, asset, model) if the JSON exists.
Errors handled	`get_params` and `load_model_status` tolerate missing files (defaults / "untrained"). The consumers raise their own explicit errors when a *required* value (target mean/std, band k) is missing.
Used by (generated)	`app/streamlit_app.py`, `src/api/app.py`, `src/data/data_collection.py`, `src/data/eda.py`, `src/data/external_data.py`, `src/data/preprocessing.py`, `src/data/sync_live_data.py`, `src/evaluation/backtesting.py`, `src/evaluation/cross_validation.py`, `src/evaluation/plots.py`, `src/experiments/audit_improvements.py`, `src/experiments/before_after.py`, `src/experiments/run_experiments.py`, `src/inference/prediction.py`, `src/models/ensemble_model.py`, `src/models/registry.py`, `src/training/train_models.py`, `src/training/tune_models.py`; tests: `test_features.py`, `test_models_and_inference.py`, `test_sequences_and_split.py`
If removed or broken	Every module fails at import: this file is the root of the dependency graph.

config.py (how tuned parameters reach training)

```
109 def get_params(model_name, asset_name=None, task=DEFAULT_TASK):
110     """Tuned params for (task, asset, model) if tuning has been run, else defaults.
111     best_params.json layout: {task: {asset: {model: params}}} (the served task is 'return_1d')."""
112     params = dict(DEFAULT_PARAMS[model_name])
113     if asset_name and os.path.exists(BEST_PARAMS_PATH):
114         with open(BEST_PARAMS_PATH) as f:
115             best = json.load(f)
116         params.update(best.get(task, {}).get(asset_name, {}).get(model_name, {}))
117     return params
```

`best.get(task, {}).get(asset_name, {}).get(model_name, {})` walks the JSON layout `{task: {asset: {model: params}}}` and returns `{}` if any level is missing, so training never crashes for lack of a tuned value; it falls back to `DEFAULT_PARAMS`.

src/data/data_collection.py

Purpose	Download daily OHLCV for Bitcoin, Gold and Silver and save `data/raw/<asset>_data.csv` (or another folder for live refreshes).
Imports	`os`, `argparse`, `pandas`, `yfinance as yf`, `sys`; project: `config` (`RAW_DATA_DIR`, `ASSET_CONFIG`, `DATA_START_DATE`), `drop_incomplete_bars`.
Functions and classes	`fetch_asset(asset_name, start=DATA_START_DATE, out_dir=RAW_DATA_DIR)` → DataFrame (`timestamp, open, high, low, price, volume`). `verify_data(df, asset_name)` prints range, price min/max, nulls and shape.
Important variables	`cfg` (ticker and type), `df`, `path`.
Key processing steps	`yf.Ticker(ticker).history(start=...)` → keep Date/Open/High/Low/Close/Volume → rename Close to `price` → remove time zone → drop NaN closes → **drop incomplete bars** → write CSV.
Errors handled	Any network exception returns an empty DataFrame with the right columns; an empty response prints a rate-limit warning and **leaves the existing CSV untouched**.
Used by (generated)	`src/data/sync_live_data.py` Run by `make collect-data` and `scripts/retrain.py`.
If removed or broken	No fresh data; the dashboard cannot sync; the frozen CSVs would never be created.

```
src/data/data_collection.py  (fetch_asset)
28      cfg = ASSET_CONFIG[asset_name]
29      print(f"\nFetching {asset_name} ({cfg['ticker']}) from {start}...")
30      try:
31          df = yf.Ticker(cfg['ticker']).history(start=start)
32      except Exception as e:
33          print(f"Network error fetching data for {asset_name}: {e}")
34          return pd.DataFrame(columns=['timestamp', 'open', 'high', 'low', 'price', 'volume'])
35      if df.empty:
36          print(f"WARNING: empty response for {asset_name} (Yahoo rate limit?). Existing CSV left
                untouched.")
37          return df
38      df = df.reset_index()[['Date', 'Open', 'High', 'Low', 'Close', 'Volume']]
39      df.columns = ['timestamp', 'open', 'high', 'low', 'price', 'volume']
40      df['timestamp'] = pd.to_datetime(df['timestamp']).dt.tz_localize(None)
41      df = df.dropna(subset=['price'])
42      n = len(df)
43      df = drop_incomplete_bars(df, cfg['type'])
44      if len(df) < n:
45          print(f"  dropped {n - len(df)} incomplete (still-running) bar(s); last complete bar
                {df['timestamp'].max().date()}")
46      os.makedirs(out_dir, exist_ok=True)
47      path = os.path.join(out_dir, cfg['filename'])
48      df.to_csv(path, index=False)
49      print(f"  {len(df)} rows, {df['timestamp'].min().date()} → {df['timestamp'].max().date()} saved
            to {path}")
50      return df
```

Block by block. (1) try/except keeps an outage from crashing the caller. (2) `df.empty` protects the previous file from being overwritten with nothing. (3) the column selection and rename give every asset the same schema. (4) `drop_incomplete_bars(df, cfg["type"])` removes today's running bar (Chapter 4.1). (5) the CSV is only written if the download succeeded.

src/data/external_data.py

Purpose	Download the five macro series and the Fear & Greed index.
Imports	`os, sys, json, pandas, yfinance`, `urllib.request` (inside a function); `config`, `drop_incomplete_bars`.
Functions and classes	`fetch_yfinance_external(source_name, out_dir)` → DataFrame (`timestamp, price`). `fetch_fear_greed_index(out_dir)` → DataFrame (`timestamp, fear_greed`). `fetch_all_external_data(out_dir)` → dict of results.
Important variables	`EXTERNAL_SOURCES` = {DXY: DX-Y.NYB, CrudeOil: CL=F, VIX: ^VIX, TreasuryYields: ^TNX, SP500: ^GSPC} with file names.
Key processing steps	Yahoo history from 2018-01-01 → keep `Date, Close` → drop incomplete US-session bar → save. Fear & Greed: GET `https://api.alternative.me/fng/?limit=0&format=json` (browser User-Agent, 30 s timeout) → convert Unix seconds to dates → sort → save.
Errors handled	Fear & Greed errors are caught: warning "This is optional" and an empty frame. Unknown source name raises `ValueError`. (Yahoo failures inside `fetch_yfinance_external` are not caught here; callers such as `update_live_data` wrap them.)
Used by (generated)	`src/data/sync_live_data.py`
If removed or broken	Macro and sentiment features disappear (preprocessing prints "WARNING: ... not found, skipping").

src/data/market_calendar.py

Purpose	The complete-bar rule and trading-calendar helpers shared by collection, live sync and inference.
Imports	`datetime`, `zoneinfo.ZoneInfo`, `pandas`.
Functions and classes	`complete_bar_cutoff(asset_type, now=None)` → first date whose bar is not complete. `drop_incomplete_bars(df, asset_type, now=None, date_col="timestamp")` → rows with date < cutoff. `next_trading_day(asset_type, date)` → next calendar day (crypto) or next weekday (futures, holidays not modelled). `expected_last_complete_bar(asset_type, now=None)`.
Important variables	`ET = ZoneInfo("America/New_York")`, `SESSION_END_ET = (17, 15)`.
Key processing steps	Crypto: cutoff = today's UTC date. Commodity/US series: today's ET date before 17:15 ET, tomorrow's after.
Errors handled	Naive `now` is treated as UTC. The module itself has no failure path.
Used by (generated)	`app/streamlit_app.py`, `src/data/data_collection.py`, `src/data/external_data.py`, `src/inference/prediction.py`; tests: `test_market_calendar.py`
If removed or broken	Live sync would keep intraday snapshots as "closes"; the dashboard could not label target dates or detect stale data.

src/data/market_calendar.py (complete_bar_cutoff)

```
24 def complete_bar_cutoff(asset_type, now=None):
25     """
26     First calendar date whose daily bar is NOT yet complete. Rows dated on or after it must be
           dropped.
27         crypto     → today's UTC date
28         commodity  → today's ET date before 17:15 ET, tomorrow's ET date after
29     """
30     now = now or dt.datetime.now(dt.timezone.utc)
```

src/data/market_calendar.py (complete_bar_cutoff) (continued)

```
    if now.tzinfo is None:
        now = now.replace(tzinfo=dt.timezone.utc)
    if asset_type == 'crypto':
        return now.astimezone(dt.timezone.utc).date()
    et = now.astimezone(ET)
    return et.date() + dt.timedelta(days=1) if (et.hour, et.minute) >= SESSION_END_ET else et.date()
```

src/data/preprocessing.py

Purpose	The heart of the data side: `DataCleaner` turns a raw CSV into stationary features, applies the close-time rule, splits chronologically and fits the scaler; `build_dataset` is the one loader used by every later stage.
Imports	`os, joblib, numpy, pandas`, `sklearn.preprocessing.MinMaxScaler`, `ta` (inside `add_advanced_ta`); project: `config` (paths, split dates, `SEQ_LEN`, `TASKS`, feature-policy constants).
Functions and classes	**Class** `DataCleaner(asset_name, raw_dir)`: `load_data`, `add_moving_averages`, `add_rsi(window=14)`, `add_macd`, `add_advanced_ta`, `add_relative_features`, `add_lag_returns`, `add_rolling_volatility`, `add_calendar_features`, `add_volume_change`, `lag_post_settlement_features`, `_align_external`, `_read_external`, `add_external_features`, `clean_data`, `model_feature_columns`, `validate_features`, `normalize_and_split`, `save_data`. **Functions:** `create_sequences(data, seq_len, target_col)` → (X, y); `_make_windows(values, seq_len)`; `_task_target(log_return, task)`; `build_dataset(asset_name, seq_len, task, train_start)` → dict of arrays; `run_cleaning_pipeline()`.
Important variables	`self.df` (working DataFrame), `self.scaler`, `self.train_scaled/val_scaled/test_scaled`; in `build_dataset`: `masks`, `mu`, `sd`, `fwd/inv` (standardise/de-standardise), `periods_per_year` (365 or 252).
Key processing steps	`clean_data`: dedupe/sort/positive price → Bitcoin re-index and ffill, metals keep trading days → `log_return` → all `add_*` → external alignment → `dropna` → `validate_features`. `normalize_and_split`: slice by date, fit scaler on train, transform all. `build_dataset`: read scaled CSVs → windows → target → drop rows without target → date masks → standardise → assemble the dict (`X_*`, `Xt_*`, `y_*`, `y_real_*`, `dates_*`, `target_dates_*`, `prev_*`, `true_*`, `naive_*`).
Errors handled	`load_data` returns False and prints a message if the file is missing; `validate_features` raises `ValueError` (NaN, inf, < 100 rows); missing macro files are skipped with a warning; unknown asset raises `ValueError`.
Used by (generated)	`src/data/sync_live_data.py`, `src/evaluation/backtesting.py`, `src/evaluation/plots.py`, `src/experiments/audit_improvements.py`, `src/experiments/run_experiments.py`, `src/models/ensemble_model.py`, `src/training/train_models.py`, `src/training/tune_models.py`; tests: `test_features.py`, `test_sequences_and_split.py` Run by `make preprocess`.
If removed or broken	No features, no scaler, no splits, no dataset for any model: the whole project stops.

src/data/preprocessing.py (split + scaler)

```
    def normalize_and_split(self):
        """
        Chronological split by the frozen dates in config (no shuffling).
        The scaler is fitted on the training window ONLY and applied to val/test.
        Only model feature columns (stationary) are scaled and saved.
        """
        cols = self.model_feature_columns()
        train_df = self.df.loc[:TRAIN_END, cols]
        val_df = self.df.loc[pd.Timestamp(TRAIN_END) + pd.Timedelta(days=1):VAL_END, cols]
        test_df = self.df.loc[pd.Timestamp(VAL_END) + pd.Timedelta(days=1):, cols]

```

src/data/preprocessing.py (split + scaler) (continued)

```
263         print(f"\n{'=' * 50}\nChronological Split for {self.asset_name}:")
264         for name, d in (('Train', train_df), ('Val', val_df), ('Test', test_df)):
265             print(f"  {name:5s}: {len(d):4d} rows | {d.index.min().date()} → {d.index.max().date()}")
266         print('=' * 50)
267
268         self.scaler = MinMaxScaler(feature_range=(0, 1)).fit(train_df)
269         self.train_scaled = pd.DataFrame(self.scaler.transform(train_df), columns=cols,
                index=train_df.index)
270         self.val_scaled = pd.DataFrame(self.scaler.transform(val_df), columns=cols,
                index=val_df.index)
271         self.test_scaled = pd.DataFrame(self.scaler.transform(test_df), columns=cols,
                index=test_df.index)
272         print("MinMaxScaler fitted on training data only.")
```

`self.df.loc[:TRAIN_END, cols]` is a **label-based date slice**: everything up to and including 2025-09-10. The validation slice starts at TRAIN_END + 1 day, the test slice after VAL_END. `MinMaxScaler(...).fit(train_df)` sees **only** the training rows; `transform` is then applied to all three. The shapes printed here are the sizes quoted in Chapter 9.

src/data/sync_live_data.py

Purpose	Refresh the live data for the dashboard without ever touching the frozen files.
Imports	`os, sys`; project: `config`, `DataCleaner`, `fetch_asset`, `fetch_all_external_data`.
Functions and classes	`update_live_data(asset_name, refresh_external=True)` → bool (True on success).
Important variables	`RAW_LIVE_DIR`, `out` (live features path), `tmp`.
Key processing steps	Optionally download all macro series → for Silver also download Gold → download the asset → run `DataCleaner(asset, raw_dir=RAW_LIVE_DIR).clean_data()` → write `<prefix>_live_features.csv.tmp` → `os.replace` to the final name (atomic).
Errors handled	Any download exception → prints and returns False; an empty download → False; `ValueError` from feature validation → file **not** written, returns False.
Used by (generated)	`app/streamlit_app.py`; tests: `test_models_and_inference.py` Also `python src/data/sync_live_data.py`.
If removed or broken	The dashboard shows only frozen data (no live forecasts, no NEW days).

src/data/sync_live_data.py (validation and atomic write)

```
38
39      cleaner = DataCleaner(asset_name, raw_dir=RAW_LIVE_DIR)
40      if not cleaner.load_data():
41          return False
42      try:
43          cleaner.clean_data()
44      except ValueError as e:                                  # feature validation failed (e.g. a corrupt
            download)
45          print(f"Live data for {asset_name} failed validation and was not written: {e}")
46          return False
47      out = os.path.join(PROCESSED_DATA_DIR, f'{get_prefix(asset_name)}_live_features.csv')
48      tmp = out + '.tmp'
49      cleaner.df.to_csv(tmp)
50      os.replace(tmp, out)                                     # atomic: a reader never sees a half-written
            file
51      print(f"Live features for {asset_name} written to {out} (last complete bar
            {cleaner.df.index[-1].date()})")
52      return True
```

src/data/eda.py

Purpose	Exploratory analysis written to `results/eda_summary.csv` and `results/figures/eda_*.png`.
Imports	`numpy, pandas, matplotlib (Agg), statsmodels (adfuller, acf, plot_acf, plot_pacf), scipy.stats`; `config`.
Functions and classes	`adf(series)` → dict(stat, p, stationary_5pct). `main()` loops over the assets.
Important variables	`COLORS`, `rows` (summary records).
Key processing steps	Training window only (`df.loc[:TRAIN_END]`): ADF test on price and return, ACF lags 1, 2, 5, mean/std/skew/kurtosis, share of |return| > 2 %; plots ACF/PACF, return histograms with a normal fit, rolling 30-day volatility.
Errors handled	None specific.
Used by (generated)	(entry-point script) Run by `make eda`.
If removed or broken	The justification for "predict returns, not prices" (Chapter 4.7) would lose its saved evidence; nothing else breaks.

src/models/registry.py

Purpose	One uniform interface (`fit`, `predict`, `save`, `load`) for every model, plus the factory and artefact helpers. This is why training, evaluation and inference never need `if model == ...` code.
Imports	`os, json, numpy, joblib`; `config` (`MODELS_DIR`, `get_prefix`), `set_all_seeds`; lazily inside methods: scikit-learn, LightGBM, CatBoost, TensorFlow/Keras, statsmodels, and the two builder modules.
Functions and classes	`BaseModel` (`__init__(params)`, `fit(Xseq_tr, Xt_tr, y_tr, Xseq_val, Xt_val, y_val, final)`, `predict(Xseq, Xt)`, `feature_importance(columns)`, `save(asset, task)`, `load(asset, task)`). **Subclasses:** `NaiveModel`, `NaiveMean`, `ARIMAModel` (`fit_series`, `forecast_walk_forward`), `RidgeModel`, `RandomForestModel`, `LightGBMModel`, `CatBoostModel` (native save/load), `_KerasSequenceModel` (`_build`, `fit`, `predict`, `save`, `load`) with `GRUModel` and `LSTMModel`. **Functions:** `artefact_stem(asset, model, task)`, `make_model(name, params, **kw)`, `load_trained(name, asset, task)`, `artefact_exists(name, asset, task)`.
Important variables	`TABULAR = (Ridge, RandomForest, LightGBM, CatBoost)`, `RECURRENT = (GRU, LSTM)`, `BASELINES = (Naive, Naive-Mean, ARIMA)`, `ALL_MODELS`, `_REGISTRY` (name → class), `fit_info` (epochs / trees actually used), `history` (Keras loss curves).
Key processing steps	Tabular models use `Xt` (last row); recurrent models use `Xseq` (30 × F). `fit(..., final=False)` is phase A (fit on train, early-stop on validation); `final=True` is phase B (refit on train+val with the stopping point fixed). `predict` returns **standardised** values.
Errors handled	`make_model` raises `ValueError` for unknown names. Loading a missing file raises the library's own `FileNotFoundError` (callers check `artefact_exists` first).
Used by (generated)	`src/evaluation/backtesting.py`, `src/evaluation/cross_validation.py`, `src/experiments/audit_improvements.py`, `src/experiments/run_experiments.py`, `src/inference/prediction.py`, `src/models/ensemble_model.py`, `src/training/train_models.py`; tests: `test_models_and_inference.py`
If removed or broken	Nothing can be trained, saved, loaded or compared.

src/models/registry.py (LightGBMModel.fit: the two-phase pattern)

```
186     def fit(self, Xseq_tr, Xt_tr, y_tr, Xseq_val=None, Xt_val=None, y_val=None, final=False):
187         import lightgbm as lgb
188         params = dict(self.params)
189         n_est = params.pop('n_estimators', 300)
190         # NB: LightGBM only applies `subsample` (row bagging) when `subsample_freq` > 0; it is left
                at the default 0, so the
191         # 0.8 in the tuning grid is inert and every tree sees all rows (the reported results were
                produced this way).
192         base = dict(random_state=42, n_jobs=-1, verbose=-1, **params)
193         if Xt_val is not None and y_val is not None:
194             # phase 1: early stopping on val to find the number of trees
195             m1 = lgb.LGBMRegressor(n_estimators=n_est, **base)
196             m1.fit(Xt_tr, np.ravel(y_tr), eval_set=[(Xt_val, np.ravel(y_val))],
197                    callbacks=[lgb.early_stopping(30, verbose=False)])
198             best_n = max(10, int(m1.best_iteration_ or n_est))
199             self.fit_info = {'n_estimators_used': best_n}
200             if not final:
201                 self.model = m1
202                 return self
203             X = np.concatenate([Xt_tr, Xt_val]); y = np.concatenate([y_tr, y_val])
204         else:
205             best_n, X, y = n_est, Xt_tr, y_tr
206             self.fit_info = {'n_estimators_used': best_n}
207         # phase 2: refit on train+val with fixed tree count
208         self.model = lgb.LGBMRegressor(n_estimators=best_n, **base).fit(X, np.ravel(y))
209         return self
```

- **Lines with** `Xt_val is not None`**:** phase 1 fits with `early_stopping(30)` on validation and stores the best tree count in `fit_info`.
- `if not final: return` ends phase A with the validation-monitored model. For phase B the code concatenates train and validation and refits with the **fixed** `n_estimators=best_n`.
- `max(10, ...)` is the floor that makes Gold's and Silver's LightGBM use 10 trees. The comment about `subsample` documents the inert parameter.

src/models/model_gru.py

Purpose	Builds the GRU network.
Imports	`tensorflow`, `Sequential`, `GRU`, `Dense`, `Dropout`, `Adam`.
Functions and classes	`build_gru_model(seq_len, n_features, units=32, dropout=0.2, learning_rate=0.001)` → compiled Keras model: Input → GRU(units) → Dropout → Dense(16, relu) → Dense(1); MSE loss, Adam.
Important variables	`units`, `dropout`, `learning_rate` come from tuned params.
Key processing steps	Single recurrent layer by design: a 2 × 100-unit stack over-fitted about 500 windows in the first version.
Errors handled	—
Used by (generated)	`src/models/registry.py` Imported lazily by `GRUModel.builder`.
If removed or broken	GRU models cannot be built or loaded.

src/models/model_lstm.py

Purpose	Same architecture as the GRU with an `LSTM` layer, so the comparison is fair.
Imports	`tensorflow`, `Sequential`, `LSTM`, `Dense`, `Dropout`, `Adam`.
Functions and classes	`build_lstm_model(...)` with the same signature.
Important variables	—
Key processing steps	See the GRU.
Errors handled	—
Used by (generated)	`src/models/registry.py`
If removed or broken	LSTM models cannot be built or loaded.

src/models/ensemble_model.py

Purpose	Stacked-ensemble experiment: learns non-negative weights for four base models from their out-of-fold predictions. **Not served.**
Imports	`os, sys, json, argparse, numpy, joblib`, `sklearn.linear_model.Ridge`; `config`, `build_dataset`, `cv_evaluate`, `load_trained`, logging helpers. Sets `TF_CPP_MIN_LOG_LEVEL=3`.
Functions and classes	`stack_path(asset)`; `fit_stack(asset, bases, data=None)` → (meta-model, weights dict); `StackedModel(asset)` with `predict(Xseq, Xt)`.
Important variables	`DEFAULT_BASES = [Ridge, LightGBM, CatBoost, GRU]`, `X_meta` (columns of out-of-fold predictions), `meta = Ridge(alpha=1.0, positive=True)`, `weights`.
Key processing steps	For each base model run walk-forward CV with `return_oof=True` → stack the out-of-fold predictions as columns → fit the meta-model on the matching true targets → save `{meta, bases, weights}` and a JSON.
Errors handled	None specific.
Used by (generated)	`src/evaluation/backtesting.py`, `src/inference/prediction.py` Run by `make stack`.
If removed or broken	Only the experiment row "Stacked" disappears from the tables (evaluation checks `os.path.exists(stack_path)`).

src/models/ensemble_model.py (fit_stack)

```
43 def fit_stack(asset, bases, data=None):
44     data = data or build_dataset(asset)
45     oof_cols, idx_ref = [], None
46     for b in bases:
47         _, oof, idx = cv_evaluate(b, get_params(b, asset, data['task']), data, n_splits=CV_FOLDS,
               return_oof=True)
48         oof_cols.append(oof)
49         idx_ref = idx
50     y = np.concatenate([data['y_train'], data['y_val']])[idx_ref].ravel()
51     X_meta = np.column_stack(oof_cols)
52     meta = Ridge(alpha=1.0, positive=True).fit(X_meta, y)
53     weights = dict(zip(bases, [float(w) for w in meta.coef_]))
54     log.info(f"{asset}: stack weights {weights}  intercept={float(meta.intercept_):.4f}")
55     joblib.dump({'meta': meta, 'bases': bases, 'weights': weights}, stack_path(asset))
```

src/models/ensemble_model.py (fit_stack) (continued)

```
    os.makedirs(os.path.join(RESULTS_DIR, 'stacking'), exist_ok=True)
    with open(os.path.join(RESULTS_DIR, 'stacking', f'{asset.lower()}_stack_weights.json'), 'w') as f:
        json.dump({'bases': bases, 'weights': weights, 'intercept': float(meta.intercept_)}, f,
            indent=2)
    return meta, weights
```

src/training/tune_models.py

Purpose	Hyper-parameter search by walk-forward cross-validation; writes the winners that `config.get_params` later reads.
Imports	`os, sys, json, time, argparse, itertools, pandas`; `config`, `build_dataset`, `cv_evaluate`, `summarise_folds`, `primary_metric`, logging.
Functions and classes	`grid(model_name)` (generator of parameter dicts via `itertools.product`); `tune(asset, model_name, data)` → (best dict, best row); `main()`.
Important variables	`GRIDS` (per model), `rows`, `best_params` (loaded and extended), `summary`.
Key processing steps	For each configuration: `cv_evaluate` over 4 folds → mean/sd of metrics → sort by mean RMSE → best row → write `results/tuning/<asset>_<model>.csv`; after **each** (asset, model) rewrite `best_params.json` and `tuning_summary.csv` (a partial run loses nothing).
Errors handled	None specific; a configuration that raises would stop the run.
Used by (generated)	(entry-point script) Run by `make tune` (about 30 minutes).
If removed or broken	Models would train with `DEFAULT_PARAMS` instead of the tuned ones; `cv_table` in backtesting would lose its model rows.

src/training/tune_models.py (tune)

```
def tune(asset, model_name, data):
    rows, t0 = [], time.time()
    pm = primary_metric(data)
    for i, params in enumerate(grid(model_name)):
        folds = cv_evaluate(model_name, params, data, n_splits=CV_FOLDS)
        s = summarise_folds(folds)
        rows.append({'config_id': i, **{f'p_{k}': v for k, v in params.items()}, **s})
        log.info(f"{asset:8s} {model_name:12s} cfg {i:2d} {params}  {pm}={s[pm + '_mean']:.5f}±{s[pm
            + '_std']:.5f}  DA={s['DirAcc_pct_mean']:.1f}%")
    df = pd.DataFrame(rows).sort_values(pm + '_mean')
    suffix = '' if data['task'] == 'return_1d' else f"_{data['task']}"
    df.to_csv(os.path.join(TUNING_DIR, f'{asset.lower()}_{model_name.lower()}{suffix}.csv'),
        index=False)
    best_id = int(df.iloc[0]['config_id'])
    best = list(grid(model_name))[best_id]
    log.info(f"{asset} {model_name}: best {best} in {time.time() - t0:.0f}s")
    return best, df.iloc[0].to_dict()
```

src/training/train_models.py

Purpose	Train the final models for every asset in two phases and save them.
Imports	`os, sys, json, time, argparse, pandas`; `config`, `build_dataset`, `make_model`, `TABULAR`, `RECURRENT`, `evaluate_task`, logging.
Functions and classes	`train_asset(asset, model_names, task)` → (metric rows, importance rows); `main()`.
Important variables	`TRAINABLE` (the six models), `m` (phase-A model), `m_final` (deployed model), `rows`, `imps`.
Key processing steps	Per model: `params = get_params(...)` → **phase A** `fit(final=False)` and metrics on train and validation → save loss history → **phase B** `fit(final=True)` → `save(asset, task)` → feature importance. Merges new rows into `train_val_metrics.csv` and `feature_importance.csv` so partial runs keep older rows.
Errors handled	None specific. The test split is never read.
Used by (generated)	(entry-point script) Run by `make train`.
If removed or broken	No deployed models: dashboard and API cannot predict.

src/training/train_models.py (train_asset: phases A and B)

```
49          # Phase A — honest validation fit
50          m = make_model(name, params)
51          m.fit(data['X_train'], data['Xt_train'], data['y_train'],
52                data['X_val'], data['Xt_val'], data['y_val'], final=False)
53          for split in ('train', 'val'):
54              pred = m.predict(data[f'X_{split}'], data[f'Xt_{split}'])
55              met = evaluate_task(data, data[f'y_real_{split}'], data['inv'](pred),
                    data[f'naive_{split}'], data[f'prev_{split}'])
        ...
63          # Phase B — deployed model on train+val
64          m_final = make_model(name, params)
65          m_final.fit(data['X_train'], data['Xt_train'], data['y_train'],
66                      data['X_val'], data['Xt_val'], data['y_val'], final=True)
67          m_final.history = m.history
68          path = m_final.save(asset, task)
69          fi = m_final.feature_importance(cols)
70          if fi:
71              imps += [{'asset': asset, 'model': name, 'task': task, 'feature': k, 'importance':
                    float(v)} for k, v in fi.items()]
```

src/evaluation/cross_validation.py

Purpose	Walk-forward validation: the only validation machinery. It never receives test data.
Imports	`numpy`, `sklearn.model_selection.TimeSeriesSplit`; `config.CV_FOLDS`, `make_model`, `evaluate_task`.
Functions and classes	`walk_forward_splits(n, n_splits, min_train)` (generator of index pairs); `trainval_arrays(data)` → concatenated train+val arrays; `cv_evaluate(model_name, params, data, n_splits, inner_val_frac=0.15, return_oof=False, feature_idx=None)` → list of per-fold metric dicts (and out-of-fold arrays); `primary_metric(data)`; `summarise_folds(folds)`.
Important variables	`tr, va` (fold indices), `tr_fit, tr_mon` (inner fit / early-stopping monitor), `oof_pred`, `oof_idx`.
Key processing steps	Per fold: purge overlapping targets (h > 1) → split off the last 15 % of the training window as the monitor → fit (ARIMA, Naive, Naive-Mean, EWMA are special-cased) → predict the validation block → `evaluate_task` → collect.
Errors handled	None specific.
Used by (generated)	`src/evaluation/backtesting.py`, `src/experiments/audit_improvements.py`, `src/experiments/run_experiments.py`, `src/models/ensemble_model.py`, `src/training/tune_models.py` Also used by the experiments.
If removed or broken	No tuning, no walk-forward table, no pooled test, no band calibration, no stack.

src/evaluation/backtesting.py

Purpose	**The only script that reads the test set.** Builds the model-selection table, evaluates every model once on the test days, calibrates the band, pools out-of-fold predictions, writes all results.
Imports	`os, sys, json, numpy, pandas`; `config`, `build_dataset`, registry helpers, `StackedModel`, `cv_evaluate`, `summarise_folds`, metrics, `BAND_*` constants from `prediction.py`.
Functions and classes	`model_exists`, `cv_table(asset, data)`, `combined_cv_row(asset, data)`, `pooled_walkforward(asset, data, member_oof)`, `band_rows(asset, data, pred_df, oof)` → (rows, k), `test_predictions(asset, data)` → (preds, infos), `regime_analysis`, `main()`, `write_markdown(...)`.
Important variables	`COMBINED = "Combined"`, `ML_MODELS`, `BASELINE_MODELS`, `PRED_DIR`, `status[asset]` (the dictionary written to `model_status.json`), `k` (band multiplier).
Key processing steps	`cv_table` (baselines + tuned models + Combined) → `pooled_walkforward` → `test_predictions` (load models, predict test once, ARIMA walk-forward, stack, **Combined = mean**) → per-model metrics and prediction CSV → `band_rows` → model selection on **validation** → regimes → write CSVs, `model_status.json`, `FINAL_RESULTS.md`.
Errors handled	A missing model file logs "no trained X found, skipping"; band calibration raises if there are no out-of-fold predictions.
Used by (generated)	(entry-point script) Run by `make evaluate`.
If removed or broken	No results, no `model_status.json`: the dashboard and the API refuse to predict (no target mean/std, no band k).

src/evaluation/backtesting.py (the Combined forecast in the evaluation)

```
    # Combined = equal-weight mean of the base models' predictions (no fitted weights, so nothing is
        selected
    # on the test set). This is the forecast the dashboard and the API serve.
    members = [m for m in ML_MODELS if m in preds]
    if members:
        preds[COMBINED] = np.mean([preds[m] for m in members], axis=0)
        infos[COMBINED] = {'members': members, 'rule': 'equal-weight mean of predicted log returns'}
    return preds, infos
```

src/evaluation/plots.py

Purpose	Report figures from the result CSVs.
Imports	`json, numpy, pandas, matplotlib (Agg)`; `config`.
Functions and classes	`save`, `shade_splits`, `price_history`, `comparison(kind)`, `per_asset(asset, status)`, `overfitting_gap`.
Important variables	`COLORS`, `MODEL_COLORS`, `PRED_DIR`.
Key processing steps	Reads `cv_results.csv`, `final_test_results.csv`, prediction CSVs, histories, importances → PNGs in `results/figures/`.
Errors handled	Skips figures whose input CSV is missing.
Used by (generated)	(entry-point script) Run by `make figures`.
If removed or broken	No PNGs for the report or the dashboard's diagnostics expander.

src/inference/prediction.py

Purpose	Everything the dashboard and the API need to produce a forecast: load live or frozen features, scale, run the models, combine, build the band, reveal actuals for demo days, score recent days live.
Imports	`os, sys, time, datetime, numpy, pandas, joblib`; `config` (paths, `SEQ_LEN`, `LEVEL_COLUMNS`, `VAL_END`, `load_model_status`), `load_trained`, `artefact_exists`, `TABULAR`, `RECURRENT`, `StackedModel`, `stack_path`, `expected_last_complete_bar`, `next_trading_day`, logger.
Functions and classes	`_features_path`, `load_features(asset, live=True)`, `live_data_is_current`, `_window(asset, feats, as_of, seq_len)`, `_band_k`, `_band(...)`, `_target_stats`, `available_models`, `combined_members`, `_load`, `_predict_one`, `_predict_from_window(..., progress)`, `_individual`, `_context`, `predict_next_day(asset, model_name)`, `predict_for_date(asset, as_of, model_name, progress)`, `recent_track_record(asset, n_days, progress)`, `prediction_history`, `predict_next_day_safe`.
Important variables	`COMBINED`, `BASE_MODELS` (six), `BAND_SIGMA_FEATURE = "ewma_vol"`, `BAND_NOMINAL = 0.6827`, `DISCLAIMER`; locals `Xseq`, `Xt`, `last_close`, `as_of`, `r_hat`, `members`.
Key processing steps	See Chapter 22 for the full trace. In short: features → scaler → last 30 rows → each model → z·σ+μ → mean → price, direction, band → add held-out metrics and the disclaimer.
Errors handled	`FileNotFoundError` (no processed data; no trained models; no target stats; no band k); `ValueError` (feature mismatch by count or name; not enough history; no bar on the requested date; non-finite model output; invalid σ).
Used by (generated)	`app/streamlit_app.py`, `src/api/app.py`, `src/evaluation/backtesting.py`; tests: `test_models_and_inference.py` Also `make predict` (prints all three assets).
If removed or broken	No forecasts at all: dashboard Forecast, Predict a Day and the API all call this module.

src/inference/prediction.py (_window: the consistency guard)

```
86  def _window(asset, feats, as_of=None, seq_len=SEQ_LEN):
87      """Scaled window ending on `as_of` (default: last available day). Only rows <= as_of are used."""
88      prefix = get_prefix(asset)
89      scaler = joblib.load(os.path.join(MODELS_DIR, f'{prefix}_scaler.pkl'))
90      cols = [c for c in feats.columns if c not in LEVEL_COLUMNS]
91      if len(cols) != scaler.n_features_in_:
92          raise ValueError(f"Feature mismatch for {asset}: scaler expects {scaler.n_features_in_}, got
                {len(cols)}")
93      expected = list(getattr(scaler, 'feature_names_in_', cols))
94      if set(cols) != set(expected):
95          raise ValueError(f"Feature mismatch for {asset}: {sorted(set(cols) ^ set(expected))}")
96      cols = expected                                          # the scaler's (= the models') column
            order, whatever the file's order
97      hist = feats if as_of is None else feats.loc[:pd.Timestamp(as_of)]
98      if len(hist) < seq_len:
99          raise ValueError(f"Not enough history before {as_of} ({len(hist)} rows, need {seq_len})")
100     scaled = scaler.transform(hist[cols].values)
101     window = scaled[-seq_len:]
102     return window[np.newaxis, :, :], window[-1:, :], float(hist['price'].iloc[-1]), hist.index[-1],
            cols
```

- `scaler.n_features_in_` and `feature_names_in_` come from the training-time scaler: the **count** is checked, then the **names** (set difference), then the columns are **re-ordered** to the scaler's order.
- `feats.loc[:pd.Timestamp(as_of)]` is the look-ahead guard: rows after the chosen day do not exist for the model.
- The function returns the (1, 30, F) window, the (1, F) last row, the close of day t, its date and the column list.

src/utils/metrics.py

Purpose	Every metric, in one place: `rmse`, `mae`, `mape`, `r2`, `directional_accuracy`, `diebold_mariano`, `strategy_backtest`, `evaluate_forecast`, volatility metrics (`qlike`, `evaluate_vol_forecast`), `band_calibration`, and the dispatcher `evaluate_task`.
Imports	`numpy`, `scipy.stats`.
Functions and classes	`rmse(y, ŷ)`, `mae`, `mape` (ignores zero targets), `r2`, `directional_accuracy(true, pred)` → (%, n, one-sided binomial p), `diebold_mariano(true, a, b, h=1)` → (stat, p), `strategy_backtest(true, pred, cost_bps=10, periods_per_year)` → dict, `evaluate_forecast(true_ret, pred_ret, prev_close, naive_pred_ret, periods_per_year)` → dict of ≈ 24 metrics, `qlike`, `evaluate_vol_forecast`, `band_calibration(true, pred, sigma)` → coverage, QLIKE, mean half-width, `evaluate_task(data, ...)`.
Important variables	`m = true_ret != 0` (non-flat days), `hits`, `d` (loss differential in DM).
Key processing steps	DM: $d_t = e_a^2 - e_b^2$; HAC variance with h−1 lags (h = 1 here); Harvey-Leybourne-Newbold correction; two-sided Student-t p. Strategy: position = 1 if ŷ > 0; cost 10 bps on position changes; Sharpe annualised with 252 or 365.
Errors handled	DM returns NaN for n < 10 or identical forecasts; directional accuracy returns NaN when no non-flat day exists; MAPE returns NaN if every target is zero.
Used by (generated)	`src/evaluation/backtesting.py`, `src/evaluation/cross_validation.py`, `src/experiments/audit_improvements.py`, `src/experiments/before_after.py`, `src/experiments/run_experiments.py`, `src/training/train_models.py`; tests: `test_metrics.py`
If removed or broken	No evaluation numbers anywhere (training logs, tuning, backtesting, experiments).

src/utils/metrics.py (directional_accuracy)

```
39 def directional_accuracy(true_ret, pred_ret):
40     """Returns (accuracy %, n_evaluated_days, one-sided binomial p-value vs 50%)."""
41     true_ret, pred_ret = np.ravel(true_ret), np.ravel(pred_ret)
42     m = true_ret != 0
43     n = int(m.sum())
44     if n == 0:
45         return np.nan, 0, np.nan
46     hits = int(np.sum(np.sign(true_ret[m]) == np.sign(pred_ret[m])))
47     p = stats.binomtest(hits, n, 0.5, alternative='greater').pvalue
48     return 100.0 * hits / n, n, float(p)
```

`m = true_ret != 0` removes days with exactly zero return (their direction is undefined). `np.sign` compares directions; `stats.binomtest(hits, n, 0.5, alternative="greater")` asks "how likely are at least this many hits if the model were guessing?".

src/utils/inverse_transform.py

Purpose	Price reconstruction `P·exp(r)` with the actual previous close given explicitly (an earlier version guessed a column and used `open` by mistake).
Imports	`numpy`.
Functions and classes	`reconstruct_price(log_return, prev_close)` → array of prices.
Important variables	—
Key processing steps	One line: `ravel(prev_close) * exp(ravel(log_return))`.
Errors handled	—
Used by (generated)	(entry-point script); tests: `test_sequences_and_split.py`
If removed or broken	Only tests use it now (`test_true_target_reconstructs_actual_close_exactly`); the regression guard for the old bug would be lost.

src/utils/reproducibility.py

Purpose	Seed everything.
Imports	`random, os, numpy`; TensorFlow imported lazily.
Functions and classes	`set_all_seeds(seed=42)`: Python `random`, NumPy, `PYTHONHASHSEED`, `tf.random.set_seed`.
Important variables	—
Key processing steps	—
Errors handled	Import errors of TensorFlow are swallowed (so the function works without it).
Used by (generated)	`src/models/registry.py`
If removed or broken	Keras results would no longer be reproducible.

src/utils/logging_config.py

Purpose	Uniform log format.
Imports	`logging, sys, os`.
Functions and classes	`setup_logging(level="INFO", log_file=None)`, `get_logger(name)`.
Important variables	—
Key processing steps	Configures a stdout (and optional file) handler with timestamp | module | level | message.
Errors handled	`ValueError` for an invalid level.
Used by (generated)	`src/evaluation/backtesting.py`, `src/experiments/audit_improvements.py`, `src/experiments/run_experiments.py`, `src/inference/prediction.py`, `src/models/ensemble_model.py`, `src/training/train_models.py`, `src/training/tune_models.py`
If removed or broken	Scripts lose their formatted logging (they would fail on the import).

src/api/app.py

Purpose	A FastAPI service giving the same forecast as the dashboard in JSON.
Imports	`fastapi (FastAPI, HTTPException, Query, CORSMiddleware)`, `pydantic.BaseModel`, `typing`; `config` (`ASSET_CONFIG`, `load_model_status`), `predict_next_day`, `available_models`, `combined_members`, `COMBINED`.
Functions and classes	Models: `MemberPrediction`, `PredictionResponse`. Endpoints: `GET /health` → `{"status": "healthy"}`; `GET /models` → per asset the status dict plus members and available models; `GET /predict/{asset}?model=` → `PredictionResponse`. Helper `_resolve(asset)`.
Important variables	`app` (FastAPI instance, title "AI-Powered Commodity Price Predictor API", version 2.1.0).
Key processing steps	`_resolve` maps `bitcoin|gold|silver` (case-insensitive) to the asset name → check the optional model name → `predict_next_day` → validate with `PredictionResponse`.
Errors handled	404 for an unknown asset or model; 503 on `FileNotFoundError` ("Model or data artefacts missing"); 500 with logging for any other exception.
Used by (generated)	(entry-point script) Started by `make api` (`uvicorn src.api.app:app`).
If removed or broken	The JSON interface disappears; the dashboard is unaffected.

src/api/app.py (the prediction endpoint)

```
82 @app.get("/predict/{asset}", response_model=PredictionResponse)
83 def predict_asset(asset: str, model: Optional[str] = Query(None, description="One trained model name;
       default = the Combined forecast")):
84     name = _resolve(asset)
85     if model and model not in available_models(name) + [COMBINED]:
86         raise HTTPException(status_code=404, detail=f"Model '{model}' not available for {name}.
               Available: {available_models(name) + [COMBINED]}")
87     try:
88         return PredictionResponse(**predict_next_day(name, model or COMBINED))
89     except FileNotFoundError as e:
```

src/api/app.py (the prediction endpoint) (continued)

```
        raise HTTPException(status_code=503, detail=f"Model or data artefacts missing: {e}")
    except Exception as e:
        logger.exception(f"Error predicting {name}")
        raise HTTPException(status_code=500, detail=str(e))
```

app/streamlit_app.py

Purpose	The dashboard. A script, not a module: Streamlit executes it top to bottom on every interaction.
Imports	`os, sys, time, logging, numpy, pandas, streamlit as st, plotly.graph_objects, plotly.subplots`; `config` (paths, split dates, `ASSETS`, `ASSET_CONFIG`, `get_prefix`, `load_model_status`), `predict_next_day`, `predict_for_date`, `prediction_history`, `combined_members`, `live_data_is_current`, `recent_track_record`, `COMBINED`, `DISCLAIMER`, `expected_last_complete_bar`, `update_live_data`.
Functions and classes	Cached loaders `load_features`, `load_csv`, `load_test_predictions` (`@st.cache_data(ttl=300)`); helpers `test_row`, `layout`, `pct`, `usd`, `usd_md`. No classes. The four tabs are `with tab_...:` blocks.
Important variables	`asset`, `status`, `members`, `df` (features), `te_all`, `cv_all`, `hist`, `comb_test`, `naive_test`; `st.session_state` keys `run_prediction`, `run_result`, `run_score`, `demo_result`, `open_tab`, `live_check_<asset>`.
Key processing steps	Page config and CSS → sidebar (asset, Run prediction, overlays, Sync) → automatic live check → load data and results → overview strip → 4 tabs → footer disclaimer. Chapter 12 describes every block.
Errors handled	The prediction blocks are wrapped in `try / except` and show `st.error("Prediction failed: ...")`; missing result files produce `st.info` / `st.warning` texts that name the `make` target to run.
Used by (generated)	(entry-point script) Started by `make serve`.
If removed or broken	No user interface (the API still works).

app/streamlit_app.py (the Run-prediction block)

```
        if not st.session_state['run_prediction'] or not members:
            st.caption("Press **Run prediction** in the sidebar. Every trained model runs on the
                latest complete bar and their "
                       "combination is shown here.")
        else:
            try:
                if st.session_state.get('run_result', (None,))[0] != asset:
                    with st.spinner("Checking for the latest complete bar…"):
                        refreshed = False
                        live_path = os.path.join(PROCESSED_DATA_DIR,
                            f'{get_prefix(asset)}_live_features.csv')
                        fresh = os.path.exists(live_path) and (time.time() -
                            os.path.getmtime(live_path)) < 30 * 60
                        if not (live_data_is_current(asset) or fresh):        # download at most once
                            per 30 min while the vendor lags
                            refreshed = update_live_data(asset, refresh_external=True)
                            if refreshed:
                                st.cache_data.clear()
                    with st.spinner(f"Running {len(members)} models…"):
                        r = predict_next_day(asset, COMBINED)
                    st.session_state['run_result'] = (asset, r, refreshed)
                    if refreshed:
```

- `st.session_state['run_result']` stores (asset, result, refreshed); the model run happens only when the stored asset differs, so changing the slider or the tab does not recompute the six models.

- `update_live_data` is called only if the live file is not current and not fresher than 30 minutes; if it refreshed, `st.cache_data.clear()` and `st.rerun()` reload everything.
- `predict_next_day(asset, COMBINED)` is the single call into the backend for the final card.

src/experiments/run_experiments.py

Purpose	Experiments E1 to E4 with fixed, strongly regularised parameters so that comparisons are like with like (validation only).
Imports	`numpy, pandas, argparse`; `build_dataset`, `make_model`, `cv_evaluate`, `summarise_folds`, `evaluate_task`.
Functions and classes	`fixed_val_eval`, `e1_data_size`, `e2_feature_ablation`, `e3_horizon`, `e4_volatility`, `write_summary`.
Important variables	`EXP_PARAMS`, `FEATURE_GROUPS` (5 groups), `TAB_MODELS`.
Key processing steps	E1 compares 3-year and full history on a fixed validation window; E2 drops one feature group at a time via `feature_idx`; E3 compares `return_1d` with `return_5d`; E4 targets 22-day volatility vs persistence and EWMA.
Errors handled	—
Used by (generated)	(entry-point script) Run by `make experiments`.
If removed or broken	Only the documentation of design choices (the dashboard's experiments expander) is lost.

src/experiments/audit_improvements.py

Purpose	Experiment E5: three candidate changes to the served forecast, judged on out-of-fold days only.
Imports	`numpy, pandas`; `build_dataset`, `cv_evaluate`, `trainval_arrays`, metrics.
Functions and classes	`run_asset(asset)`, `main()`.
Important variables	`REDUNDANT` (10 features dropped in E5b).
Key processing steps	Builds candidates (E5a tabular-only Combined, E5b compact features, E5c zero-drift) from the same out-of-fold predictions; compares RMSE vs served and DM p; writes `E5_audit_candidates.csv`, `E5_summary.md`.
Errors handled	—
Used by (generated)	(entry-point script)
If removed or broken	Documentation of why the rejected ideas were rejected.

src/experiments/before_after.py

Purpose	Compares the old 3-year system with the final one on identical unseen days.
Imports	`json, numpy, pandas`; `config`, `evaluate_forecast`.
Functions and classes	`metrics_from`, `main()`.
Important variables	`OLD = results/archive_3y_final`.
Key processing steps	Restricts both prediction files to the common target dates, scores each system and its own naive forecast, writes `before_after.csv`.
Errors handled	—
Used by (generated)	(entry-point script)
If removed or broken	Only the before/after table is lost.

scripts/retrain.py

Purpose	End-to-end orchestration: runs each stage as a separate Python process and stops on the first failure.
Imports	`os, sys, logging, argparse, subprocess`.
Functions and classes	`run(cmd, description)`, `main()`; flags `--no-fetch`, `--no-tune`.
Important variables	`ROOT`, `env` (sets `PYTHONPATH` and `TF_CPP_MIN_LOG_LEVEL`).
Key processing steps	collection → external data → preprocessing → EDA → (tuning) → training → stacking → evaluation → figures → experiments; logs to `logs/retrain.log` (the folder is created at run time; none exists now).
Errors handled	A failing stage logs the last 3,000 characters of stderr and exits with status 1.
Used by (generated)	(entry-point script)
If removed or broken	Manual `make` targets still work.

tests/ (6 files, 47 test cases)

File	What it protects	Key tests
`conftest.py`	Fixtures: `synthetic_price_data` (300 random-walk days), `synthetic_sequences`, `project_root`; adds the root to `sys.path`	—
`test_features.py`	Feature correctness and **no look-ahead**	`test_no_lookahead_in_any_feature` (Bitcoin, Gold), `test_commodity_external_features_are_previous_day_values`, `test_commodity_high_low_features_are_lagged_one_session`, `test_commodities_keep_trading_calendar`, `test_rsi_bounds_and_bollinger_ordering`
`test_market_calendar.py`	Complete-bar rule	`test_complete_bar_cutoff_...`, `test_drop_incomplete_bars_removes_todays_running_row`, `test_next_trading_day_skips_weekend_for_futures_only`
`test_metrics.py`	Metric definitions	directional accuracy excludes flat days, DM prefers the better forecast, band coverage and QLIKE
`test_models_and_inference.py`	Registry contract and end-to-end inference on the saved models	served prediction is the mean of six members; `predict_for_date` ignores future rows; corrupt live data raises; missing calibration is reported; column-order independence; live scorecard consistency
`test_sequences_and_split.py`	Target alignment and split integrity on the real processed data	target is the next row; chronological, disjoint splits; scaler fitted on train only; exact price reconstruction

Result of `venv/bin/python -m pytest tests/ -q` on 2026-10-09: **47 passed** (about 70 seconds, 74 warnings that are cosmetic scikit-learn / pandas notices). Tests that need trained models are skipped when `data/models/model_status.json` is absent (e.g. in CI on a fresh clone).

notebooks/ and the empty `__init__.py` files

- **Notebooks** (`01_data_exploration`, `02_feature_engineering`, `03_model_training`, `04_evaluation`) import the project's functions and display saved CSVs and PNGs; they train nothing. Notebook 01 still says "Three years of daily OHLCV" (stale, Appendix C).
- `__init__.py` files (9: root `src`, and the eight sub-packages) only mark directories as Python packages. They contain at most one line.

Chapter 18: Function-by-Function Reference

115 functions and methods that matter for the workflow, with **exact names** (every row was checked against the parsed source: none is invented). "Called by" was generated by static analysis of the calls in the repository; `<module>` means module-level code (a script body). Methods named `fit`, `predict`, `save`, `load` are reached through `make_model` / `load_trained` and are listed by class.

Configuration and data acquisition

Function or method	File	Purpose	Inputs	Outputs	Called by
`get_prefix`	`config`	Short file prefix of an asset	asset name	`"btc"/"gold"/"silver"`	`backtesting.main`, `before_after.main`, `eda.main`, `ensemble_model.stack_path`, `plots.<module>` +13
`get_params`	`config`	Default hyper-parameters overlaid with tuned ones	model, asset, task	dict	`audit_improvements.run_asset`, `backtesting.combined_cv_row`, `backtesting.main`, `ensemble_model.fit_stack`, `train_models.train_asset`
`load_model_status`	`config`	Read `model_status.json` (or an untrained placeholder)	(none)	dict per asset	`app.list_models`, `before_after.main`, `config.<module>`, `plots.<module>`, `prediction._target_stats` +4
`fetch_asset`	`data_collection`	Download one asset's daily OHLCV, drop the running bar, write CSV	asset, start date, out dir	DataFrame (empty on failure)	`data_collection.<module>`, `sync_live_data.update_live_data`
`verify_data`	`data_collection`	Print range, price span, nulls, shape	DataFrame, name	(prints)	`data_collection.<module>`
`fetch_yfinance_external`	`external_data`	Download one macro series	source key, out dir	DataFrame `timestamp, price`	`external_data.fetch_all_external_data`
`fetch_fear_greed_index`	`external_data`	Download the Fear & Greed history	out dir	DataFrame `timestamp, fear_greed`	`external_data.fetch_all_external_data`
`fetch_all_external_data`	`external_data`	All macro series + Fear & Greed	out dir	dict of DataFrames	`external_data.<module>`, `sync_live_data.update_live_data`
`complete_bar_cutoff`	`market_calendar`	First date whose bar is not yet complete	asset type, now	date	`market_calendar.drop_incomplete_bars`, `market_calendar.expected_last_complete_bar`, `tests`
`drop_incomplete_bars`	`market_calendar`	Remove rows of unfinished bars	DataFrame, asset type	DataFrame	`data_collection.fetch_asset`, `external_data.fetch_yfinance_external`, `tests`
`next_trading_day`	`market_calendar`	Target date label for a forecast	asset type, date	date	`prediction.predict_next_day`, `tests`
`expected_last_complete_bar`	`market_calendar`	Newest bar that should exist now	asset type, now	date	`prediction.live_data_is_current`, `streamlit_app.<module>`
`update_live_data`	`sync_live_data`	Refresh live data and rebuild live features	asset, refresh_external	bool	`streamlit_app.<module>`, `sync_live_data.<module>`, `tests`

Preprocessing and feature engineering

Function or method	File	Purpose	Inputs	Outputs	Called by
DataCleaner.load_data	preprocessing	Read the raw CSV	(self.raw_path)	bool; self.df	preprocessing.run_cleaning_pipeline, sync_live_data.update_live_data
DataCleaner.clean_data	preprocessing	Whole cleaning + feature pipeline	self.df	cleaned feature table	preprocessing.run_cleaning_pipeline, sync_live_data.update_live_data, tests
DataCleaner.add_moving_averages	preprocessing	EMA 14 and 30	price	EMA_14, EMA_30	preprocessing.clean_data
DataCleaner.add_rsi	preprocessing	Wilder RSI	price, window=14	RSI	preprocessing.clean_data
DataCleaner.add_macd	preprocessing	MACD and signal	price, 12/26/9	MACD, MACD_Signal	preprocessing.clean_data
DataCleaner.add_advanced_ta	preprocessing	Bollinger, ATR, ADX, ROC via ta	high, low, price	BB_*, ATR, ADX, ROC	preprocessing.clean_data
DataCleaner.add_relative_features	preprocessing	Scale-free ratios	indicator levels, price	ema*_ratio, macd_*, bb_pctb, bb_width, atr_norm, hl_range	preprocessing.clean_data
DataCleaner.add_lag_returns	preprocessing	Lagged returns and 20-day momentum	log_return	return_1d/2d/5d/10d/20d	preprocessing.clean_data
DataCleaner.add_rolling_volatility	preprocessing	Rolling, HAR and EWMA volatility	log_return	volatility_*, rv_*, ewma_vol	preprocessing.clean_data
DataCleaner.add_calendar_features	preprocessing	Weekday sine/cosine (Bitcoin)	date index	dow_sin, dow_cos	preprocessing.clean_data
DataCleaner.add_volume_change	preprocessing	Clipped log volume change (Bitcoin)	volume	log_volume_change	preprocessing.clean_data
DataCleaner.lag_post_settlement_features	preprocessing	Lag High/Low features one session (metals)	hl_range, atr_norm, ADX	shifted columns	preprocessing.clean_data
DataCleaner._align_external	preprocessing	Close-time alignment of an external series	series	series on the asset's calendar	preprocessing.add_external_features
DataCleaner.add_external_features	preprocessing	Macro / sentiment / gold returns	raw external CSVs	*_return, fear_greed	preprocessing.clean_data
DataCleaner.validate_features	preprocessing	NaN / inf / size check	self.df	raises ValueError or passes	preprocessing.clean_data
DataCleaner.normalize_and_split	preprocessing	Chronological split and train-only scaler	self.df	scaled train/val/test, self.scaler	preprocessing.run_cleaning_pipeline
DataCleaner.save_data	preprocessing	Write features, scaled splits, scaler	cleaner state	CSV and .pkl files	preprocessing.run_cleaning_pipeline
_task_target	preprocessing	Forward target of the chosen task	log-return series, task	target series (real units)	preprocessing.build_dataset

Function or method	File	Purpose	Inputs	Outputs	Called by
`_make_windows`	`preprocessing`	Sliding windows	array, seq_len	array (n, seq, F)	`preprocessing.build_dataset`
`build_dataset`	`preprocessing`	**The single loader**: windows, target, standardisation, date masks	asset, seq_len, task, train_start	dict of arrays (`X_*`, `Xt_*`, `y_*`, `dates_*`, ...)	`audit_improvements.run_asset`, `backtesting.main`, `ensemble_model.fit_stack`, `plots.overfitting_gap`, `run_experiments.e1_data_size` +6
`create_sequences`	`preprocessing`	Window / next-row target builder (kept for tests)	DataFrame, seq_len	(X, y)	`tests`
`run_cleaning_pipeline`	`preprocessing`	Run the cleaner for all assets	(none)	files saved	`make preprocess`; `preprocessing.<module>`

Models, saving and loading

Function or method	File	Purpose	Inputs	Outputs	Called by
make_model	registry	Factory	name, params	model object	backtesting.test_predictions, cross_validation.cv_evaluate, registry.load_trained, run_experiments.fixed_val_eval, tests +1
load_trained	registry	Create and load a saved model	name, asset, task	loaded model	backtesting.test_predictions, ensemble_model.__init__, prediction._load
artefact_exists	registry	Is there a saved file for this model?	name, asset	bool	backtesting.model_exists, prediction.available_models, prediction.combined_members
artefact_stem	registry	File stem <prefix>_<model>	asset, model, task	path string	registry.artefact_exists, registry.load, registry.save
ARIMAModel.fit_series	registry	Choose ARIMA order by AIC	training series	fitted model	backtesting.test_predictions, cross_validation.cv_evaluate
ARIMAModel.forecast_walk_forward	registry	One-step forecasts appending truth afterwards	future series	predictions	backtesting.test_predictions, cross_validation.cv_evaluate
LightGBMModel.fit	registry	Two-phase LightGBM fit	train/val arrays, final	self	cross_validation.cv_evaluate, ensemble_model.fit_stack, registry.fit_series, run_experiments.fixed_val_eval, tests +1
CatBoostModel.fit	registry	Two-phase CatBoost fit	train/val arrays, final	self	cross_validation.cv_evaluate, ensemble_model.fit_stack, registry.fit_series, run_experiments.fixed_val_eval, tests +1
_KerasSequenceModel.fit	registry	Two-phase Keras fit with early stopping	windows, targets, final	self	cross_validation.cv_evaluate, ensemble_model.fit_stack, registry.fit_series, run_experiments.fixed_val_eval, tests +1
_KerasSequenceModel.predict	registry	Keras inference	Xseq	standardised predictions	backtesting.test_predictions, cross_validation.cv_evaluate, ensemble_model.predict, prediction._predict_one, prediction.recent_track_record +3
BaseModel.save	registry	joblib save of model, params, fit_info	asset, task	path	train_models.train_asset

Function or method	File	Purpose	Inputs	Outputs	Called by
`CatBoostModel.save`	`registry`	Native `.cbm` + JSON save	asset, task	path	`train_models.train_asset`
`_KerasSequenceModel.save`	`registry`	`.keras` + JSON save	asset, task	path	`train_models.train_asset`
`build_gru_model`	`model_gru`	GRU architecture	seq_len, n_features, units, dropout, lr	compiled Keras model	script entry point / indirect
`build_lstm_model`	`model_lstm`	LSTM architecture	same	compiled Keras model	script entry point / indirect
`fit_stack`	`ensemble_model`	Fit the non-negative stacking meta-model	asset, base names	(meta-model, weights)	`ensemble_model.<module>`
`StackedModel.predict`	`ensemble_model`	Blend base predictions with learned weights	`Xseq`, `Xt`	standardised predictions	`backtesting.test_predictions`, `prediction._predict_one`, `prediction.recent_track_record`

Tuning, training, validation, evaluation

Function or method	File	Purpose	Inputs	Outputs	Called by
grid	tune_models	Iterate all parameter combinations	model name	dicts	tune_models.tune
tune	tune_models	Score every configuration by walk-forward CV and pick the best	asset, model, data	(best params, best row)	tune_models.main
train_asset	train_models	Two-phase training of all models of one asset	asset, model names, task	(metric rows, importance rows)	train_models.main
walk_forward_splits	cross_validation	Expanding-window fold indices	n samples, n splits	(train idx, val idx) pairs	cross_validation.cv_evaluate
trainval_arrays	cross_validation	Concatenate train and val	dataset dict	dict of arrays	audit_improvements.run_asset, cross_validation.cv_evaluate
cv_evaluate	cross_validation	Walk-forward evaluation of one model	model, params, data, folds	list of fold metrics (+ out-of-fold arrays)	audit_improvements.run_asset, backtesting.combined_cv_row, backtesting.cv_table, backtesting.pooled_walkforward, ensemble_model.fit_stack +4
summarise_folds	cross_validation	Mean and sd of each metric	fold dicts	dict *_mean, *_std	backtesting.combined_cv_row, backtesting.cv_table, run_experiments.e2_feature_ablation, run_experiments.e3_horizon, run_experiments.e4_volatility +1
cv_table	backtesting	Walk-forward rows for baselines, models, Combined	asset, data	(rows, oof, member oof)	backtesting.main
combined_cv_row	backtesting	Equal-weight mean of the members' out-of-fold predictions scored per fold	asset, data	rows	backtesting.cv_table
pooled_walkforward	backtesting	DM and direction tests on all out-of-fold days	asset, data, member oof	rows	backtesting.main
band_rows	backtesting	Calibrate k and score three bands	asset, data, prediction table, oof	(rows, k)	backtesting.main
test_predictions	backtesting	Predict the test set once with every model; build Combined	asset, data	(predictions, fit infos)	backtesting.main
regime_analysis	backtesting	Errors by volatility tercile, trend, up/down day	asset, data, predictions	rows	backtesting.main
main	backtesting	Run the whole evaluation and write all results	(none)	CSVs, model_status.json, markdown	make evaluate; backtesting.<module>
write_markdown	backtesting	Render FINAL_RESULTS.md	tables, status	file	backtesting.main

Metrics and utilities

Function or method	File	Purpose	Inputs	Outputs	Called by
rmse	metrics	Root mean squared error	y, ŷ	float	audit_improvements.run_asset, backtesting.pooled_walkforward, metrics.evaluate_forecast, metrics.evaluate_vol_forecast, tests
mae	metrics	Mean absolute error	y, ŷ	float	metrics.evaluate_forecast, metrics.evaluate_vol_forecast
mape	metrics	Mean absolute percentage error	y, ŷ	float	metrics.evaluate_forecast, metrics.evaluate_vol_forecast
r2	metrics	Coefficient of determination	y, ŷ	float	metrics.evaluate_forecast, metrics.evaluate_vol_forecast, tests
directional_accuracy	metrics	Hit-rate on non-flat days + binomial p	true, pred	(%, n, p)	audit_improvements.run_asset, backtesting.pooled_walkforward, metrics.evaluate_forecast, metrics.evaluate_vol_forecast, tests
diebold_mariano	metrics	DM test of squared errors	true, pred A, pred B	(stat, p)	audit_improvements.run_asset, backtesting.pooled_walkforward, metrics.evaluate_forecast, metrics.evaluate_vol_forecast, tests
strategy_backtest	metrics	Long/flat backtest with costs	true, pred, cost, periods	dict	metrics.evaluate_forecast, tests
evaluate_forecast	metrics	Full metric set for a return forecast	true, pred, prev close	dict (≈ 24 metrics)	before_after.main, before_after.metrics_from, metrics.evaluate_task, tests
band_calibration	metrics	Coverage, QLIKE, width of a band	true, pred, sigma	dict	backtesting.band_rows, tests
evaluate_task	metrics	Dispatch to return or volatility metrics	data, true, pred, naive, prev	dict	backtesting.combined_cv_row, backtesting.main, cross_validation.cv_evaluate, run_experiments.e1_data_size, run_experiments.fixed_val_eval +1
reconstruct_price	inverse_transform	P·exp(r)	returns, previous closes	prices	tests
set_all_seeds	reproducibility	Seed Python, NumPy, TensorFlow	seed	(none)	registry.fit

Inference and API

Function or method	File	Purpose	Inputs	Outputs	Called by
load_features	prediction	Read live or frozen feature CSV	asset, live	DataFrame	prediction.predict_for_date, prediction.predict_next_day, prediction.recent_track_record, streamlit_app.<module>, tests
live_data_is_current	prediction	Does the live file end on the newest bar?	asset, now	bool	streamlit_app.<module>
_window	prediction	Scale and cut the last 30-row window; guard features	asset, features, as_of	(Xseq, Xt, last close, date, columns)	prediction.predict_for_date, prediction.predict_next_day
_target_stats	prediction	Training mean / std of the target	asset	(μ, σ)	prediction._predict_one, prediction.recent_track_record, tests
_band_k	prediction	Calibrated band multiplier	status	float (or error)	prediction.predict_for_date, prediction.predict_next_day, tests
_band	prediction	68 % price band	features, date, close, ŷ, k	dict	prediction.predict_for_date, prediction.predict_next_day
combined_members	prediction	Members that exist on disk	asset	list of names	app.list_models, prediction._predict_from_window, prediction.recent_track_record, streamlit_app.<module>, tests
available_models	prediction	Members plus Stacked if present	asset	list	app.list_models, app.predict_asset
_load	prediction	Load one model (or the stack)	asset, model name	model	prediction._predict_one, prediction.recent_track_record
_predict_one	prediction	One model's real log return	asset, model, window	float ŷ	prediction._predict_from_window
_predict_from_window	prediction	Run members and average	asset, model, window	(ŷ, member dict)	prediction.predict_for_date, prediction.predict_next_day
_context	prediction	Held-out metrics, disclaimer, horizon	asset, model, status	dict	prediction.predict_for_date, prediction.predict_next_day
predict_next_day	prediction	**Live next-day forecast**	asset, model name	result dict	app.predict_asset, prediction.predict_next_day_safe, streamlit_app.<module>, tests
predict_for_date	prediction	**Demo: predict the day after a chosen day, reveal the actual**	asset, as_of, model, progress	result dict	streamlit_app.<module>, tests
recent_track_record	prediction	**Live scorecard of every model on the last N days**	asset, n_days, progress	dict (rows, days, provenance)	streamlit_app.<module>, tests

Function or method	File	Purpose	Inputs	Outputs	Called by
prediction_history	prediction	Stored test-day predictions	asset	DataFrame or None	streamlit_app.load_test_predictions
predict_next_day_safe	prediction	Wrapper returning None on error	asset, model	dict or None	prediction.<module>
_resolve	app	Map URL asset to a name; 404 if unknown	string	asset name	app.predict_asset
health_check	app	GET /health	(none)	{"status": "healthy"}	via the registry / Streamlit / framework
list_models	app	GET /models	(none)	dict	via the registry / Streamlit / framework
predict_asset	app	GET /predict/{asset}	asset, optional model	PredictionResponse	via the registry / Streamlit / framework

Streamlit, scripts, experiments

Function or method	File	Purpose	Inputs	Outputs	Called by
load_features	streamlit_app	Cached load of the live or frozen features (Streamlit side)	asset	DataFrame	streamlit_app.<module>
load_csv	streamlit_app	Cached read of a results CSV	file name	DataFrame or None	streamlit_app.<module>
load_test_predictions	streamlit_app	Cached prediction_history	asset	DataFrame	streamlit_app.<module>
test_row	streamlit_app	One model's row in the test table	table, asset, model	Series	streamlit_app.<module>
layout	streamlit_app	Common Plotly layout (dark theme)	figure, height, title	figure	streamlit_app.<module>
pct	streamlit_app	Format %	number	string	streamlit_app.<module>
usd	streamlit_app	Format dollars	number	string	streamlit_app.<module>, streamlit_app.usd_md
usd_md	streamlit_app	Dollar string safe for markdown	number	string	streamlit_app.<module>
run	retrain	Run one stage in a subprocess	command list, description	exits on failure	retrain.main
main	retrain	Orchestrate the pipeline	flags	(none)	python scripts/retrain.py; retrain.<module>
e1_data_size	run_experiments	Experiment E1	(none)	CSV + DataFrame	run_experiments.<module>
e2_feature_ablation	run_experiments	Experiment E2	(none)	CSV	run_experiments.<module>
e3_horizon	run_experiments	Experiment E3	(none)	CSV	run_experiments.<module>
e4_volatility	run_experiments	Experiment E4	(none)	CSV	run_experiments.<module>
run_asset	audit_improvements	Experiment E5 for one asset	asset	rows	audit_improvements.main

Chapter 19: How the Files Connect

19.1 Which file runs first?

There is **no single main program**. The project has independent entry points; each is started by a `make` target or a command. The one that reaches the user is the dashboard.

Activity	Command	First file that runs	First function it calls
Download data	`make collect-data`	`src/data/data_collection.py`, then `external_data.py`	`fetch_asset` per asset; `fetch_all_external_data`
Build features and splits	`make preprocess`	`src/data/preprocessing.py`	`run_cleaning_pipeline` → `DataCleaner(...).clean_data / normalize_and_split / save_data`
Tune	`make tune`	`src/training/tune_models.py`	`main` → `build_dataset` → `tune` → `cv_evaluate`
Train and save models	`make train`	`src/training/train_models.py`	`main` → `train_asset` → `make_model(...).fit` → `save`
Evaluate once	`make evaluate`	`src/evaluation/backtesting.py`	`main` → `cv_table`, `pooled_walkforward`, `test_predictions`, `band_rows`
Show results to users	`make serve`	`app/streamlit_app.py`	module-level code (Streamlit script); on a click, `predict_next_day`
JSON API	`make api`	`src/api/app.py` (via uvicorn)	route functions → `predict_next_day`
Everything	`python scripts/retrain.py`	`scripts/retrain.py`	`main` → `run([...])` per stage

19.2 Layered module map

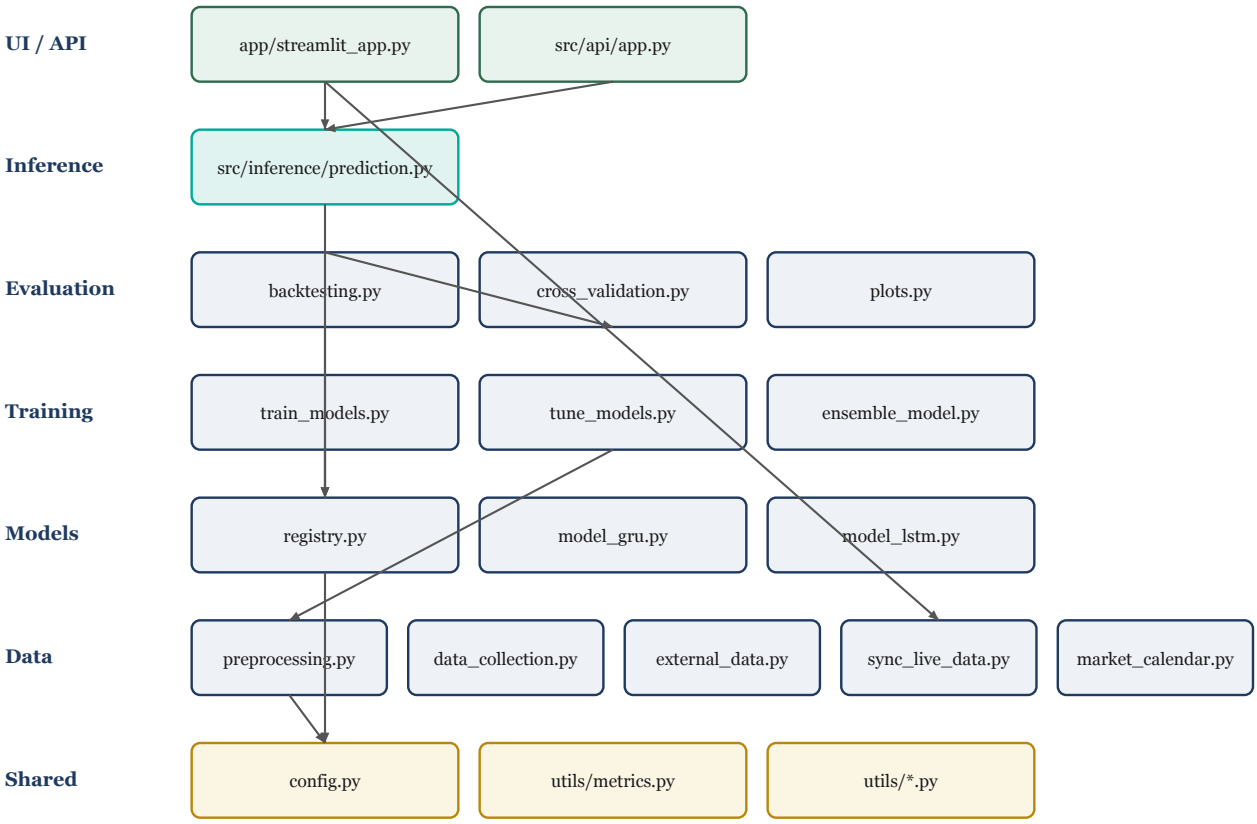


Figure 19.1: Modules by layer. Arrows point from importer to imported file (a selection of the most important links; the exact list follows).

In simple English: `config.py` is the shared settings sheet everybody reads. The *data* layer produces clean tables. The *models* layer knows how to build, save and load every model. *Training* and *evaluation* use the models layer and the dataset loader. *Inference* uses saved models to make a forecast. The *dashboard* and the *API* only call inference (and the live-sync helper); they never train anything.

19.3 Exact import graph (generated from the source)

File	Project files it imports	External libraries
app/streamlit_app.py	config.py, data/market_calendar.py, data/sync_live_data.py, inference/prediction.py	numpy, pandas, plotly, streamlit
config.py	—	—
scripts/retrain.py	—	—
src/api/app.py	config.py, inference/prediction.py	fastapi, pydantic
src/data/data_collection.py	config.py, data/market_calendar.py	pandas, yfinance
src/data/eda.py	config.py	matplotlib, numpy, pandas, scipy, statsmodels
src/data/external_data.py	config.py, data/market_calendar.py	pandas, yfinance
src/data/market_calendar.py	—	pandas
src/data/preprocessing.py	config.py	joblib, numpy, pandas, sklearn, ta
src/data/sync_live_data.py	config.py, data/data_collection.py, data/external_data.py, data/preprocessing.py	—
src/evaluation/backtesting.py	config.py, data/preprocessing.py, evaluation/cross_validation.py, inference/prediction.py, models/ensemble_model.py, models/registry.py, utils/logging_config.py, utils/metrics.py	numpy, pandas
src/evaluation/cross_validation.py	config.py, models/registry.py, utils/metrics.py	numpy, sklearn
src/evaluation/plots.py	config.py, data/preprocessing.py	matplotlib, numpy, pandas
src/experiments/audit_improvements.py	config.py, data/preprocessing.py, evaluation/cross_validation.py, models/registry.py, utils/logging_config.py, utils/metrics.py	numpy, pandas
src/experiments/before_after.py	config.py, utils/metrics.py	numpy, pandas
src/experiments/run_experiments.py	config.py, data/preprocessing.py, evaluation/cross_validation.py, models/registry.py, utils/logging_config.py, utils/metrics.py	numpy, pandas
src/inference/prediction.py	config.py, data/market_calendar.py, models/ensemble_model.py, models/registry.py, utils/logging_config.py	joblib, numpy, pandas
src/models/ensemble_model.py	config.py, data/preprocessing.py, evaluation/cross_validation.py, models/registry.py, utils/logging_config.py	joblib, numpy, sklearn
src/models/model_gru.py	—	tensorflow

File	Project files it imports	External libraries
src/models/model_lstm.py	—	tensorflow
src/models/registry.py	config.py, models/model_gru.py, models/model_lstm.py, utils/reproducibility.py	catboost, joblib, lightgbm, numpy, sklearn, statsmodels, tensorflow
src/training/train_models.py	config.py, data/preprocessing.py, models/registry.py, utils/logging_config.py, utils/metrics.py	pandas
src/training/tune_models.py	config.py, data/preprocessing.py, evaluation/cross_validation.py, utils/logging_config.py	pandas
src/utils/inverse_transform.py	—	numpy
src/utils/logging_config.py	—	—
src/utils/metrics.py	—	numpy, scipy
src/utils/reproducibility.py	—	numpy, tensorflow

19.4 Data flow: which file writes what, and who reads it

Producer	Artefact	Consumers
data_collection.py, external_data.py	data/raw/*.csv	DataCleaner (preprocessing)
DataCleaner.save_data	data/processed/<p>_features.csv, <p>_{train,val,test}_scaled.csv, data/models/<p>_scaler.pkl	build_dataset; dashboard (charts); plots.py, eda.py; predict_for_date (frozen data)
sync_live_data.update_live_data	data/raw_live/*, data/processed/<p>_live_features.csv	prediction.load_features, dashboard
tune_models.py	results/tuning/best_params.json, tuning_summary.csv	config.get_params (training, CV), backtesting.cv_table
train_models.py → BaseModel.save	data/models/<p>_<model>.pkl|.cbm|.keras (+ json), results/train_val_metrics.csv, feature_importance.csv, histories/*.json	load_trained (inference, evaluation, stack); dashboard (importance); plots.py
ensemble_model.fit_stack	data/models/<p>_stack.pkl, results/stacking/*.json	StackedModel (evaluation, "Stacked" row)
backtesting.py	results/cv_results.csv, final_test_results.csv, walkforward_pooled.csv, band_calibration.csv, regime_analysis.csv, predictions/*.csv, FINAL_RESULTS.md, data/models/model_status.json	prediction.py (target mean/std, band k, held-out metrics), dashboard (all tables), API (/models)
prediction.predict_next_day / predict_for_date	result dictionary (in memory)	Streamlit blocks; PredictionResponse in the API

19.5 Where does each key event happen?

Question	Answer
Where are the models trained?	`train_asset` in `src/training/train_models.py` (`make train`), using `make_model(...).fit` from `registry.py` and arrays from `build_dataset`.
Where are model artefacts saved?	`BaseModel.save` / `CatBoostModel.save` / `_KerasSequenceModel.save` (registry) into `data/models/`; scaler by `DataCleaner.save_data`; `model_status.json` by `backtesting.main`.
Where are saved models loaded?	`load_trained` → `BaseModel.load` etc. Called by `prediction._load` (every forecast), `backtesting.test_predictions`, and `StackedModel.__init__`.
Where is the final prediction calculated?	`prediction._predict_from_window` (mean of six) inside `predict_next_day` / `predict_for_date`; price `last_close * np.exp(r_hat)`; band in `_band`.
How does the result reach the dashboard?	The Forecast tab calls `predict_next_day(asset, COMBINED)` and stores the dictionary in `st.session_state["run_result"]`; the HTML card, tables and Plotly charts are drawn from it.
Does the dashboard train at start-up?	**No.** At start-up it reads CSV/JSON results, may download new data, and loads models only when a prediction is requested.
Are all artefacts pre-trained?	Yes on this machine (18 models, 3 stacks, 3 scalers exist). On a fresh clone **none exist** until `make train` has run: `data/models/` is git-ignored except `model_status.json`, and `predict_next_day` raises `FileNotFoundError` / `ValueError`.

> **Explain the whole diagram in one breath**
> "`config.py` holds the settings. `preprocessing.py` makes features and one loader, `build_dataset`. `train_models.py` fits the six models through the registry and saves them. `backtesting.py` evaluates them once on the test days and writes `model_status.json`. `prediction.py` loads the saved models, averages their predictions and builds the band. `streamlit_app.py` and `api/app.py` just call `prediction.py` and display or return the result."

Chapter 20: Important Code Explained

These are the code sections to understand before the viva, in the order an examiner would probe them: target, features, split, validation, training, evaluation, ensemble, band, prediction, dashboard. Line numbers are the real ones in VS Code.

20.1 Target generation and the split-by-target-date masks

src/data/preprocessing.py

```
365     pos = features.index.get_indexer(dates_t)
366     first_target = features.index[pos + 1]
367     last_target = features.index[np.minimum(pos + h, len(features) - 1)]
368     t_end, v_end = pd.Timestamp(TRAIN_END), pd.Timestamp(VAL_END)
369     masks = {'train': (last_target <= t_end) & ((dates_t >= pd.Timestamp(train_start)) if train_start
            else True),
370             'val': (first_target > t_end) & (last_target <= v_end),
371             'test': first_target > v_end}
372     masks = {k: np.asarray(v, bool) for k, v in masks.items()}
373
```

What it does	For every sample (day t) finds the date of its first and last **target** day and decides if the sample belongs to train, validation or test.
Why needed	A sample's label lies in the future. If splits were decided by the day t alone, the last training sample's label could fall in the validation period (and for multi-day targets overlap it).
How it works	`pos` is the row number of each day t in the feature table; `first_target` = the next row's date; `last_target` = the row h days later (h = 1 for the served task). Train: `last_target <= TRAIN_END`; validation: `first_target > TRAIN_END` **and** `last_target <= VAL_END`; test: `first_target > VAL_END`.
Receives	`dates_t` (day t of every window), the feature table's index, `h`, `TRAIN_END`, `VAL_END`.
Returns / changes	Boolean masks; later used to slice `X`, the target and the dates into `*_train`, `*_val`, `*_test` entries of the dataset dict. Nothing is changed in the data files.
Called by	`build_dataset` (and therefore tuning, training, stacking, evaluation, inference tests, experiments).
If removed or changed	Splitting by day t instead would let training targets reach into validation (a small but real leak); changing `TRAIN_END`/`VAL_END` in `config.py` moves all splits and invalidates every saved model and result.

20.2 Feature alignment: the close-time rule

src/data/preprocessing.py

```
146     def _align_external(self, series):
147         """
148         Align an external daily series onto THIS asset's calendar using only values known when P_t is
149         observed. Crypto (00:00 UTC close, after the US close): the same day's value, forward-filled.
150         Commodities (13:30 ET settlement, before the US close): the value from the latest external
                date
151         STRICTLY before day t.
152         """
153         if EXTERNAL_SAME_DAY[self.config['type']]:
154             return series.reindex(self.df.index, method='ffill')
155         pos = series.index.searchsorted(self.df.index, side='left') - 1      # last external date < t
156         vals = np.where(pos >= 0, series.values[np.clip(pos, 0, None)], np.nan)
157         return pd.Series(vals, index=self.df.index)
```

What it does	Puts an external daily series (e.g. S&P 500 returns) onto the asset's own calendar using **only values known when the asset's close is fixed**.

Why needed	Gold and Silver settle at 13:30 ET, before the US equity close. Using the S&P value of the same date would let a feature contain news from inside the interval being predicted: this produced the fake 62 % Silver result.
How it works	Crypto (`EXTERNAL_SAME_DAY["crypto"] = True`): reindex with forward fill, i.e. the same day's value (and Friday's value repeated on weekends). Commodity: `series.index.searchsorted(self.df.index, side="left") - 1` is the position of the **last external date strictly before t**, and the value at that position is used; `np.where(pos >= 0, ..., np.nan)` guards the first rows.
Receives	a Series of external log returns on its own calendar.
Returns / changes	A Series indexed like `self.df`; the caller applies `.fillna(0)`.
Called by	`add_external_features` (once per macro series, and for Fear & Greed).
If removed or changed	Replace `side="left"` logic by a plain reindex and the metals leak again; two tests (`test_commodity_external_features_are_previous_day_values`, `test_no_lookahead_in_any_feature`) would fail or the reported accuracy would become unrealistically good.

src/data/preprocessing.py (companion rule for High/Low features)

```
136     def lag_post_settlement_features(self):
137         """
138         Futures only. Yahoo's daily High/Low for GC=F / SI=F cover the whole Globex session (until
                17:00 ET),
139         but the Close is the 13:30 ET settlement. Features built from the day's High/Low would
                therefore
140         contain up to 3.5 hours of trading that happens AFTER P_t is fixed — inside the target
                interval.
141         They are lagged one session so that row t only uses the previous session's range.
142         """
143         for c in POST_SETTLEMENT_FEATURES:
144             self.df[c] = self.df[c].shift(1)
```

20.3 Time-series cross-validation: the walk-forward loop

src/evaluation/cross_validation.py

```
57      for k, (tr, va) in enumerate(walk_forward_splits(len(y), n_splits)):
58          if h > 1:
59              tr = tr[:-(h - 1)]                        # purge overlapping targets
60          n_inner = int(len(tr) * inner_val_frac)
61          tr_fit, tr_mon = tr[:-n_inner], tr[-n_inner:]
62
63          if model_name == 'ARIMA':
64              m = make_model('ARIMA').fit_series(np.ravel(y[tr]))
65              pred = m.forecast_walk_forward(np.ravel(y[va]))
66          elif model_name == 'Naive':
67              pred = data['fwd'](A['naive'][va])
68          elif model_name == 'Naive-Mean':
69              pred = np.full(len(va), float(y[tr].mean()))
70          elif model_name == 'EWMA':
71              pred = data['fwd'](np.log(data['features']['ewma_vol'].reindex(
72                  np.concatenate([data['dates_train'], data['dates_val']])[va]).values.clip(1e-6)))
73          else:
74              m = make_model(model_name, params)
75              m.fit(Xs[tr_fit], Xt[tr_fit], y[tr_fit], Xs[tr_mon], Xt[tr_mon], y[tr_mon], final=True)
76              pred = m.predict(Xs[va], Xt[va])
77
78          met = evaluate_task(data, A['y_real'][va], data['inv'](pred), A['naive'][va], A['prev'][va])
79          met.update({'fold': k + 1, 'n_train': len(tr), 'n_val': len(va)})
80          folds.append(met)
```

What it does	For each of the 4 folds: train a model on everything before the validation block and score it on the block.
Why needed	It gives several out-of-sample scores in time order, used both to pick hyper-parameters and as the pooled six-year evidence.
How it works	`tr` and `va` come from `TimeSeriesSplit`. For multi-day targets the last h−1 training samples are removed. `n_inner = 15 %` of the training part becomes the early-stopping monitor (`tr_mon`), the rest `tr_fit`. Special cases: ARIMA (walk-forward forecasts), Naive, Naive-Mean, EWMA. Otherwise `make_model(...).fit(..., final=True)` then `predict(Xs[va], Xt[va]).evaluate_task` computes the fold metrics; predictions and indices are collected for pooling.
Receives	model name, params, the dataset dict, `n_splits`.
Returns	list of fold dicts (+ out-of-fold predictions and indices if `return_oof=True`).
Called by	`tune`, `cv_table`, `combined_cv_row`, `pooled_walkforward`, `fit_stack`, experiments.
If removed or changed	Without it there is no tuning, no walk-forward table and no pooled test. If it accidentally used shuffled or random splits, every validation number would be inflated.

20.4 Model training: the two-phase fit of the neural networks

src/models/registry.py

```
278     def fit(self, Xseq_tr, Xt_tr, y_tr, Xseq_val=None, Xt_val=None, y_val=None, final=False):
279         from tensorflow.keras.callbacks import EarlyStopping
280         set_all_seeds(42)
281         bs = self.params.get('batch_size', 32)
282         epochs = self.params.get('epochs', 80)
283         patience = self.params.get('patience', 10)
284         seq_len, n_features = Xseq_tr.shape[1], Xseq_tr.shape[2]
285 
286         if Xseq_val is not None and y_val is not None:
287             m1 = self._build(seq_len, n_features)
288             # Only early stopping: phase B has no validation signal, so phase A must not use any other
289             # validation-driven schedule (an LR schedule here would make the two fits different
                    procedures).
290             cbs = [EarlyStopping(monitor='val_loss', patience=patience, restore_best_weights=True)]
291             h = m1.fit(Xseq_tr, y_tr, validation_data=(Xseq_val, y_val), epochs=epochs,
292                        batch_size=bs, callbacks=cbs, verbose=0, shuffle=False)
293             self.history = {k: [float(x) for x in v] for k, v in h.history.items()}
294             best_epoch = int(np.argmin(h.history['val_loss'])) + 1
295             self.fit_info = {'epochs_used': best_epoch}
296             if not final:
297                 self.model = m1
298                 return self
299             X = np.concatenate([Xseq_tr, Xseq_val]); y = np.concatenate([y_tr, y_val])
300         else:
301             best_epoch, X, y = epochs, Xseq_tr, y_tr
302             self.fit_info = {'epochs_used': best_epoch}
303         set_all_seeds(42)
304         self.model = self._build(seq_len, n_features)
305         self.model.fit(X, y, epochs=best_epoch, batch_size=bs, verbose=0, shuffle=False)
306         return self
```

What it does	Phase A (`Xseq_val` given): fit on train with early stopping on validation loss, record the best epoch; phase B (`final=True`): rebuild the network from scratch and fit on train+validation for exactly that number of epochs.
Why needed	The deployed model should use all pre-test data, but the stopping epoch must be chosen *without* validation leaking into its own monitor, and phase B has no monitor.

How it works	`set_all_seeds(42)` before each build. `EarlyStopping(monitor="val_loss", patience=patience, restore_best_weights=True)`; `shuffle=False` keeps time order; `best_epoch = argmin(val_loss) + 1` is stored in `fit_info["epochs_used"]` and the loss history in `self.history`.
Receives	windows `Xseq_*`, standardised targets `y_*`, `final` flag, parameters from `get_params`.
Returns / changes	`self.model` (Keras), `self.fit_info`, `self.history`.
Called by	`train_asset` (both phases), `cv_evaluate` (with `final=True` on the inner split).
If removed or changed	Removing early stopping would let the networks overfit; removing `shuffle=False` would shuffle windows; changing the seeds changes every network's results.

20.5 Model evaluation: the Diebold-Mariano test

src/utils/metrics.py

```python
def diebold_mariano(true_ret, pred_a, pred_b, h=1):
    """
    Diebold–Mariano test on squared-error loss differential  d_t = e_a² − e_b².
    H0: equal forecast accuracy. Negative statistic ⇒ model A better than model B.
    Uses HAC (Newey–West) variance with h−1 lags, and the Harvey et al. small-sample correction.
    Returns (statistic, two-sided p-value).
    """
    true_ret, pred_a, pred_b = map(np.ravel, (true_ret, pred_a, pred_b))
    d = (true_ret - pred_a) ** 2 - (true_ret - pred_b) ** 2
    n = len(d)
    if n < 10 or np.allclose(d, 0):
        return np.nan, np.nan
    d_bar = d.mean()
    gamma0 = np.mean((d - d_bar) ** 2)
    var = gamma0
    for k in range(1, h):
        gk = np.mean((d[k:] - d_bar) * (d[:-k] - d_bar))
        var += 2 * gk
    var = max(var, 1e-18)
    dm = d_bar / np.sqrt(var / n)
    dm *= np.sqrt((n + 1 - 2 * h + h * (h - 1) / n) / n)      # Harvey–Leybourne–Newbold
    p = 2 * (1 - stats.t.cdf(abs(dm), df=n - 1))
    return float(dm), float(p)
```

What it does	Tests whether forecast A and forecast B have different squared errors; negative statistic means A is better.
Why needed	A 0.3 % lower RMSE can be luck. The test says how likely a difference this size is if both forecasts were equally good. Without it one could wrongly claim an improvement.
How it works	Loss differential `d = (y − a)² − (y − b)²`; `dm = d̄ / sqrt(var/n)` (for h = 1 the variance is just the variance of d); small-sample correction `sqrt((n+1−2h+h(h−1)/n)/n)`; p from Student-t with n−1 degrees of freedom, two-sided.
Receives	true returns, predictions A and B (B = zeros for the random walk).
Returns	(statistic, p-value); `(nan, nan)` for fewer than 10 days or identical forecasts.
Called by	`evaluate_forecast` (every test/CV metric dict), `pooled_walkforward`, `audit_improvements`.
If removed or changed	The "DM p" columns, the overview strip's "does not beat the random walk" sentence and every significance claim would disappear.

20.6 Ensemble calculation in the evaluation (walk-forward)

src/evaluation/backtesting.py

```
93      mean_pred = np.mean([oof[m] for m in members], axis=0)
94      y_real = np.concatenate([data['y_real_train'], data['y_real_val']])
95      naive = np.concatenate([data['naive_train'], data['naive_val']])
96      prev = np.concatenate([data['prev_train'], data['prev_val']])
97      folds, start = [], 0
98      for n_val in fold_sizes:
99          va = idx[start:start + n_val]
100         folds.append(evaluate_task(data, y_real[va], data['inv'](mean_pred[start:start + n_val]),
                naive[va], prev[va]))
101         start += n_val
102     log.info(f"{asset}: Combined walk-forward RMSE_ret={np.mean([f['RMSE_ret'] for f in folds]):.5f}
            over {len(folds)} folds "
```

What it does	Scores the Combined forecast on the walk-forward folds: averages the six members' out-of-fold predictions and evaluates the average block by block.
Why needed	The served forecast must be evaluated *exactly like* a single model, without choosing anything on the test set.
How it works	`oof[m]` holds each member's out-of-fold predictions from `cv_evaluate(..., return_oof=True)`; `np.mean(..., axis=0)` is the equal-weight combination; `data["inv"]` converts back to real log returns; `evaluate_task` scores each validation block.
Called by	`cv_table`; the same member predictions feed `pooled_walkforward`.
If removed or changed	The Combined row would vanish from `cv_results.csv`; changing the mean to a weighted sum would turn "no fitted weights" into something that must be fitted on out-of-fold data.

src/inference/prediction.py (the same average at prediction time)

```
162     if model_name != COMBINED:
163         return _predict_one(asset, model_name, Xseq, Xt), None
164     members = {}
165     for m in combined_members(asset):
166         t0 = time.perf_counter()
167         members[m] = _predict_one(asset, m, Xseq, Xt)
168         if progress:
169             progress(m, members[m], time.perf_counter() - t0)
170     if not members:
171         raise FileNotFoundError(f"No trained models found for {asset}. Run `make train` first.")
172     return float(np.mean(list(members.values()))), members
173
```

20.7 The calibrated 68 % band

src/evaluation/backtesting.py

```
163     fit = idx < n_tr                                                # out-of-fold days of the
            training split → calibration
164     err = np.abs(y_all - pred)
165     k = float(np.quantile(err[fit] / sig_all[fit], BAND_NOMINAL))
166     w = float(np.quantile(err[fit], BAND_NOMINAL))
167     log.info(f"  band calibration on {int(fit.sum())} training-split out-of-fold days: k = {k:.3f},
            fixed half-width = {w * 100:.2f}%")
```

What it does	Finds k, the multiplier of σ_t, so that the band $P_t \cdot \exp(\hat{y} \pm k \cdot \sigma_t)$ contains the truth about 68.27 % of the time.
Why needed	Daily returns are fat-tailed, so the plain ±1σ band covered 73 to 75 % instead of 68 %. A band that claims 68 % must deliver it on unseen data.

How it works	`fit = idx < n_tr` selects out-of-fold days whose target lies in the **training split**; `k` is the 68.27 % quantile of |error| / σ_t on those days only (0.825 / 0.909 / 0.870); the same rule gives the fixed-width comparison band. The validation and test days are then out-of-sample checks. `k` is saved to `model_status.json["band"]["k"]` and read by `prediction._band_k`.
Receives	out-of-fold Combined predictions and their indices, `σ_t` from the `ewma_vol` feature.
Called by	`backtesting.main`; consumed by `predict_next_day`, `predict_for_date`, the dashboard and the API.
If removed or changed	`_band_k` raises `FileNotFoundError` and **no forecast can be produced** (deliberately: a stale file must not silently serve the old ±1σ band).

20.8 Prediction: the live next-day forecast

src/inference/prediction.py

```
197 def predict_next_day(asset, model_name=COMBINED):
198     """Forecast for the trading day after the last complete bar (default: the combined forecast)."""
199     status = load_model_status().get(asset, {})
200     model_name = model_name or COMBINED
201     feats = load_features(asset)
202     Xseq, Xt, last_close, as_of, cols = _window(asset, feats)
203     r_hat, members = _predict_from_window(asset, model_name, Xseq, Xt)
204     pred_price = last_close * np.exp(r_hat)
205     ctx = _context(asset, model_name, status)
206     result = {
207         'asset': asset, 'as_of_date': str(as_of.date()), 'current_price': last_close,
208         'target_date': str(next_trading_day(ASSET_CONFIG[asset]['type'], as_of)),
209         'predicted_price': float(pred_price), 'predicted_return_pct': r_hat * 100,
210         'direction': 'UP' if r_hat > 0 else 'DOWN', 'n_features': len(cols),
211         **_band(feats, as_of, last_close, r_hat, _band_k(status)), 'band_calibration':
                status.get('band'),
212         'data_source': 'live download' if _features_path(asset).endswith('_live_features.csv') else
                'stored dataset',
213         'individual': _individual(members, last_close), **ctx,
214     }
215     logger.info(f"{asset}: {model_name} as of {as_of.date()} close ${last_close:,.2f} →
            ${pred_price:,.2f} ({r_hat*100:+.2f}%)")
216     return result
```

What it does	Produces everything the user sees for "Run prediction": price, direction, band, individual member predictions, held-out metrics and the disclaimer.
Why needed	It is the single backend entry point for the dashboard's Forecast tab and the API.
How it works	Status → `load_features` (live if available) → `_window` (scale, last 30 rows) → `_predict_from_window` (six models, mean) → `last_close * exp(r_hat)` → `_band(..., _band_k(status))` → merge `_context` (test metrics) → return the dict. `next_trading_day` gives the target date label.
Receives	asset name, optional model name (default Combined).
Returns	dict with `predicted_price`, `predicted_return_pct`, `direction`, `uncertainty_band`, `individual`, `test_metrics`, … (see `PredictionResponse`).
Called by	`app/streamlit_app.py` (Forecast tab), `src/api/app.py` (`/predict`), `predict_next_day_safe`, `make predict`.
If removed or changed	No forecast; the unit test `test_served_prediction_is_sane_and_combines_every_trained_model` checks price = P·exp(r), direction, and mean-of-members.

20.9 The main dashboard control flow

app/streamlit_app.py

```
138 if st.sidebar.button("Run prediction", type="primary", width='stretch',
139                      help="Downloads the latest complete bar, runs every trained model and combines
                             them into one forecast.",
140                      on_click=lambda: st.session_state.update(open_tab="Forecast")):
141     st.session_state['run_prediction'] = True
142     st.session_state.pop('run_result', None)
143     st.session_state.pop('run_score', None)                # the live scorecard is recomputed on
            every press
144 st.session_state.setdefault('run_prediction', False)
145 st.sidebar.caption(f"Runs **all {len(members)} trained models** ({', '.join(members) or 'none'}) and
    serves their "
146                    f"equal-weight combination. No model to choose.")
    ...
209 TAB_LABELS = ["Forecast", "Predict a Day (unseen test)", "Model Performance", "Methodology"]
210 # a button press reruns the script; `open_tab` (set by the button's on_click) keeps the user on the
    tab they pressed it in
211 tab_fc, tab_demo, tab_perf, tab_meth = st.tabs(TAB_LABELS, default=st.session_state.get('open_tab',
    TAB_LABELS[0]))
212
```

What it does	The Run-prediction button sets state flags; the tab object is created with the remembered tab as default so a click does not jump back to "Forecast".
Why needed	Streamlit re-runs the whole script on every click, so state must live in `st.session_state`; otherwise the prediction would vanish after the next click.
How it works	`on_click=lambda: st.session_state.update(open_tab="Forecast")` runs *before* the rerun; the body then sets `run_prediction = True` and pops the cached results (`run_result`, `run_score`) so everything is recomputed for the new press. `st.tabs(TAB_LABELS, default=...)` reads `open_tab`.
Called by	Streamlit itself, on every interaction.
If removed or changed	Without the button logic no prediction is started; without `open_tab`, pressing buttons in other tabs would reset the view.

Chapter 21: Configuration, Dependencies and Running the Project

21.1 Required software

Item	Version / note
Python	3.9.6 in the project's `venv` (README badge "3.9+"); the CI workflow uses 3.11. TensorFlow 2.16.2 supports both
pip	any recent version; installs the pinned list
GNU Make	optional, but every stage has a `make` target (macOS ships with it)
Internet access	needed only for `collect-data`, the live sync and the dashboard's automatic refresh; Yahoo may rate-limit
Disk space	about 30 MB of project data (`data/raw` 1 MB, `data/processed` 16 MB, `data/models` 2.1 MB, `results` 9.5 MB) plus the virtual environment (TensorFlow is the largest package)
A web browser	for the dashboard (default `http://localhost:8501`) and the API docs (`/docs`)

21.2 Dependencies (`requirements.txt`)

Group	Packages
Core	`pandas==2.3.3`, `numpy==1.26.4`, `scikit-learn==1.6.1`, `joblib==1.5.3`
Deep learning	`tensorflow==2.16.2` (brings Keras 3.10.0)
Trees	`lightgbm==4.6.0`, `catboost>=1.2.0`, `scipy>=1.10.0`
Time series	`statsmodels==0.14.6`
Data sources	`yfinance==1.2.0`, `requests==2.32.5` (listed, not imported), `tzdata>=2024.1`
Visualisation and apps	`matplotlib==3.9.4`, `plotly==6.7.0`, `streamlit==1.50.0`, `fastapi>=0.100.0`, `uvicorn>=0.20.0`
Indicators / tests / notebooks	`ta>=0.11.0`, `pytest>=7.0.0`, `jupyter==1.1.1`

21.3 Configuration and environment

- **One configuration file:** `config.py` (Chapter 17). To change the split, the tickers, the sequence length or the default hyper-parameters, edit it, then re-run `make preprocess` and everything after.
- **Environment variables:** none required. Optional `TF_CPP_MIN_LOG_LEVEL=3` (silences TensorFlow start-up messages; the scripts already set it). Copy `.env.example` to `.env` only if you want to set it; `.env` is git-ignored.
- **Secrets:** none exist. Do not put keys in `config.py`.

21.4 What must exist before each step

To run	Needs
`make preprocess`	`data/raw/*.csv` (9 files; from `make collect-data`)
`make tune` / `make train`	processed splits and scalers; `train` also uses `results/tuning/best_params.json` (committed, so tuning can be skipped)
`make evaluate`	trained models in `data/models/`; `make stack` first if you want the Stacked row
`make serve` / `make api` / `make predict`	processed features, all model files, **and** `data/models/model_status.json` **written by** `make evaluate` (it holds the target mean/std and band k)
`make test`	tests that need models or data are skipped automatically if they are absent

21.5 Commands (exact, from the Makefile)

`Makefile` defines `PY ?= python`. On this Mac there is no `python` command (only `python3` and the venv), so either activate the environment or pass `PY=venv/bin/python`.

```
# one-time setup
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# full pipeline from scratch (needs internet for the first line)
make collect-data      # download raw data (Yahoo Finance, alternative.me)
make preprocess        # features, frozen split, scaler
make eda               # stationarity tests, ACF/PACF, distributions
make tune              # walk-forward grid search (about 30 min; optional, params are committed)
make train             # two-phase training of the 6 models x 3 assets
make stack             # stacked-ensemble experiment
make evaluate          # the single test evaluation; writes results/ and model_status.json
make figures           # report figures
make experiments       # E1 to E5 and before/after
make test              # 47 tests

# or everything except downloading
make pipeline
python scripts/retrain.py --no-fetch          # same idea, with logging; --no-tune skips tuning

# use the system
make predict           # print today's combined forecast for each asset
make serve             # Streamlit dashboard: http://localhost:8501
make api               # FastAPI: http://localhost:8000/docs
```

Without Make (what the targets expand to): `python src/data/preprocessing.py`, `python src/training/train_models.py`, `python src/evaluation/backtesting.py`, `streamlit run app/streamlit_app.py`, `uvicorn src.api.app:app --reload`. A fresh clone needs `make collect-data preprocess train stack evaluate` (about 15 minutes without tuning) before `make serve`.

21.6 Reproducing the results

1. Use the pinned environment (`requirements.txt`; Python 3.9 or 3.11).
2. Keep the frozen data: `data/raw/` as downloaded on 2026-09-12/14. **A new download changes the numbers slightly** (Yahoo revises history), so reproduction to 1e-13 requires the original files.
3. Run `make preprocess train stack evaluate` (tuning is optional because `results/tuning/best_params.json` is tracked).
4. Compare `results/final_test_results.csv`, `cv_results.csv`, `walkforward_pooled.csv` with the committed copies; the 2026-10-04 audit found agreement to ≤ 1e-13.
5. Run `make test` (47 should pass).

21.7 Troubleshooting

Symptom	Likely cause	Fix
`make: python: No such file`	No `python` alias on macOS	`PY=venv/bin/python make <target>` or activate the venv
Dashboard: "No evaluation found. Run make evaluate"	`results/*.csv` or `model_status.json` missing	`make evaluate`
`FileNotFoundError: model_status.json has no calibrated band multiplier`	Stale `model_status.json` (older than the band calibration)	`make evaluate`
`FileNotFoundError ... No processed data`	Preprocessing not run	`make preprocess`
`ValueError: Feature mismatch for Gold`	Feature file and scaler disagree (e.g. a macro CSV missing at preprocessing time)	Restore the raw CSVs and re-run `make preprocess`, then retrain
Warning "Yahoo Finance has not yet published the ... bar"	Vendor lag (Bitcoin's previous-day bar can be hours late)	Wait and press Run prediction again; the forecast starts from the last complete bar
First prediction takes 15 to 20 s	TensorFlow and models load from disk	Normal; later runs are faster
Yahoo returns nothing	Rate limit	Use the stored data (frozen or last live file) and retry later
Tests skipped	`data/models/model_status.json` absent	Train and evaluate first

Chapter 22: Trace One Prediction Through the Actual Code

Scenario: **a user selects Gold in Streamlit and presses Run prediction.** Path: user action → UI → data loading → feature pipeline → model inference → ensemble → result formatting → display. Real values come from running the project's own code on 2026-10-09: `predict_for_date("Gold", "2026-05-12")` on the frozen dataset (reproducible any day) for the intermediate objects, and the numbers the live dashboard actually displayed that afternoon for the final card.

22.1 The call chain

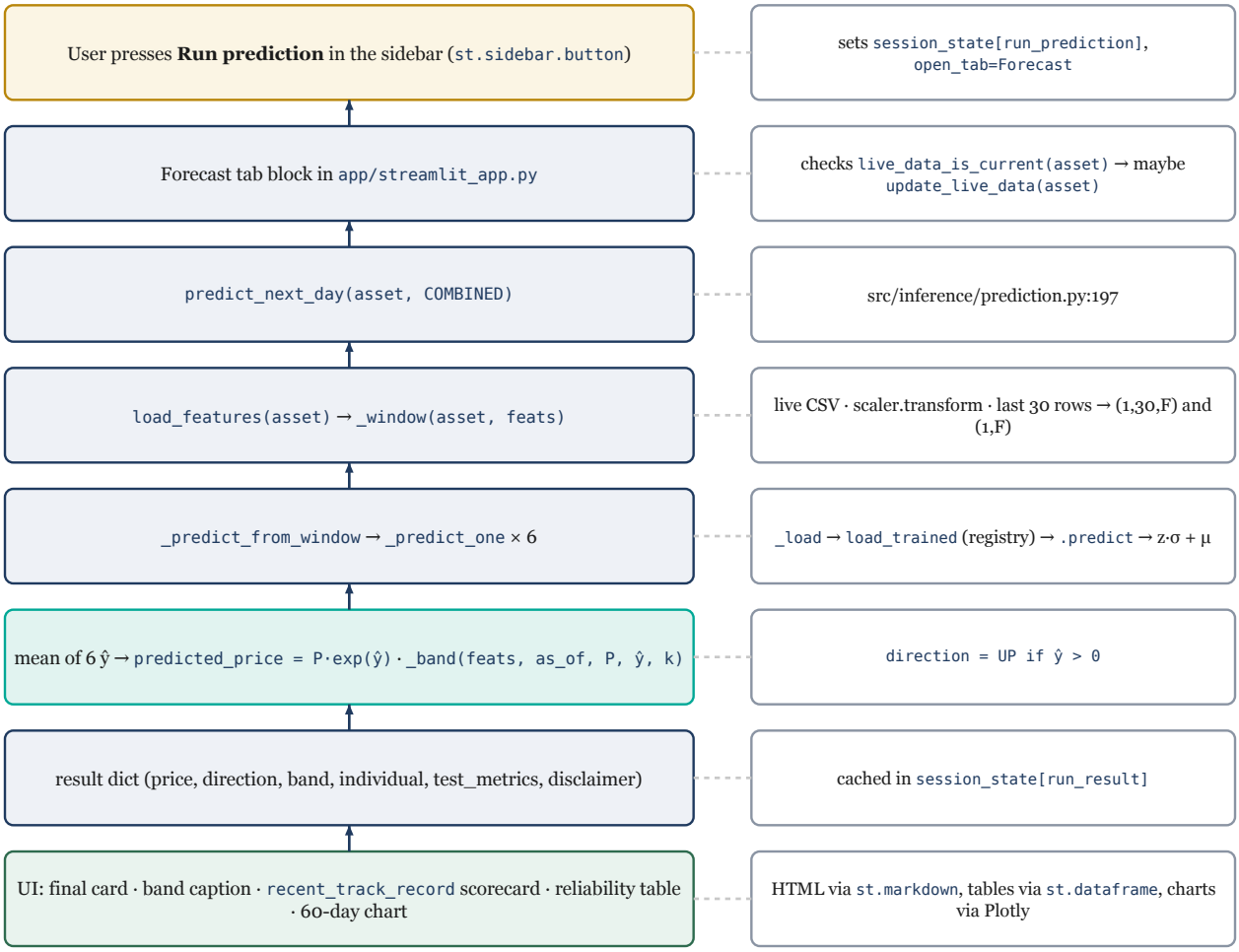


Figure 22.1: From the button to the screen. Left: what runs. Right: what it needs or produces.

22.2 Step by step with file and function names

#	File : function (line)	What happens	Data after the step
1	`app/streamlit_app.py` sidebar button (line 138)	Streamlit re-runs the script; `st.sidebar.button` returns True; `on_click` already stored `open_tab="Forecast"`; the body sets `run_prediction=True` and deletes `run_result` and `run_score` from `session_state`.	flags in `session_state`
2	`streamlit_app.py` Forecast tab (line 234)	The tab block sees `run_prediction` and a non-empty `members` list (`combined_members(asset)` = six names).	`members = [Ridge, RandomForest, LightGBM, CatBoost, GRU, LSTM]`
3	`live_data_is_current`, `update_live_data`	If the live file does not end on the newest expected bar and is older than 30 minutes, Yahoo is queried and `<p>_live_features.csv` rewritten atomically.	live CSV (Gold on 2026-10-09: last bar 2026-10-08)
4	`prediction.predict_next_day("Gold", "Combined")` (line 197)	Entry to the backend; reads `load_model_status()["Gold"]`.	`status` dict (target mean/std, band k, test metrics)
5	`prediction.load_features` → `_features_path`	Chooses the live file when it exists, else the frozen file; reads the CSV with the date index.	DataFrame, 28 model columns + chart columns
6	`prediction._window`	Loads `gold_scaler.pkl`, checks feature count and **names**, re-orders columns, `scaler.transform`, keeps the last 30 rows.	`Xseq` shape (1, 30, 28); `Xt` shape (1, 28); `last_close` = P_t
7	`_predict_from_window` → `_predict_one` (x6)	For each member: `_load` → `registry.load_trained` → `BaseModel.load` (or Keras / CatBoost load) → `.predict(Xseq, Xt)` → `z * sd + mu` (`_target_stats`).	six floats $\hat{y}^{(m)}$
8	`np.mean`	Equal-weight combination.	$\hat{y}$ (one float)
9	`predict_next_day` (rest)	`pred_price = last_close * exp(r_hat)`; `direction`; `_band(feats, as_of, last_close, r_hat, _band_k(status))`; `_context(...)` adds test metrics; `next_trading_day` labels the target date.	result dict (about 30 keys)
10	`streamlit_app.py` (after the call)	Stores `(asset, r, refreshed)` in `session_state["run_result"]`; draws the final card with `st.markdown(..., unsafe_allow_html=True)`.	HTML card
11	`prediction.recent_track_record(asset, n_days, progress)`	Scales the whole live frame once, builds N windows, runs each member on all N days, averages for Combined, scores against the next closes and a "tomorrow = today" error.	dict with `rows`, `days`, provenance
12	Streamlit drawing code	Per-model table, LIVE badge, day-by-day expander, Plotly chart (last 60 days + forecast + band), reliability table from `final_test_results.csv`.	what the user sees

22.3 How the data changes along the way (real example)

Object	Type / shape	Real value (Gold, standing on 2026-05-12, frozen dataset)
Raw bar	CSV row	close $4,677.60 on 2026-05-12
Feature row (unscaled)	28 floats	e.g. `RSI` = 48.92, `ewma_vol` = 0.0160, `sp500_return` = 0.0019 (previous trading day), `return_20d` = -0.0310
Scaled window `Xseq`	float array (1, 30, 28)	rows 2026-03-31 to 2026-05-12; `ewma_vol` scaled = 0.524; values in [0, 1] (may exceed for new highs)
Last row `Xt`	float array (1, 28)	what Ridge, RandomForest, LightGBM, CatBoost receive
Model outputs z	6 floats (standardised)	Ridge +0.0311, RandomForest +0.0092, LightGBM +0.0047, CatBoost +0.0106, GRU -0.0098, LSTM +0.0917
Returns $\hat{y} = z \cdot \sigma + \mu$ (σ = 0.00976, μ = 0.00054)	6 floats	Ridge +0.084 %, RandomForest +0.063 %, LightGBM +0.059 %, CatBoost +0.064 %, GRU +0.045 %, LSTM +0.144 %
Combined $\hat{y}$	float	+0.00076 (+0.076 %)
Predicted close	float	$4,681.18 = 4,677.60 · exp(+0.00076)
68 % band	two floats	$4,613.69 to $4,749.65 ($\sigma_t$ = 1.598 %, k = 0.909, half-width 1.452 %)
Revealed (demo only)	float	actual close on 2026-05-13 = $4,697.70; error -0.35 % (-16.52 USD); direction HIT; inside band

The rows hidden from the models in this demo run: 84 later days (`rows_hidden`); 2,072 days were used (`rows_used`).

> **What the live dashboard displayed on 2026-10-09 (Gold, press of Run prediction at about 17:42, live data to 2026-10-08)**
> Close on 2026-10-08 **$4,157.00** → predicted close for 2026-10-09 **$4,162.97**, **UP ▲ +0.14 %** (+5.97 USD); 68 % band **$4,117.66 to $4,208.78** (±1.09 % = 0.91 × σ, σ = 1.20 %); all six models said UP; the live scorecard (last 30 days, 6 models) finished in 1.3 s. These numbers depend on the day the page is opened and are shown only to illustrate the format; they are not a result of the evaluation.

22.4 The same request through the API

`src/api/app.py` (FastAPI route)

```
82  @app.get("/predict/{asset}", response_model=PredictionResponse)
83  def predict_asset(asset: str, model: Optional[str] = Query(None, description="One trained model name;
        default = the Combined forecast")):
84      name = _resolve(asset)
85      if model and model not in available_models(name) + [COMBINED]:
86          raise HTTPException(status_code=404, detail=f"Model '{model}' not available for {name}.
                Available: {available_models(name) + [COMBINED]}")
87      try:
88          return PredictionResponse(**predict_next_day(name, model or COMBINED))
89      except FileNotFoundError as e:
90          raise HTTPException(status_code=503, detail=f"Model or data artefacts missing: {e}")
91      except Exception as e:
92          logger.exception(f"Error predicting {name}")
93          raise HTTPException(status_code=500, detail=str(e))
```

1. `GET /predict/gold` → `predict_asset("gold", model=None)`.
2. `_resolve("gold")` → `"Gold"` (404 if unknown). If `?model=` is given it must be in `available_models("Gold") + ["Combined"]`, otherwise 404.
3. `predict_next_day("Gold", "Combined")` runs steps 4 to 9 above.
4. `PredictionResponse(**result)` validates types; FastAPI serialises it to JSON. `FileNotFoundError` → HTTP 503; other exceptions → 500.

22.5 What differs in "Predict a Day"

```
src/inference/prediction.py  (predict_for_date: choose the data, cut the future)
229     frozen = load_features(asset, live=False)            # the frozen dataset the test results were
            computed on
230     ts = pd.Timestamp(as_of)
231     in_frozen = ts in frozen.index and frozen.index.get_loc(ts) + 1 < len(frozen)
232     feats = frozen if in_frozen else load_features(asset, live=True)
233     t_start = time.perf_counter()
234     Xseq, Xt, last_close, as_of_ts, cols = _window(asset, feats, as_of)
235     if as_of_ts.normalize() != ts.normalize():
```

`predict_for_date` first decides whether the chosen day lies in the frozen test period (then it uses the frozen file so the result reproduces `results/predictions`) or is newer (then the live file). `_window(asset, feats, as_of)` cuts `feats.loc[:as_of]`, which is the **hiding of the future**; after the models have run, `pos + 1 < len(feats)` decides whether an actual close exists to reveal, and the error, hit and in-band flags are computed. Everything before the reveal is identical to Run prediction.

Chapter 23: Likely Code-Related Viva Questions

Questions an examiner can ask **while looking at the VS Code project**, each with a short answer to say first and a fuller explanation. File names, function names and line numbers were checked against the repository on 2026-10-09.

Q1. Which file is the application entry point?

Short answer: `app/streamlit_app.py` for the user (`streamlit run app/streamlit_app.py` or `make serve`).

Fuller explanation: There is no single main. Each stage has its own script (`make` targets), the dashboard is `app/streamlit_app.py`, the JSON API is `src/api/app.py` (`uvicorn src.api.app:app`), and `scripts/retrain.py` runs the whole offline pipeline. Streamlit executes the dashboard file top to bottom on every interaction.

Q2. Where is the dataset loaded?

Short answer: Raw CSVs: `DataCleaner.load_data` in `src/data/preprocessing.py`. Model-ready arrays: `build_dataset` in the same file.

Fuller explanation: `DataCleaner.load_data` reads `data/raw/<asset>_data.csv`. After preprocessing, **every later stage** (tuning, training, stacking, evaluation) gets its data from `build_dataset(asset)`, which reads `data/processed/<prefix>_{train,val,test}_scaled.csv`, `<prefix>_features.csv` and the scaler. The dashboard reads `<prefix>_live_features.csv` (or the frozen file) through `load_features`.

Q3. Where is the prediction target created?

Short answer: `_task_target` and `build_dataset` in `src/data/preprocessing.py` (lines 305 and 321).

Fuller explanation: For the served task the target at row t is `log_return.shift(-1)`, i.e. $\ln(P_{t+1}/P_t)$. It is then standardised with the **training** mean and standard deviation (`y_tr_real.mean()`, `.std()`). `create_sequences` is an older, test-covered helper that does the same alignment for a plain DataFrame.

Q4. Where are the features calculated?

Short answer: `DataCleaner.clean_data` and its `add_*` methods in `src/data/preprocessing.py`.

Fuller explanation: `clean_data` (line 199) calls `add_moving_averages`, `add_rsi`, `add_macd`, `add_advanced_ta` (Bollinger, ATR, ADX, ROC via `ta`), `add_relative_features`, `add_lag_returns`, `add_rolling_volatility`, `add_calendar_features` / `add_volume_change` (Bitcoin), `lag_post_settlement_features` (metals) and `add_external_features`.

Q5. Where are the models trained?

Short answer: `train_asset` in `src/training/train_models.py`; the models themselves are the classes in `src/models/registry.py`.

Fuller explanation: `train_asset` calls `make_model(name, params).fit(...)` twice per model: phase A on train with validation monitoring, phase B on train+validation. Hyper-parameters come from `config.get_params`, which reads `results/tuning/best_params.json` written by `tune_models.py`.

Q6. How is the test set protected from leakage?

Short answer: Split by target date, scaler fitted on train only, tuning on train+validation folds only, and only `backtesting.py` ever reads the test arrays.

Fuller explanation: Details: `build_dataset` masks (test = target after `VAL_END`); `normalize_and_split` fits `MinMaxScaler` on train rows; `tune_models.py` and `train_models.py` never touch `X_test`; the Combined forecast has no fitted weights; model selection uses walk-forward results. **Honest caveat:** the test window was inspected in several development rounds, which is why we also report six years of pooled walk-forward results.

Q7. Where are model artefacts saved?

Short answer: `data/models/`, written by `BaseModel.save`, `CatBoostModel.save` and `_KerasSequenceModel.save` in `registry.py`; the scaler by `DataCleaner.save_data`.

Fuller explanation: File names follow `artefact_stem`: `<prefix>_<model>.pkl|.cbm|.keras` (+ a `.json` sidecar for CatBoost and Keras). `model_status.json` is written by `backtesting.main`. Only `model_status.json` is tracked by git.

Q8. Where is the final prediction calculated?

Short answer: `_predict_from_window` (the mean of six) and `predict_next_day` (price and band) in `src/inference/prediction.py`.

Fuller explanation: `_predict_one` returns each model's log return; `_predict_from_window` averages them; `predict_next_day` computes `last_close * np.exp(r_hat)`, the direction, and the band through `_band`.

Q9. How does the dashboard call the backend?

Short answer: It imports `predict_next_day`, `predict_for_date`, `recent_track_record`, `update_live_data` from `src` and calls them as normal Python functions; there is no HTTP between them.

Fuller explanation: The dashboard process runs the models itself. The FastAPI service is a separate, optional front end that calls the same `predict_next_day`.

Q10. How would you debug a failed prediction?

Short answer: Read the red "Prediction failed: ..." text, then reproduce with `make predict` or in a Python shell: `python -c "from src.inference.prediction import predict_next_day; print(predict_next_day('Gold'))"`.

Fuller explanation: Most errors name the fix: missing `model_status.json` or band `k` → `make evaluate`; no processed data → `make preprocess`; feature mismatch → files and scaler disagree; NaN in the newest row → corrupt download (delete the live file or press Sync). Then step through `_window` (shapes), `_predict_one` (finite output), `_band`. `tests/test_models_and_inference.py` shows the expected behaviour for each failure.

Q11. What happens if a model file is missing?

Short answer: If all are missing, `FileNotFoundError("No trained models found ... Run make train")`; if only some are missing, Combined is built from the rest.

Fuller explanation: `combined_members(asset)` keeps only models with an artefact (`artefact_exists`). The dashboard caption shows the number of models and the API returns 503 for `FileNotFoundError`. The unit test asserts that all six members are present for the three assets on this machine.

Q12. How would you add a feature safely?

Short answer: Add a method in `DataCleaner`, check its close-time availability, add a no-look-ahead test, re-run preprocess, tune, train, evaluate.

Fuller explanation: Steps: (1) compute it only from rows ≤ t (`rolling`, `ewm`, `shift(+k)`); for the metals decide whether it uses information after 13:30 ET and lag it (`POST_SETTLEMENT_FEATURES`) or align it with `_align_external`; (2) call it in `clean_data`; (3) make it stationary (a ratio or return) so it is not in `LEVEL_COLUMNS`; (4) extend `test_no_lookahead_in_any_feature`; (5) run `make preprocess` (the scaler and column list change, so every model must be retrained: `make tune train stack evaluate`); (6) compare against the previous results with the walk-forward folds, not the test set.

Q13. What does `build_dataset` return, and why one loader?

Short answer: A dictionary of arrays for train / val / test plus the helper functions `fwd` and `inv`, so that every stage uses identical data.

Fuller explanation: Keys: `X_*` (n, 30, F), `Xt_*` (n, F), `y_*` standardised, `y_real_*`, `dates_*`, `target_dates_*`, `prev_*`, `true_*`, `naive_*`, `columns`, `scaler`, `inv`, `fwd`, `target_mean`, `target_std`, `periods_per_year`. A single loader prevents the classic bug where training and evaluation prepare data differently.

Q14. Why is `Xt` the last row of the window?

Short answer: Tabular models take today's features only; the lagged information is already inside lag and rolling features.

Fuller explanation: `out[f"Xt_{split}"] = X[m][:, -1, :]`. GRU and LSTM use the whole (30, F) window instead.

Q15. Where is the scaler fitted and how is it reused at inference?

Short answer: Fitted in `DataCleaner.normalize_and_split` on training rows, saved as `<prefix>_scaler.pkl`, loaded by `build_dataset` and `prediction._window`.

Fuller explanation: `_window` also checks the feature count, compares the **names** with `scaler.feature_names_in_` and re-orders the columns, so a feature file with a different column order still gives the same forecast (test `test_prediction_does_not_depend_on_feature_file_column_order`).

Q16. Why train twice (phase A and phase B)?

Short answer: Phase A finds the stopping point using validation; phase B refits on train+validation with that point fixed.

Fuller explanation: Using validation as the early-stopping monitor and then reporting it as "unseen" would be optimistic. Phase A gives the honest train/validation metrics, phase B the final model with the most pre-test data. `fit(..., final=False/True)` implements both.

Q17. How are hyper-parameters chosen and where are they stored?

Short answer: Grid search scored by 4-fold walk-forward CV in `tune_models.py`; stored in `results/tuning/best_params.json`.

Fuller explanation: `config.get_params(model, asset)` returns the default parameters overlaid by the tuned ones. For example Ridge `alpha` = 100 for all assets and Random Forest `max_depth` = 3.

Q18. What is `model_status.json` and why does inference need it?

Short answer: The evaluation summary per asset; inference needs it for the target mean/std (to convert model output to a return) and for the band multiplier `k`.

Fuller explanation: `_target_stats` and `_band_k` raise `FileNotFoundError` if the values are absent, because silently using a wrong scale or the old band would be worse than failing. The dashboard also reads the test metrics and the best single model from it.

Q19. Where is the uncertainty band calculated and where does `k` come from?

Short answer: `_band` in `prediction.py`; `k` is calibrated in `band_rows` in `backtesting.py` and stored in `model_status.json`.

Fuller explanation: Band = $P_t \cdot \exp(\hat{y} \pm k \cdot \sigma_t)$ with σ_t = the `ewma_vol` feature on the forecast day; k is the 68.27 % quantile of $|\text{error}|/\sigma_t$ over training-split walk-forward errors (0.825, 0.909, 0.870).

Q20. How does `predict_for_date` differ from `predict_next_day`?

Short answer: It cuts the data at a chosen day (hiding the future), predicts the following day and then reveals the actual close.

Fuller explanation: Stored test days use the frozen dataset (so results reproduce `results/predictions`), newer days use the live file; it also returns timing and provenance (`computed_at`, `rows_hidden`) for the LIVE badge.

Q21. How is look-ahead prevented when you predict a past day?

Short answer: `feats.loc[:as_of]` in `_window`: rows after the chosen day are not passed to the scaler or the models.

Fuller explanation: The test `test_predict_for_date_uses_only_past_data_and_reveals_actual` multiplies every future row by 1.37 and checks that the prediction does not change while the revealed actual does.

Q22. What is the complete-bar rule and where is it?

Short answer: `drop_incomplete_bars` in `src/data/market_calendar.py`, used by `fetch_asset` and the macro downloads.

Fuller explanation: Yahoo returns today's running bar as if it were a close; the function removes rows dated on or after `complete_bar_cutoff`: today's UTC date for Bitcoin, today (before 17:15 ET) or tomorrow (after) for futures and US series.

Q23. Why do Gold and Silver use previous-day macro values?

Short answer: Their close is the 13:30 ET settlement, before the US markets close, so same-day macro returns would not be known yet.

Fuller explanation: `config.EXTERNAL_SAME_DAY = {"crypto": True, "commodity": False}` and `DataCleaner._align_external`. Without it Silver showed a fake 62 % directional accuracy.

Q24. How is the Combined forecast computed in evaluation and in inference?

Short answer: The same way: the equal-weight mean of the six members' predicted returns.

Fuller explanation: Evaluation: `test_predictions` and `combined_cv_row` in `backtesting.py`; inference: `_predict_from_window` in `prediction.py`. The test `test_combined_test_row_is_the_mean_of_its_members` guards equality.

Q25. Which tests would you show, and what do they prove?

Short answer: `test_no_lookahead_in_any_feature`, `test_split_is_chronological_and_disjoint`, `test_predict_for_date_uses_only_past_data_and_reveals_actual`.

Fuller explanation: They prove respectively: changing the last bar never changes earlier features; no training target reaches validation or test; changing every future row does not change a forecast. `venv/bin/python -m pytest tests/ -q` gives 47 passed.

Q26. What happens if Yahoo Finance is down?

Short answer: Downloads return an empty frame or `False`; the existing files stay untouched and the dashboard says it is using the stored dataset.

Fuller explanation: `fetch_asset` catches exceptions; `update_live_data` returns False without raising (`test_live_sync_survives_network_failure`). No retry/back-off is implemented, and the frozen evaluation data never depends on the network.

Q27. Why is `shuffle=False` in Keras training?

Short answer: To keep the windows in time order.

Fuller explanation: The code does not state a reason. Shuffling already-built windows would not by itself leak future data, but keeping the original order makes training deterministic and consistent with time-series practice; the project avoids any randomisation of order (and has no `train_test_split`).

Q28. How is `TimeSeriesSplit` used and with what sizes?

Short answer: `walk_forward_splits` uses `TimeSeriesSplit(n_splits=4)` on train+validation; blocks are 582 (Bitcoin) or 396 (metals) samples.

Fuller explanation: Fold 1 trains on the first 582 samples and validates on the next 582; fold 4 trains on 2,328 and validates on the last 582: 2,328 out-of-fold days in total.

Q29. How are the inputs of the GRU and LSTM shaped?

Short answer: (batch, 30 days, F features); F = 29 (Bitcoin, Silver) or 28 (Gold).

Fuller explanation: `_make_windows(values, seq_len)` builds the sliding windows over the scaled rows; `Input(shape=(30, F))` in `build_gru_model`.

Q30. How would you add a new model, for example XGBoost?

Short answer: Write a subclass of `BaseModel` in `registry.py`, register it in `_REGISTRY`, add a grid in `tune_models.py` and re-run tune, train, evaluate.

Fuller explanation: Implement `fit` with the `final` flag semantics, `predict`, `feature_importance`, and `save`/`load`; add the name to `TABULAR`; because `BASE_MODELS` is derived from `TABULAR + RECURRENT`, it would automatically join the Combined forecast, so decide deliberately (the audit rule: only keep it if it beats the served forecast on walk-forward folds for every asset).

Q31. How do you retrain everything?

Short answer: `python scripts/retrain.py` (add `--no-fetch` to keep the frozen data, `--no-tune` to reuse tuned parameters) or `make pipeline`.

Fuller explanation: Each stage runs as a subprocess; the first failure stops the run and logs the last 3,000 characters of stderr to `logs/retrain.log`.

Q32. Where is directional accuracy defined, and why are flat days excluded?

Short answer: `directional_accuracy` in `src/utils/metrics.py`.

Fuller explanation: A day with exactly zero return has no direction. The function counts non-flat days, hits where `sign(true) == sign(pred)`, and a one-sided binomial p-value against 50 %. The share of UP calls is reported next to it because always saying UP scores the share of up days.

Q33. Why is MAPE not accuracy?

Short answer: "Tomorrow = today" has the same MAPE (1.52 / 1.29 / 2.49 %).

Fuller explanation: MAPE measures the typical % price error, dominated by the daily volatility itself. Skill is the **difference from the baseline**, tested with Diebold-Mariano.

Q34. Why does `config.py` create folders when imported?

Short answer: Convenience: `os.makedirs(..., exist_ok=True)` guarantees the output folders exist.

Fuller explanation: It is a side effect that surprises people (importing config in a test creates `results/tuning/`), but it prevents "folder not found" errors in a fresh clone.

Q35. What happens if you change `TRAIN_END`?

Short answer: All splits, scalers, tuned parameters, models and results become invalid and must be regenerated.

Fuller explanation: It is used by `normalize_and_split`, `build_dataset`, `eda.py`, plots, the dashboard shading and the metadata in `model_status.json`. Re-run `make preprocess tune train stack evaluate`.

Q36. How does the dashboard avoid recomputing the six models on every click?

Short answer: It stores the result in `st.session_state["run_result"]` keyed by asset and only recomputes if the asset changes or Run prediction is pressed again.

Fuller explanation: The live scorecard is cached the same way under `run_score` with key (asset, as_of date, N days). Data and CSVs are cached for 300 s with `@st.cache_data`.

Q37. Why does the page stay on the same tab after a button click?

Short answer: `st.session_state["open_tab"]` is set in each button's `on_click` and passed as the `default` of `st.tabs`.

Fuller explanation: Streamlit re-runs the script on every click; without this the page would jump back to the first tab.

Q38. Why is LightGBM `subsample` inert?

Short answer: Row bagging in LightGBM needs `subsample_freq` > 0, which is left at 0.

Fuller explanation: The 0.8 in the grid therefore has no effect; the code comment in `registry.py` documents it. Changing it would change the served models without evidence of benefit, so it was documented instead of altered.

Q39. How do you start the API and what does it return?

Short answer: `make api` (`uvicorn src.api.app:app --reload`), then `GET /health`, `GET /models`, `GET /predict/gold`.

Fuller explanation: `/predict/{asset}` returns the `PredictionResponse` fields: asset, dates, price, return %, direction, members, band and calibration, test metrics and the disclaimer. Unknown assets or models give 404; missing artefacts 503.

Q40. What cannot an examiner find after a fresh `git clone`?

Short answer: The data, processed features and trained models: they are git-ignored.

Fuller explanation: `.gitignore` excludes `data/raw*`, `data/processed`, `data/models/*` except `model_status.json`. Run `make collect-data preprocess train stack evaluate`. Results will differ marginally because Yahoo revises history.

Q41. How does the dashboard avoid showing stale or hard-coded numbers?

Short answer: Every number is read from `model_status.json`, `results/*.csv`, the feature files or computed live by `prediction.py`.

Fuller explanation: Even the sentences (overview strip, honest reading, methodology) are f-strings built from those files. Exceptions: some figure images in the diagnostics expander are older PNGs (Appendix C).

Q42. Where is UP / DOWN decided?

Short answer: `"UP" if r_hat > 0 else "DOWN"` in `predict_next_day`, `predict_for_date` and `_individual`.

Fuller explanation: It is just the sign of the (combined) predicted return, so a model that predicts a tiny positive average return says UP almost every day: that is why the metals' direction is drift.

Q43. What is `recent_track_record` and why is it labelled LIVE?

Short answer: It re-runs every model now on each of the last N days and scores them against the actual next close.

Fuller explanation: Nothing is read from stored results, and days newer than the frozen test period are counted separately (`n_after_frozen_test`), so the examiner sees scoring that did not exist before the button was pressed.

Q44. Why do Naive-Mean and ARIMA exist?

Short answer: They are baselines: the historical mean separates drift from timing, ARIMA is the classic time-series benchmark.

Fuller explanation: `NaiveMean` and `ARIMAModel` in `registry.py`; ARIMA's order is chosen by AIC and forecasts are produced walk-forward.

Q45. What would you change first if you had one more month?

Short answer: Make volatility / range forecasting the main target and test a clean forward window.

Fuller explanation: Because E4 shows volatility is partly predictable (EWMA beats persistence by 4 to 8 %), and the current test window was inspected during development (Chapter 13.3).

45 questions. FINAL_AUDIT.md contains 110 further questions on the project as a whole (basic, machine learning, dataset, models, prediction, validation, Streamlit, difficult).

Glossary

Term	Meaning
ADF test (Augmented Dickey-Fuller)	Statistical test of whether a series has a unit root (wanders). Prices fail it ($p \approx 1$), returns pass ($p = 0$).
ARIMA(p, d, q)	Linear time-series model combining autoregression, differencing and moving-average terms; our baseline uses $d = 0$ on returns.
ATR (Average True Range)	Average of the daily trading range including gaps; we use ATR(14) / price.
Band (68 %)	Our price interval $P \cdot \exp(\hat{y} \pm k \cdot \sigma_t)$ intended to contain the next close about 68 % of the time.
Baseline / naive forecast	The simplest forecast to beat. Here: tomorrow's return = 0 ("tomorrow = today").
Bollinger Bands	A 20-day moving average ± 2 standard deviations; we use %B (position) and width.
CatBoost	Gradient boosting library using symmetric trees and ordered boosting.
Close-time rule	Our rule that a feature may only use information available when the asset's daily close is fixed (same-day macro for Bitcoin, previous-day for the metals).
Combined forecast	The equal-weight mean of the six models' predicted log returns; the served forecast.
Complete-bar rule	Drop the still-running bar of the current day from downloads.
Data leakage	Using information in training or inputs that would not be available at prediction time, which inflates results.
Diebold-Mariano (DM) test	Test of whether two forecasts have different squared errors; $p < 0.05$ would mean a real difference.
Directional accuracy (DA)	Share of non-flat days on which the predicted sign equals the actual sign.
Drift	The average return of the training period; "always UP" for an asset that rose.
Dropout	Neural-network regularisation: randomly ignore a fraction of units during training.
Early stopping	Stop training when validation error stops improving.
EMA (exponential moving average)	Moving average that weights recent values more.
Efficient-market hypothesis	Theory that prices already reflect available information, making next-day returns unpredictable.
Embargo / purge	Removing samples whose targets overlap another split's targets.
Ensemble	Combination of several models' predictions.
Epoch	One pass of a neural network over the training data.
EWMA volatility	Exponentially weighted volatility, $\sigma^2_t = 0.94 \cdot \sigma^2_{t-1} + 0.06 \cdot r^2_t$; the σ_t of the band.
Expanding window	Walk-forward scheme where each fold trains on all earlier data.
Fat tails (excess kurtosis)	Extreme moves are more frequent than a normal distribution predicts; Bitcoin's excess kurtosis is 15.2.
Feature	One input number describing day t.
Frozen data	The raw files behind every reported result (`data/raw/`, last bar 2026-09-12 / 2026-09-11).
GRU	Gated Recurrent Unit: a recurrent neural network with update and reset gates.
HAR realised volatility	Daily, weekly and monthly realised volatility (`rv_1d`, `rv_5d`, `rv_22d`).
Hyper-parameter	A setting chosen before training (depth, learning rate, α…).
LightGBM	Gradient-boosted decision trees with leaf-wise growth.
Log return	$\ln(P_{t+1}/P_t)$; close to the percentage change.

Term	Meaning
LSTM	Long Short-Term Memory network with a cell state and three gates.
MACD	$EMA_{12} - EMA_{26}$, with a 9-day EMA signal line.
MAE / RMSE / MAPE	Mean absolute error / root mean squared error / mean absolute percentage error.
MinMaxScaler	Scales each feature to [0, 1] using the training minimum and maximum.
Out-of-fold (OOF) prediction	A prediction for a validation block made by a model that did not train on that block.
Overfitting	Fitting noise: good in-sample, poor out-of-sample.
Pooled walk-forward	All out-of-fold predictions of all folds evaluated together (2,328 or 1,584 days).
QLIKE	Loss for variance forecasts, mean(ratio − ln(ratio) − 1); used for volatility experiments and band scoring.
R^2	Share of variance explained; 0 = no better than predicting the mean.
Random walk	Process whose next change is unpredictable; its best forecast is the current value.
Random Forest	Average of many decision trees grown on bootstrap samples.
Regularisation	Techniques that limit model complexity (penalties, shallow trees, dropout, early stopping).
Ridge regression	Linear regression with an L2 penalty on the weights.
RSI	Relative Strength Index (0 to 100) of recent gains versus losses.
Stacking	Learning weights for combining models from their out-of-fold predictions (our experiment).
Stationary	Statistical properties (mean, variance) do not drift over time.
Streamlit	Python framework used for the dashboard.
Target	The quantity to predict: the next-day log return.
Train / validation / test	Data to fit, to choose settings, and to grade once at the end.
Two-phase training	Phase A: fit on train and monitor validation; phase B: refit on train+validation with the stopping point fixed.
UP calls	Share of days the model predicted a rise.
Volatility clustering	Large moves tend to follow large moves; the reason volatility is partly predictable.
Walk-forward validation	Repeatedly train on the past and validate on the next block, moving forward in time.

Appendix A: Quick reference of the key numbers

Item	Bitcoin	Gold	Silver
Ticker	BTC-USD	GC=F	SI=F
Raw rows / first / last day	3,177 · 2018-01-01 · 2026-09-12	2,186 · 2018-01-02 · 2026-09-11	2,186 · 2018-01-02 · 2026-09-11
Processed rows	3,147	2,156	2,156
Model features	29	28	29
Samples train / val / test	2750 / 160 / 207	1874 / 109 / 143	1874 / 109 / 143
Test targets	2026-02-18 to 2026-09-12	2026-02-18 to 2026-09-11	2026-02-18 to 2026-09-11
Target μ / σ (train)	0.000852 / 0.03360	0.000541 / 0.00976	0.000496 / 0.01920
Best single model by walk-forward	CatBoost	CatBoost	GRU
Combined test RMSE vs naive (DM p)	-0.34 % (0.49)	-0.04 % (0.97)	-0.29 % (0.46)
Combined direction accuracy (p)	46.9 % (0.83)	50.3 % (0.50)	53.8 % (0.20)
Band k / test coverage	0.825 / 65.7 %	0.909 / 69.2 %	0.870 / 69.2 %
Pooled out-of-fold days	2,328	1,584	1,584

Appendix B: Verification record (what was checked, and how)

Everything in this guide was checked against the repository at commit `83323b4` on 2026-10-09. The checks below were **executed**, not assumed.

Check	Result
Repository state	Working tree clean at the start (`git status` empty); no tracked file was modified while preparing this guide. The only addition is this PDF as an untracked file in `docs/`; the generator scripts were kept outside the repository.
Complete source read	All 33 Python files, `Makefile`, `config.py`, `requirements*.txt`, CI workflow, README, CHANGELOG, FINAL_AUDIT.md and the notebooks' cells were read.
Test suite	`venv/bin/python -m pytest tests/ -q`: **47 passed** in 70 s (2026-10-09).
Dataset statistics	Row counts, date ranges, columns, dtypes, nulls, duplicates, calendar gaps, price ranges and zero-volume days were recomputed from `data/raw/*.csv` and `data/processed/*` (Chapter 4).
Split sizes and dates	Computed by running `build_dataset` for every asset and `walk_forward_splits` for every fold (Chapter 9).
Results	All tables read from `results/*.csv`. The Combined test metrics (RMSE, MAE, direction accuracy) were **re-derived from the stored per-day predictions** with the project's own `evaluate_forecast` and matched the stored values to 1e-15.
Inference reproducibility	`predict_for_date` was re-run for 3 stored test days per asset (first, middle, last): maximum difference to `results/predictions/*.csv` 2.7e-10 (Bitcoin), 4.8e-11 (Gold), 1.9e-10 (Silver).
Model internals	Parameter counts, epochs used and trees/iterations used were read from the saved artefacts (GRU 6593 / 6497 / 6593, LSTM 8481 / 8353 / 8481 parameters).
Dashboard	Started with `.claude/launch.json` (port 8501); Gold with `?asset=Gold&run=1`: Forecast tab (final card, band, live scorecard in 1.3 to 1.9 s), Predict a Day for 2026-10-07 (live run, 1.7 s), Model Performance and Methodology read in full; the Bitcoin page loaded. Screenshots in Appendix D.
API	Started with `.claude/launch.json` (port 8001): `/health` → healthy; `/models` lists six members + Stacked for each asset; `/predict/gold` returned the same price as the dashboard ($4,162.97, UP); `/predict/silver?model=GRU` returned a single-model forecast (`is_served_model` false); unknown asset and unknown model → HTTP 404.
Folder tree, function table, import graph	Generated by walking the file system and parsing the source with the `ast` module; every function in Chapter 18 was matched to an `ast` definition (115 of 115 found).
Code excerpts	Extracted programmatically from the real files with their real line numbers (never retyped).
Text claims vs repository	An automatic pass over this PDF's text checked every file path, `make` target and function name in code font against the repository: 1,406 distinct code-font tokens in the PDF (331 path-like, 1,075 identifiers) were extracted from the finished file and resolved against the repository (files and folders, function, class, method and variable names, CSV columns, model_status.json keys). Every token resolved except 50 that were reviewed by hand: line-wrap fragments, HTTP routes (/health, /predict/{asset}), shell commands, logs/retrain.log (created at run time) and library or file names mentioned only to say they are not used (requests, KFold, train_test_split); none was a wrong path or function name. The 33 distinct feature names documented in Chapter 6 are exactly the union of the three saved scalers' feature_names_in_.

Appendix C: Discrepancies and stale items found while checking

None of these changes a result. They are things an examiner could notice, listed so that you can fix or explain them.

#	Finding	Where	Suggested action
1	Notebook text says "Three years of daily OHLCV" although the data now start in 2018	`notebooks/01_data_exploration.ipynb`, cell "1. Raw data"	Edit the sentence
2	`requirements-lock.txt` (2026-07-25) does not list `catboost`, `fastapi`, `uvicorn` or `ta`; `requests` is in `requirements.txt` but never imported	`requirements-lock.txt`, `requirements.txt`	Regenerate the lock from the venv; drop `requests`
3	Figures in `results/figures/` (Sep 12 to 14) pre-date the final retrain of 2026-10-04; the dashboard's "Training diagnostics" expander shows `overfitting_gap.png` and the loss-curve PNGs. Their content still agrees with the current CSVs	`results/figures/*.png`, `app/streamlit_app.py`	`make figures`
4	The Predict a Day caption always ends "(frozen dataset)", even for NEW live days, where the LIVE badge correctly says "latest live download"	`app/streamlit_app.py` line 493	Use `r["data_source"]` in the caption
5	`config.MODEL_STATUS` is a module-level snapshot that nothing uses (code calls `load_model_status()` instead)	`config.py` line 136	Remove or keep as documentation
6	`docs/FYP_Technical_Report.pdf` and `docs/FYP_Complete_Study_Guide.pdf` still describe the pre-calibration band (FINAL_AUDIT.md §23); the study guide also says 43 tests	`docs/`	Replace the study guide with this document
7	`Makefile` uses `PY ?= python`; this Mac has no `python` command	`Makefile`	Use `PY=venv/bin/python make ...` or `PY ?= python3`
8	For Bitcoin, `sp500_return` and `vix_return` repeat Friday's value on Saturday and Sunday (verified on 100 % of Saturdays); not a leak, but a stale input	`DataCleaner._align_external`	Document; optional: use 0 on non-trading days
9	Empty folder `results/thumbnail/cutouts/` and `catboost_info/` logs (rewritten on every CatBoost fit)	repository root / `results/`	Delete; `make clean`
10	README says Python 3.9+, CI runs 3.11, local venv is 3.9.6; the audit noted CI was not re-run	README, `ci.yml`	Run CI once
11	FINAL_AUDIT.md Phase 1 mentions 45 tests; the suite now has 47	`FINAL_AUDIT.md`	Update the number
12	`fetch_yfinance_external` has no `try/except`; failures in `make collect-data` raise (inside the dashboard they are caught)	`src/data/external_data.py`	Wrap and retry

Appendix D: Dashboard screenshots (captured 2026-10-09, Gold, live data)

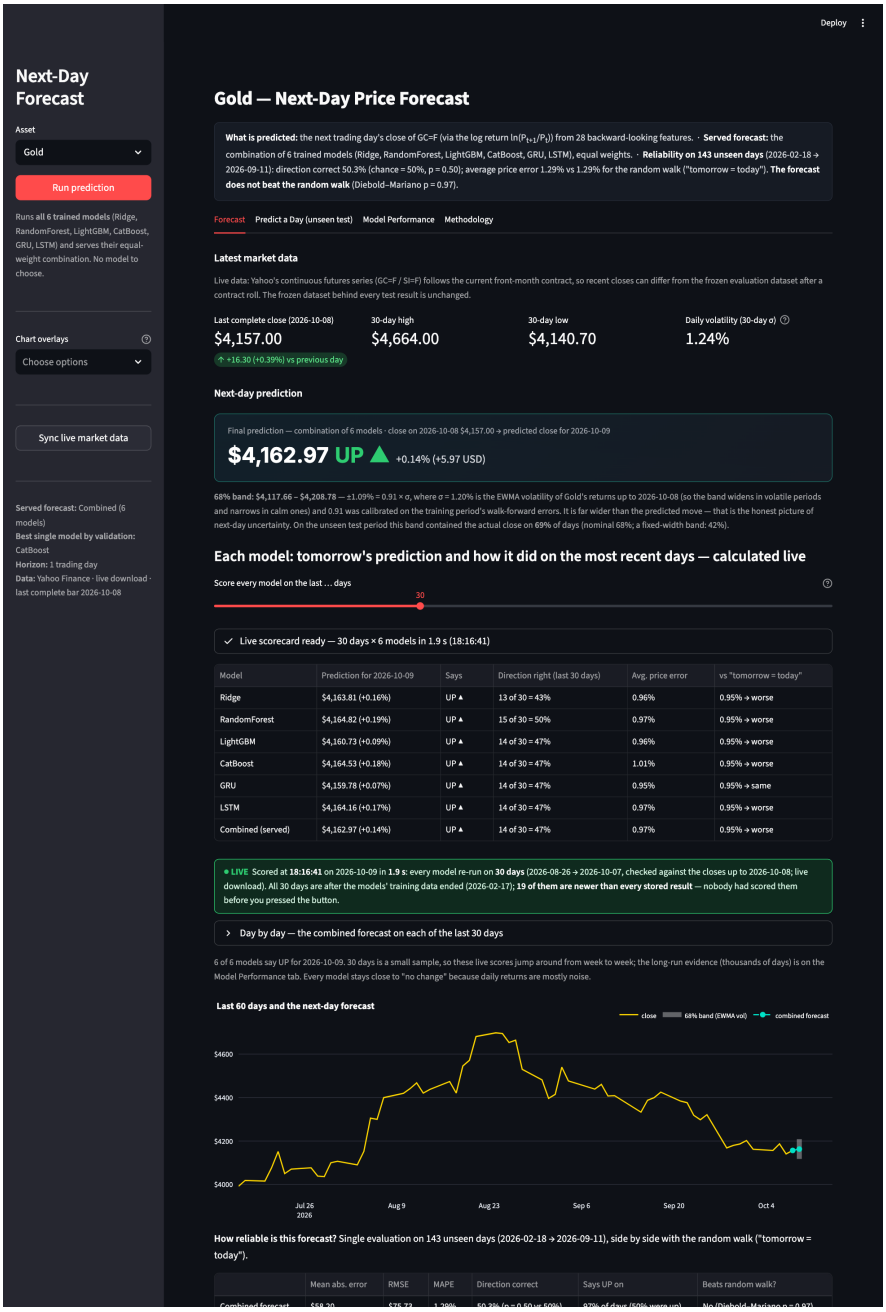


Figure D.1: Forecast tab after pressing Run prediction: overview strip, latest data, final card with UP and the 68 % band, the live scorecard of the six models, and the 60-day chart.

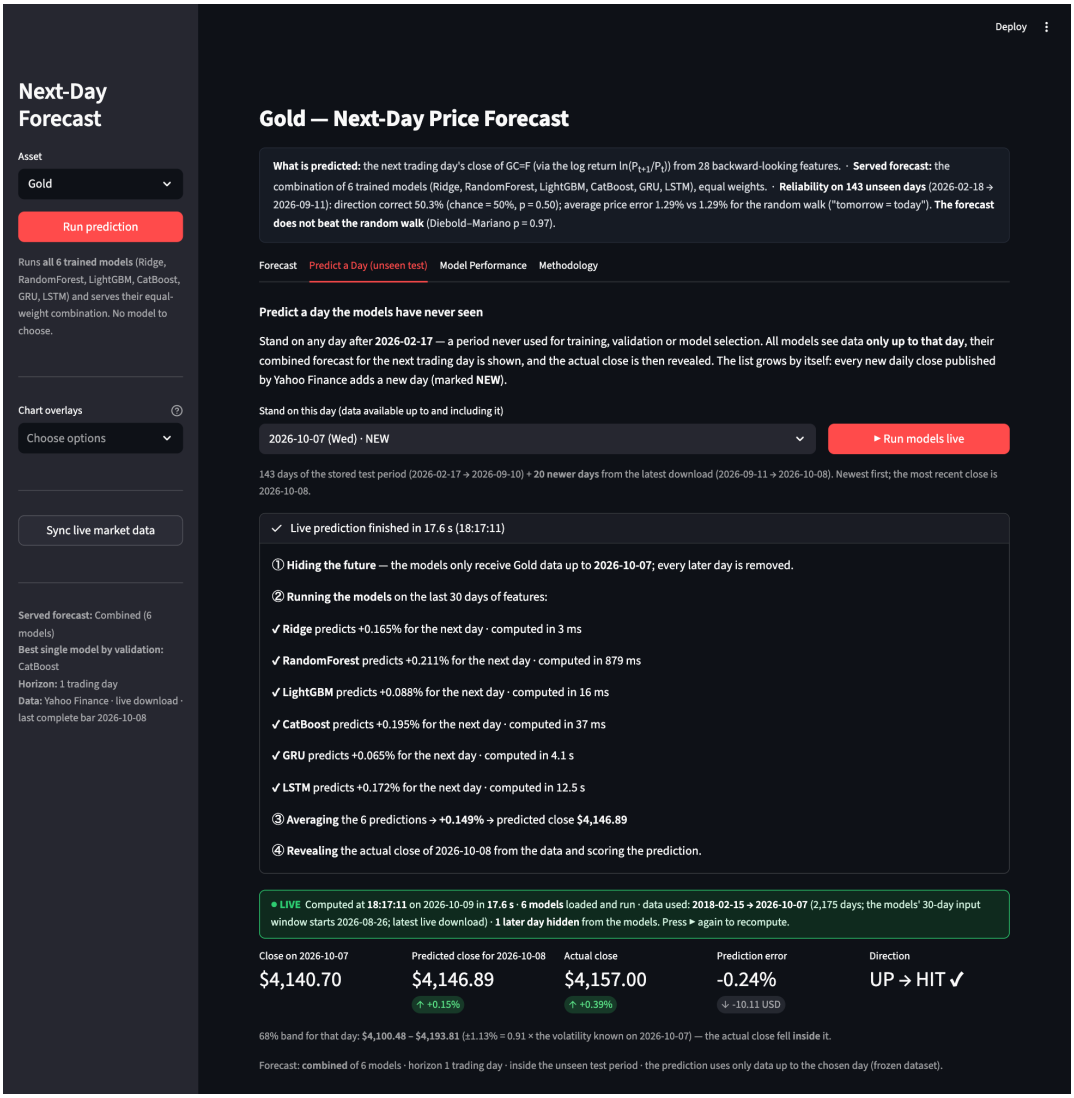


Figure D.2: Predict a Day for 2026-10-07 (a NEW live day): the live run box, LIVE badge, predicted versus actual close, error and HIT.



Figure D.3: Model Performance tab: unseen-test table with the "honest reading", the comparison bars and the walk-forward table.

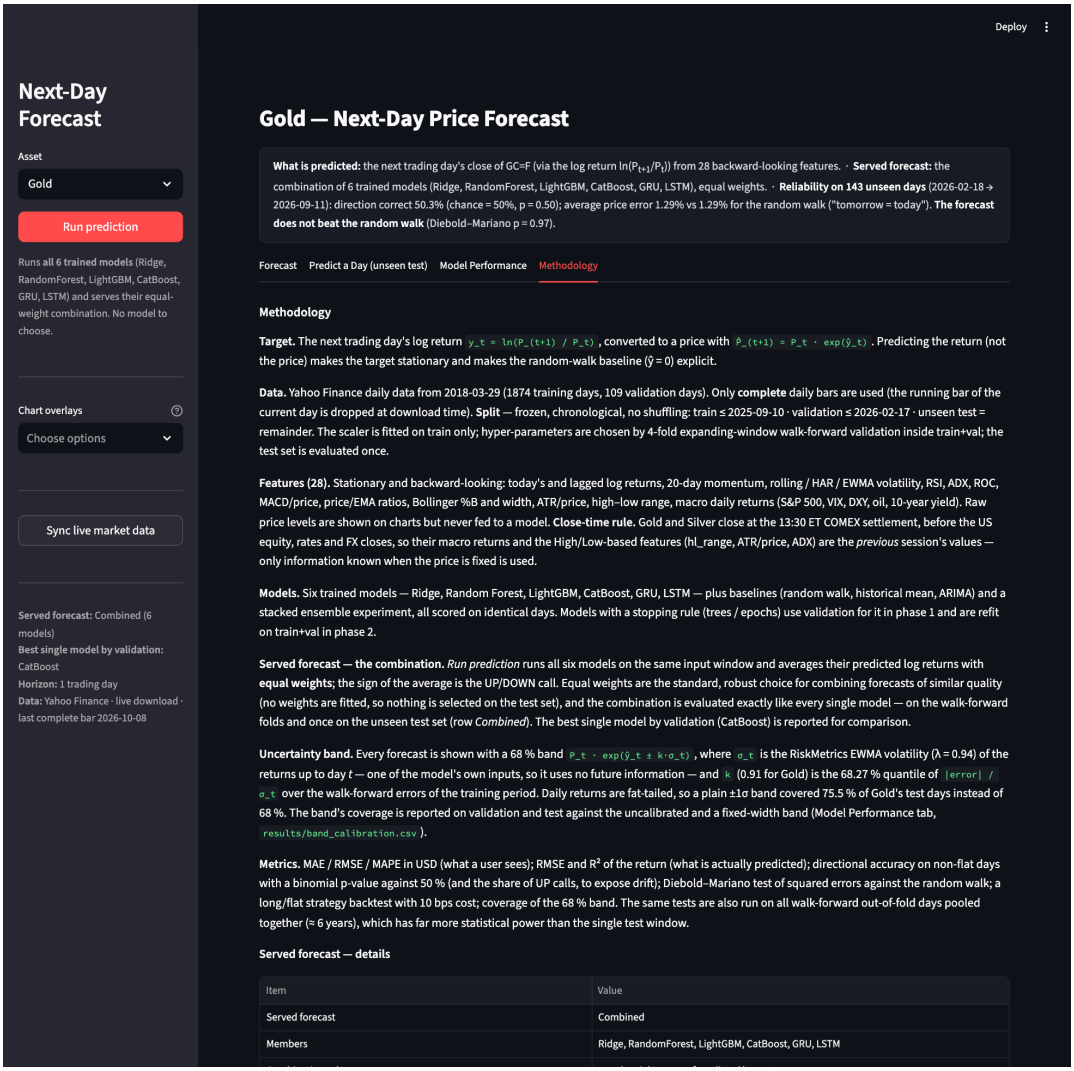


Figure D.4: Methodology tab.